



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Blockchain & CryptoCurrencies

APPUNTI LEZIONI

Prof. Catalano & Prof. Di Raimondo

Anno Accademico 2025 - 2026

Indice

1	Funzioni Hash	7
1.1	Hash Pointers e Struttura della Blockchain	8
1.1.1	Il Merkle Tree	9
1.2	Firma digitale	10
1.3	Basic Cryptocurrencies	11
1.3.1	GoofyCoin	11
1.3.2	ScroogeCoin	12
2	Come Bitcoin ottiene la Decentralizzazione	14
2.1	Le domande fondamentali del sistema	14
2.2	I tre pilastri della decentralizzazione	15
2.3	Il Consenso Distribuito	16
2.3.1	Incentivi e Proof of Work	18
2.4	Struttura delle Transazioni e Modello UTXO	22
3	Consenso di Nakamoto	24
3.1	Dimostrazione Matematica dell'attacco	26
3.1.1	La scelta della Difficoltà e il Network Delay	27
3.1.1.1	Il Discount Rate della potenza onesta	28
3.1.2	Vantaggi e Svantaggi	29
4	Anonimato e Privacy	31
4.1	Perché vogliamo l'anonimato?	32
4.1.1	Anonymous e-cash e Blind Signatures	33
4.1.2	Anonimato grazie a nuovi indirizzi	34
4.2	De-anonimizzazione a livello Network	36
4.2.1	Rete Tor	36
4.3	Mixing e Anonimato nelle Criptovalute	37
4.3.1	Online Wallet	38
4.3.2	Principi di Mixing	39
4.4	Coinjoin	40

5	Zero Knowledge Proofs	42
5.1	Prove Interattive	42
5.1.1	Isomorfismo tra Grafi	43
5.2	Diffie-Hellman Zero-Knowledge	45
5.3	Prove di Conoscenza	48
5.3.1	Formalizzazione di Conoscenza	48
5.3.2	ZeroKnowledge Proof Non Interattiva	49
6	Zerocoin e Zerocash	51
6.1	Zerocoin	51
6.1.1	Minting	52
6.1.2	Redeeming	52
6.1.3	Limiti Tecnici	53
6.1.4	Accumulatori crittografici	53
6.1.4.1	RSA	54
6.1.5	Implementazione degli Accumulatori	54
6.1.6	Integrazione in Zerocoin	55
6.1.7	Sfide di Integrazione e Problematiche	56
6.2	Zerocash	57
6.2.1	Il problema del Trusted Setup in Zerocash	57
6.3	Il Continuum dell'Anonimato nelle Criptovalute	58
7	Alternative Mining Puzzles	59
7.1	Requisiti dei Mining Puzzles	59
7.1.1	Il Problema dei Puzzle Sequenziali (Bad Puzzle)	61
7.1.2	Good Puzzle: Campionamento Pesato e Proprietà Progress-Free	61
7.2	Puzzle Resistenti agli ASIC	62
7.2.1	Script e i Memory-Hard Puzzles	63
7.2.2	Approcci Alternativi alla Resistenza agli ASIC	64
7.3	Proof-of-Useful Work	65
7.3.1	Primecoin e le Catene di Cunningham	66
7.3.2	Virtual Mining: Proof of Stake e Proof of Deposit	67
8	Bitcoin Scripting	68
8.1	Il linguaggio Bitcoin Script	70
8.1.1	Il processo di validazione: Sblocco e Blocco	70
8.1.2	Multi Signatures con Threshold	71
8.1.3	Proof-of-Burn	72
8.1.4	Timestamping	72
8.1.5	Real-life transaction con Escrow	73

8.1.6	Micro-pagamenti e Lightning Network	73
8.1.7	Freezing Funds	74
8.1.8	Pay-to-Script-Hash (P2SH)	75
9	La Blockchain	76
9.1	Il Blocco	76
9.1.1	Merkle Tree e Proof of Membership	76
9.1.2	Coinbase transaction	77
9.2	Il Network Bitcoin	78
9.2.1	Gossip Protocol	79
9.2.2	Propagazione dei Blocchi	80
9.2.3	Tipologie di Nodi	80
9.3	Limiti Strutturali di Bitcoin	82
9.3.1	Evoluzione del Protocollo: Hard Fork e Soft Fork	83
9.4	Conservare Bitcoin	83
9.4.1	Wallet	84
9.4.2	Strategie di Conservazione: Hot Storage e Cold Storage	85
9.5	Algoritmo di Firma Digitale ECDSA	88
9.5.1	Processo di Firma	89
9.5.2	Secret Sharing e Threshold Cryptography	90
9.5.3	Principi del Secret Sharing	90
9.5.4	Threshold Secret Sharing (Shamir's Scheme)	90
9.5.5	Threshold Signatures	91
9.5.6	Tecniche di Mining	93
9.5.6.1	Evoluzione dell'Hardware	93
9.5.7	Mining Pool	94
10	Smart Contract ed Ethereum	95
10.1	Limitazioni di Bitcoin Script	95
10.1.1	Il Caso Namecoin: Limiti della Logica di Bitcoin	96
10.2	La Piattaforma Ethereum	97
10.2.1	L'unità monetaria: Ether	98
10.2.2	Consenso: dal PoW al Proof of Stake	98
10.2.3	Tipologie di Account	98
10.2.4	Merkle Patricia Trie (MPT)	100
10.2.5	World State	101
10.2.6	Transazioni	102
10.2.7	Messaggi	103
10.2.8	Block Building	104
10.2.8.1	Struttura dei Blocchi	105
10.2.9	Il sistema di Gas	107

10.2.9.1	Meccanismo Legacy	108
10.2.9.2	Meccanismo EIP-1559	110

Introduzione

Una **moneta virtuale** è, fondamentalmente, uno strumento utilizzato per effettuare transazioni all'interno di un ecosistema digitale. Per comprenderne a pieno la natura e le sfide tecnologiche, è essenziale confrontarla prima con la moneta tradizionale (reale).

Le monete reali possiedono un valore che deriva storicamente da due paradigmi principali:

- **Valore Intrinseco:** Basato sulla scarsità e sulle proprietà del materiale stesso (come nel caso dell'oro). Generare una copia di un bene con valore intrinseco ha un costo equivalente al valore del bene stesso.
- **Valore Stabilito:** Il valore della banconota è imposto e garantito da un'autorità centrale, come una Banca Centrale. In questo caso, per mantenere il valore della valuta, la sua riproduzione **falsificazione** deve essere resa artificialmente complessa, affinché il costo, la difficoltà o il rischio di creare una copia eguagli o superi il valore nominale della moneta stessa.

Passando al mondo informatico, il paradigma cambia radicalmente: fare copie di un dato digitale è un'operazione estremamente semplice e a costo quasi zero. Di conseguenza, impedire la riproduzione non autorizzata e la spesa multipla della stessa moneta virtuale rappresenta una sfida tecnologica notevole, nota in informatica come il problema del *double-spending* (o doppia spesa).

Nei sistemi di pagamento digitale tradizionali, come le carte di credito o i bonifici online, la soluzione a questo problema non risiede nella natura del dato digitale in sé. Piuttosto, il sistema si affida a **TTP (Trusted Third Parties - Enti Terzi Fiduciari)**, ovvero banche, istituti finanziari o circuiti di pagamento. Questi enti centralizzati non si limitano semplicemente a "inviare dati digitali", ma fungono da garanti assoluti: gestiscono i registri contabili, verificano le disponibilità e si occupano di tutta l'infrastruttura amministrativa e burocratica (le classiche "scartoffie") che lega e converte il dato virtuale alla valuta reale corrispondente.

È in questo contesto che si inserisce la rivoluzione del **Bitcoin**. Ideato nel 2008 sotto lo pseudonimo di **Satoshi Nakamoto**, Bitcoin nasce con il preciso obiettivo di superare i limiti strutturali legati alla necessità di fiducia e alla centralizzazione dei sistemi tradizionali.

Le sue caratteristiche fondanti, che lo distinguono dalle precedenti monete virtuali, sono:

- **Assenza di TTP:** Non esiste un'autorità centrale, un server principale o un ente terzo fiduciario incaricato di validare le transazioni o emettere nuova moneta.
- **Sistema Distribuito:** La validazione delle transazioni e la tenuta dei registri avvengono in modo decentralizzato tramite un network *peer-to-peer*, supportato da una struttura dati pubblica chiamata Blockchain.
- **Pseudo-anonimato:** Gli utenti operano e scambiano valore tramite indirizzi crittografici, senza la stretta necessità di associare un'identità anagrafica diretta per poter partecipare al network.
- **Accessibilità ed Efficienza:** Fin dalla sua concezione, e in particolare nelle sue prime fasi storiche, il protocollo ha dimostrato la possibilità di effettuare transazioni globali, incensurabili e a costi estremamente ridotti rispetto ai circuiti interbancari internazionali.

Capitolo 1

Funzioni Hash

Una funzione Hash è una funzione che prende in input una stringa di lunghezza arbitraria e ritorna in un output una stringa di lunghezza fissata che, nel nostro caso, sarà una stringa di 256 bit. Questa funzione gode di tre proprietà fondamentali:

- **Collision-Resistant:** Ossia non è possibile per avversari computazionalmente limitati trovare due input x e y tali che il $H(x) = H(y)$.
- **Hiding:** Diciamo che la funzione Hash gode della proprietà di Hiding poiché è computazionalmente intrattabile trovare a partire da $H(x)$ il valore x . Inoltre, è anche impossibile a partire da $H(x|r)$ cosa non è x , evitando quindi attacchi di brute force.
- **Puzzle-Friendly:** Questa proprietà delle funzioni hash fa' sì che noi possiamo trasformarle in una sorta di puzzle o sfida. In pratica, quello che faremo sarà fissare dei dati k e aggiungeremo una randomness x tale che $H(k|x) = y$ con y che una certo pattern. Questo pattern è, in genere, un certo numero di zeri all'inizio alla fine dell'hash e, maggiori sono gli zeri che vogliamo, maggiore sarà la difficoltà. Esattamente come un puzzle, sarà "difficile" da risolvere ma facile da verificare.

Queste proprietà sono fondamentali per implementare uno schema di **Commitment**. Possiamo immaginare il commitment come l'equivalente digitale del mettere un messaggio in una busta sigillata e lasciarla in bella vista sul tavolo: tutti possono vedere la busta (il valore è "pubblicato" e quindi non può più essere modificato), ma nessuno può verificarne il contenuto finché non decidiamo di aprirla. Formalmente, questo meccanismo si articola in due fasi, gestite da altrettanti algoritmi:

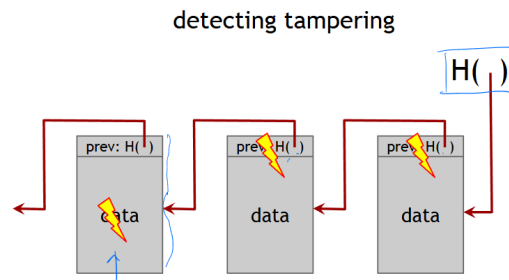
- **Fase di Commit:** L'algoritmo prende in input il messaggio m che vogliamo vincolare e un valore casuale r (necessario per garantire la proprietà di *hiding*). Restituisce in output il commitment com , ovvero la nostra "busta chiusa". Da questa stringa com è computazionalmente impossibile risalire alla coppia (m, r) .
- **Fase di Reveal e Verifica:** Quando si decide di svelare l'informazione, vengono resi pubblici m e r . A questo punto interviene l'algoritmo di *Verify*, che riceve in input i valori com , m e r . Esso calcola internamente $Commit(m, r)$ e restituisce 1 (Vero) se il risultato corrisponde esattamente al com pubblicato in precedenza, oppure 0 (Falso) altrimenti. Questo garantisce la certezza matematica che il messaggio non sia stato alterato dopo la sua pubblicazione.

1.1 Hash Pointers e Struttura della Blockchain

Da un punto di vista strutturale, una Blockchain può essere concettualmente assimilata a una *linked list* (lista concatenata). A differenza delle liste tradizionali, però, essa utilizza dei **Hash Pointers** (puntatori hash). Un hash pointer non si limita a indicare dove recuperare l'elemento precedente, ma contiene anche l'hash crittografico dei dati di quel blocco.¹

In una blockchain, ogni blocco contiene l'hash pointer al blocco **precedente**. Grazie a questo meccanismo, la catena diventa un registro a prova di manomissione *tamper-evident*: ogni blocco si fa garante dell'integrità di tutta la cronologia passata. Modificare anche un solo bit all'interno di un blocco altererebbe inevitabilmente il suo hash, rompendo la validità del puntatore nel blocco successivo e generando un effetto a cascata che invaliderebbe l'intera catena da quel punto in poi. L'unico modo per mascherare un attacco del genere sarebbe trovare una collisione nella funzione hash ma, come già discusso, per funzioni sicure ciò è computazionalmente intrattabile.

¹È importante notare che l'hash crittografico di per sé non è un indirizzo di memoria fisico o di rete; nei sistemi reali (come nei nodi Bitcoin), il client utilizza l'hash come chiave per cercare e recuperare il blocco corretto all'interno del proprio database locale.



1.1.1 Il Merkle Tree

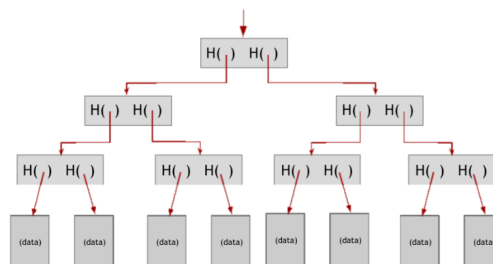
All'interno di ogni singolo blocco, le transazioni (i dati) non sono semplicemente elencate, ma sono organizzate utilizzando un'altra struttura basata sugli hash pointers: il **Merkle Tree** (Albero di Merkle).

Si tratta di un albero binario costruito con un approccio **bottom-up**:

- Le **foglie** dell'albero contengono gli hash delle singole transazioni.
- Ogni **nodo padre** viene generato concatenando gli hash dei suoi due figli e calcolandone a sua volta l'hash (es. $H(H(A) \parallel H(B))$).

Questo processo si ripete e si restringe fino a ottenere un unico hash radice, chiamato **Merkle Root**, che rappresenta crittograficamente tutte le transazioni e viene inserito nell'intestazione (header) del blocco.

Essendo costruito dal basso verso l'alto, l'albero risulta bilanciato. Questa struttura è estremamente efficiente poiché permette di verificare se una specifica transazione è inclusa in un blocco richiedendo solo il percorso dalla foglia alla radice (la cosiddetta *Merkle Proof*). Questo approccio abbate i costi computazionali e di memoria, consentendo verifiche con una complessità logaritmica ($O(\log n)$).



Verifica di inclusione: La Merkle Proof

Una delle caratteristiche più potenti del Merkle Tree è la possibilità di verificare se una specifica transazione è inclusa in un blocco senza dover scaricare

o scorrere l'intero albero. Questo meccanismo, noto come **Merkle Proof**, è alla base del funzionamento dei nodi leggeri (SPV - *Simplified Payment Verification*).

Per effettuare questa verifica, a un client sono necessari solo tre elementi:

1. L'**hash della transazione** che si vuole verificare (la foglia di partenza).
2. La **Merkle Root**, che è pubblicamente disponibile nell'header del blocco.
3. Il **Merkle Path** (o percorso di prova), ovvero la lista degli hash dei nodi "fratelli" necessari per risalire l'albero fino alla radice.

Il processo di verifica: Il client calcola l'hash della propria transazione e lo concatena con il primo hash fornito nel Merkle Path (il nodo fratello), calcolando il nuovo hash padre. Questo procedimento viene ripetuto iterativamente, salendo di livello in livello, fino a ottenere un singolo hash finale. Se l'hash calcolato dal client **coincide esattamente** con la Merkle Root del blocco, si ha la certezza crittografica che la transazione è valida e regolarmente inserita nel blocco. Poiché l'albero è binario, la quantità di hash fratelli da richiedere (il Merkle Path) cresce in modo logaritmico rispetto al numero totale di transazioni ($O(\log n)$), garantendo un'enorme efficienza in termini di banda e computazione.

1.2 Firma digitale

Una firma digitale è l'equivalente informatico di una tipica firma convenzionale. La struttura è molto simile a quella del Message Authentication, tuttavia qui abbiamo una struttura simmetrica in cui abbiamo una **chiave segreta che serve a firmare** e una **chiave pubblica che serve per verificare**. Uno schema di firma digitale è composto da tre algoritmi

1. **generateKeys:** algoritmo che genera la chiave di verifica che sarà pubblica e la chiave di firma che sarà privata. È importante notare che Bitcoin garantisce "pseudoanonimato" poiché anziché usare la propria identità, le transazioni avvengono tramite la chiave di verifica. Dunque, non abbiamo un riferimento diretto all'identità e, inoltre, a ogni persona potrebbero corrispondere infinite coppie di chiavi.
2. **sign:** è l'algoritmo che ci permetterà di firmare le transazioni.
3. **verify:** algoritmo che controlla che una certa firma sia valida

In particolare, lo schema di firma adottato da Bitcoin è **ECDSA** uno schema di firma particolarmente efficace che sfrutta punti di curve ellittiche per permettere una cifratura sicura con chiavi crittografiche a soli 256 bit. Per semplicità, spiegheremo lo schema DSA che è praticamente lo stesso di ECDSA ma usa spazi Z_N^* anziché curve ellittiche. Questo schema di firma si basa sul problema del **logaritmo discreto** che consiste nel problema del non riuscire a ricavare in tempo efficiente quel valore $x : g^x = X$ dati solo X e g . Gli algoritmi di firma e verifica sono dunque:

$$\begin{array}{l}
 \text{Keygen } q, G, g \\
 x \leftarrow \{1, \dots, q\} \quad h \leftarrow g^x \\
 H : \{0, 1\}^* \rightarrow \{1, \dots, q\} \\
 v_K = (G, g, q, h, H) \quad M = \{0, 1\}^* \\
 s_K = x \\
 \\
 \text{Sign}_K(m) \\
 k \leftarrow \{1, \dots, q\} \\
 R \leftarrow g^k; \quad r \leftarrow R \bmod q \\
 s \leftarrow k^{-1} (H(m) + r x) \bmod q \\
 \text{output } (R, s) \\
 \\
 \text{Ver}(v_K, m, (r, s)) \\
 w \leftarrow s^{-1} \bmod q \\
 u_1 \leftarrow g^{w H(m)} \\
 u_2 \leftarrow h^{w r} \\
 v \leftarrow u_1 \cdot u_2 \\
 \text{if } (v \bmod q = r) \text{ return 1} \\
 \text{ELSE RETURN 0}
 \end{array}$$

Come già detto tutto fa riferimento alle chiavi pubbliche che saranno le identità chiamate **indirizzi** in Bitcoin.

1.3 Basic Cryptocurrencies

Vediamo adesso una carrellata di Cryptocurrencies basilari utili a mostrare tutte le problematiche di implementare delle monete digitali.

1.3.1 GoofyCoin

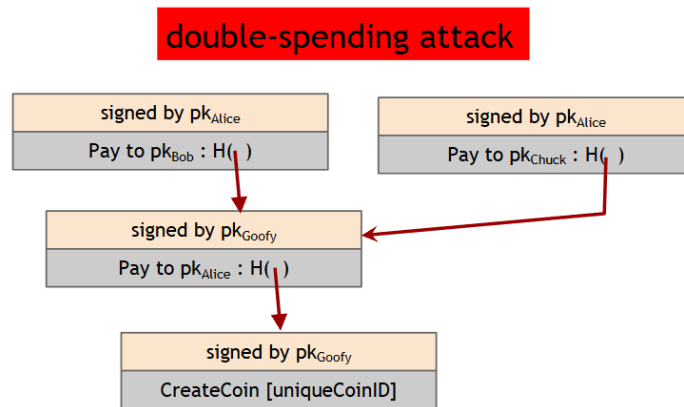
Immaginiamo questa moneta che possiede le seguenti operazioni o **transazioni**:

- **CreateCoin(uniqueCoinID)**: Questa operazione permette di creare una nuova moneta e, banalmente, colui che effettua questa transazione

la firma con la sua chiave privata. Così facendo, tutti possono verificare che quella moneta gli appartiene.

- **PayTo(indirizzo):** Questa operazione permette a qualcuno di pagare una moneta (in questo momento unitaria) a un altro indirizzo. Per farlo, oltre a fornire l'indirizzo a cui inviare la moneta, è necessario fornire l'hash pointer del blocco contenente la transazione di creazione o dell'avvenuto ricevimento della moneta.

Questo schema, seppur basilare, ci è utile a mostrare il problema del **double spending**, infatti, poiché l'unica cosa necessaria da fornire per un pagamento è un'hash pointer che provi che ci appartiene quella specifica moneta, ci basta fare tutti i pagamenti con lo stesso punto di partenza. Così facendo, riusciremo a spendere infinite volte la stessa moneta.

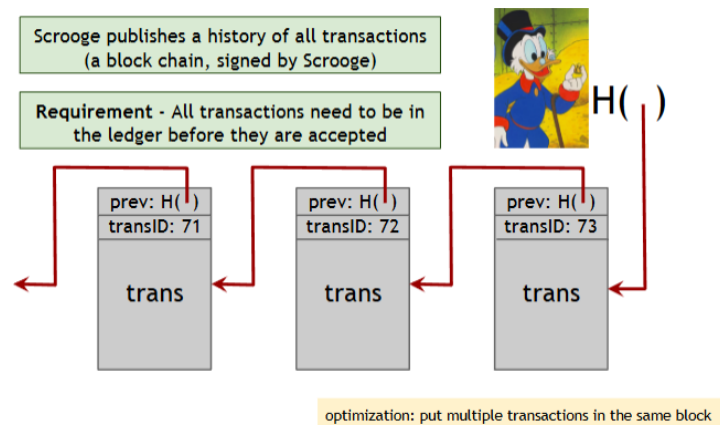


Inoltre, un altro problema è che allo stato attuale, **ognuno può creare la sua moneta gratis** e che, all'aumentare di scambi della moneta, si vanno a creare delle **catene di verifica sempre più grandi**.

Tutti questi problemi nascono dal fatto che abbiamo un sistema del tutto **decentralizzato**. Passiamo dunque alla prossima tipologia di moneta.

1.3.2 ScroogeCoin

A differenza della moneta precedente, immaginiamo di voler creare un sistema completamente centralizzato. Poiché centralizzato, questo sistema sarà **controllato da un'unica entità** che, in questo caso, sarà "Zio Paperone". Questa entità sarà l'unica a poter creare nuova moneta e a gestire il registro (cioè la Blockchain) e, quindi, decidere quali operazioni inserire e quali no.



Inoltre, anziché fare pagamenti unitari, possiamo spezzettare i pagamenti facendo sì che, ad esempio, se l'unità è 100 euro, per pagare un caffè pago 1 euro al commerciante e 99 euro a me stesso. Naturalmente ciò complica le cose perché si devono fare tante verifiche, tuttavia è fattibile.

Eliminiamo anche il problema del double spendig poiché, essendo l'entità a decidere quali operazioni accettare e quali no sul registro e, dunque, non permetterebbe mai una cosa del genere...

Tutto ciò accade se e solo se l'entità non è malevola, in caso contrario, nessun potrebbe contrastarla, rendendo inutilizzabile la moneta.

Capitolo 2

Come Bitcoin ottiene la Decentralizzazione

Uno degli obiettivi primari di Bitcoin è la creazione di un **sistema di pagamento puramente distribuito e decentralizzato**. Tuttavia, implementare e mantenere una rete senza alcun punto centrale di controllo rappresenta una sfida ingegneristica e di teoria dei giochi estremamente complessa.

Esistono molti sistemi informatici che utilizzano architetture distribuite ma non completamente decentralizzate. Un esempio quotidiano è il sistema delle email (protocollo SMTP): pur non essendoci un unico server mondiale, il sistema è "federato", ovvero si appoggia a un numero ristretto di grandi provider (es. Google, Microsoft) che fungono da intermediari per gli utenti. Bitcoin, al contrario, ambisce a eliminare del tutto queste figure centrali.

2.1 Le domande fondamentali del sistema

Per comprendere come Bitcoin realizzi questa decentralizzazione, dobbiamo capire come il protocollo risponde a cinque domande cruciali che, nei sistemi tradizionali, vengono risolte da un Ente Centrale (come una Banca o uno Stato):

1. **Gestione del registro:** Chi mantiene e aggiorna la copia ufficiale della blockchain?
2. **Validazione:** Chi possiede l'autorità per stabilire quali transazioni sono valide e quali no?
3. **Emissione:** Chi crea nuova moneta e secondo quali tempistiche?

4. **Governance:** Chi determina le regole del protocollo e come vengono implementati gli aggiornamenti software?
5. **Valore di scambio:** Come si determina il valore della moneta rispetto al mercato reale? (*A differenza delle valute fiat, il protocollo Bitcoin non gestisce questo aspetto, lasciandolo interamente alle dinamiche di libero mercato, ovvero alla legge della domanda e dell'offerta sui vari exchange*).

Oltre a questi interrogativi di sistema, vi sono questioni legate all'utente finale: come e dove conservare le proprie monete (i *wallet* crittografici) e come interfacciarsi con la rete per effettuare le operazioni.

2.2 I tre pilastri della decentralizzazione

La risposta di Bitcoin a queste sfide si articola su tre livelli principali:

- **Network Peer-to-Peer (P2P):** La rete non ha server centrali. Ogni nodo (computer) connesso è uguale agli altri, propaga le transazioni e mantiene una copia indipendente e verificata dell'intera blockchain.
- **Mining (Consenso e Creazione):** La validazione delle transazioni e la creazione di nuove monete avvengono tramite un processo competitivo noto come *Proof of Work*. Chiunque può, in linea di principio, dedicare potenza di calcolo per partecipare alla sicurezza della rete.
- **Sviluppo e Aggiornamenti:** Il codice sorgente di Bitcoin è *open-source*. Le modifiche al software vengono proposte pubblicamente e richiedono un ampio consenso da parte della rete (nodi e miner) per essere adottate.

Decentralizzazione: Teoria vs. Pratica

È fondamentale sottolineare che il sistema è aperto a tutti *sulla carta*, ma la realtà operativa attuale presenta delle sfumature. Ad esempio, il mining è teoricamente accessibile a chiunque possieda un computer; tuttavia, l'enorme competizione ha reso impossibile minare in solitaria in modo redditizio senza l'ausilio di hardware specializzato (ASIC) e senza unirsi a grandi *Mining Pool* (cooperative di miner). Questo introduce, di fatto, un certo grado di "centralizzazione pratica" in un sistema nato per essere totalmente decentralizzato.

2.3 Il Consenso Distribuito

L'obiettivo principale della rete è fare in modo che tutti i nodi raggiungano un accordo condiviso (consenso) sullo stato del registro, stabilendo, ad esempio, quale debba essere il prossimo blocco da aggiungere alla catena.

Supponiamo che l'utente A voglia inviare un pagamento all'utente B. A si preoccuperà di trasmettere (in *broadcast*) la transazione a tutti i nodi a cui è connesso. Questi la verificheranno e, se risulta valida, la manterranno in memoria pronti a inserirla in un blocco. Tuttavia, in un dato istante, non tutti i nodi hanno la stessa identica visione della rete: a causa dei fisiologici tempi di propagazione, i nodi potrebbero avere set di transazioni in sospeso leggermente diversi. Il consenso in Bitcoin, infatti, non è **immediato**, ma **emergente e a lungo termine**. I blocchi vengono proposti in media ogni 10 minuti, ma il consenso assoluto sull'irreversibilità di una transazione si consolida solo dopo circa un'ora (attendendo la conferma di almeno 6 blocchi successivi).

Le sfide del consenso: Latenza e Generali Bizantini

Immaginiamo che tre nodi (es. rosso, blu e verde) propongano contemporaneamente il proprio set di operazioni valide sotto forma di blocco. La rete deve avere un meccanismo per sceglierne uno solo, che tutti gli altri dovranno poi verificare e accettare. Questo processo è ostacolato da diversi fattori del mondo reale:

- Latenza e partizioni di rete (non tutti i nodi comunicano alla stessa velocità).
- Nodi che crashano o si disconnettono improvvisamente.
- **Nodi malevoli** che tentano di ingannare il sistema.

In informatica, la difficoltà di raggiungere un accordo in presenza di attori malevoli o fallaci è nota come il **Problema dei Generali Bizantini**.

Per risolverlo, Bitcoin assume un modello in cui i nodi sono *razionali*: essi seguono le regole del protocollo non per "bontà", ma perché il sistema di incentivi economici (la ricompensa in bitcoin) rende l'onestà molto più redditizia rispetto a un tentativo di attacco.

Il problema dell'identità: Sybil Attack

Nei sistemi distribuiti classici o chiusi, il problema dei nodi malevoli si argina legando ogni nodo a un'identità reale e verificata (es. tramite un'autorità

centrale). In Bitcoin, però, questo approccio non è applicabile: il sistema è progettato per garantire lo **pseudo-anonimato**, dove l'unica "identità" è rappresentata da una coppia di chiavi crittografiche.

Poiché chiunque può generare infinite chiavi a costo zero, la rete risulta esposta al **Sybil Attack**. In questo scenario, un singolo attaccante potrebbe creare migliaia di nodi fittizi per acquisire la maggioranza artificiale nella rete, sovrastare i nodi onesti e prendere il controllo delle decisioni.

Per neutralizzare il Sybil Attack e raggiungere il consenso senza fare affidamento su identità fisse, Bitcoin introduce un meccanismo innovativo. L'elezione del nodo che avrà il diritto di proporre il blocco successivo non avviene per votazione (dove i nodi fittizi vincerebbero), ma tramite una sorta di "lotteria" competitiva.

Algoritmo di Consenso

Ad ogni round, viene estratto in maniera randomica un leader temporaneo, che sceglie il blocco da aggiungere. Gli altri nodi della rete non possono influenzare l'estrazione, ma mantengono il potere di **validazione** e se il leader propone un blocco contenente operazioni non valide, il resto della rete lo ignorerà, rifiutandosi di estendere la blockchain a partire da quel blocco e, di fatto, creando una biforcazione. Tuttavia, poiché l'euristica per continuare una catena è quella di **continuare la catena più lunga**, quella biforcazione nata dall'inserimento di transazioni non valide morirà lì e nessuno la considererà valida. Questo meccanismo viene detto **Consenso implicito**.

Supponendo quindi di voler descrivere un algoritmo di consenso nella sua forma più semplice potremmo fare come segue:

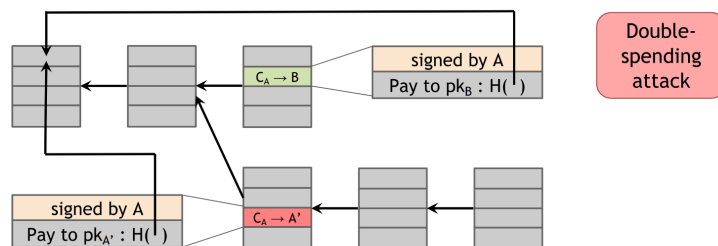
1. Le nuove transazioni vengono **inviare in broadcast** a tutta la rete.
2. Ogni nodo le mette da parte e forma dei blocchi.
3. Ad ogni round, viene scelto random un leader che propagherà il suo blocco a tutti gli altri nodi.
4. Gli altri nodi controlleranno il blocco verificando che le transazioni sono valide e, nel caso in cui lo vogliano considerare valido, faranno la prossima aggiunta inserendo in testa a quel nodo.

Analizziamo quindi cosa può e cosa non può cercare di fare un nodo malevolo:

- **Può rubare Bitcoin?** Per quanto possa provarci, se le chiavi crittografiche di un utente sono state generate in maniera sicura, questo non

potrà mai accadere poiché l'attaccante non riuscirebbe a generare una firma valida.

- **Denial of Service vs Bob?** Supponiamo che un nodo non voglia ammettere una certa transazione valida, questa potrebbe non venire mai scelta? Di fatto no poiché per quanto possa stare antipatico Bob, alcuni nodi onesti potrebbero comunque inserirla e, dunque, l'unico effetto sarebbe il delay della transazione.
- **Double-Spending?** Immaginiamo questo scenario: A fa' una transazione verso B e ottiene (senza attendere) un certo bene. La transazione con il pagamento $A \rightarrow B$ viene salvata nel prossimo blocco della blockchain, tuttavia successivamente A diventa il leader e crea una fork con un altro pagamento $A \rightarrow C$. Poiché entrambe le fork sono valide, i nodi potrebbero scegliere arbitrariamente quale allungare (di solito si procede con il principio di allungare la catena più lunga). Così facendo A ha effettivamente fatto due pagamenti spendendo, però, una sola volta monete.



Per evitare di questi problemi, è necessario che il blocco sia sufficientemente "in fondo" nella catena e, generalmente, lo consideriamo permanente dopo circa 6 blocchi successivi.

2.3.1 Incentivi e Proof of Work

Come abbiamo detto, assumeremo che tutti gli utenti della rete o quasi si comportino in maniera *razionale*. Dunque, faremo sì di dare incentivi e penalità per far sì che questi si comportino secondo le regole. Innanzitutto abbiamo due incentivi per la partecipazione attiva all'allungamento della blockchain:

1. **Block Reward:** Quando viene inserito un blocco nella catena, colui che lo ha proposto guadagna dei bitcoin tramite la *coin create transaction*. Poiché i Bitcoin hanno un valore reale di mercato fare ciò

conviene. Inoltre, scoraggiamo la creazione di blocchi non validi poiché questi non verrebbero considerati e, dunque, si "perderebbero" soldi. (parleremo tra poco di Proof of Work).

2. **Transaction Fees:** Quando viene aggiunto un blocco, oltre che un guadagno dato dalla coin create transaction, coloro che mandano le transazioni aggiungono una mancia che viene guadagnata da colui che propone il blocco. Così facendo, si incentiva a mettere transazioni con più mancia all'interno della blockchain.

Come possiamo facilmente intuire, è chiaro che il potere di aggiungere un nuovo blocco è qualcosa di molto ambito e che non possiamo effettuare tramite scelte casuali (anche perché come detto, saremmo soggetti a Sybil attack). L'idea diventa quindi non quella di scegliere un nodo del tutto a caso, bensì **scegliamo il prossimo nodo in base a quanto questo ha contribuito in termini di risorse allo sviluppo della rete**. Queste risorse devono essere non monopolizzabili e non duplicabili. Questo può essere effettuato tramite **Proof of Work** cioè quanta potenza computazionale si è data alla rete e tramite **Proof of Stake** cioè si sceglie in base a quanto è disposto a "spendere" un nodo per aggiungere il prossimo blocco. Questi sono rispettivamente adottati in Bitcoin ed Ethereum.

Hash Puzzle

Per quanto riguarda nello specifico Bitcoin, abbiamo la Proof of Work che è l'**hash puzzle**. In pratica quello che facciamo è creare il blocco includendo tutte le transazioni e l'hash pointer precedente e cercando una nonce tale che è l'intero Hash del blocco abbia una serie di zero all'inizio. Maggiori saranno gli zeri da cercare, più sarà difficile procurarsi quest'hash e, se la funzione hash è sicura, allora l'unico modo per trovare questa nonce è essere fortunati o provare tantissime combinazioni di Nonce. Come dicevamo, quindi, colui che cerca l'hash mette potenza computazionale a disposizione e fare questo ha un certo costo anche in termini di elettricità. Dunque, fare questo sforzo per inserire transazioni non valide è chiaramente uno spreco di soldi. Inoltre, l'hash puzzle gode di due proprietà principali:

1. **Difficile da calcolare:** come detto è necessaria una grande quantità di prove per trovare la giusta Nonce.
2. **Parametrizzabile:** è facile regolare la difficoltà della ricerca fixando il numero di zeri da trovare. Questo viene fatto poiché vogliamo far sì che venga aggiunto un blocco all'incirca ogni 10 minuti.

3. **Veloce da verificare:** Nonostante sia molto difficile da calcolare, resta il fatto che, una volta fornita la Nonce, questo sia estremamente facile da calcolare.

Nell'intestazione del blocco Bitcoin, lo spazio dedicato alla Nonce standard è di soli **32 bit**. Questo significa che esistono "solo" circa 4,3 miliardi di combinazioni, un numero che i moderni hardware esauriscono in frazioni di secondo senza trovare un hash valido. Per ovviare a questo problema, i miner sfruttano l'**extraNonce**, un campo inserito all'interno della transazione di ricompensa (*Coinbase Transaction*). Modificando l'**extraNonce**, cambia l'hash di quella specifica transazione, generando un effetto a cascata che modifica l'intero Merkle Tree e aggiorna la **Merkle Root** nell'header. Questa piccola modifica fornisce al miner una base di partenza completamente nuova, permettendogli di testare altri 4,3 miliardi di combinazioni di Nonce.

Sostenibilità e Difficoltà Dinamica

Perché la struttura sia sostenibile, deve garantire ai *miner* un guadagno (Block Reward + Transaction Fees) mediamente superiore ai costi operativi (hardware ed elettricità). Se questo profitto non fosse possibile, i miner spegnerebbero i loro macchinari, compromettendo la sicurezza della blockchain.

Tuttavia, il protocollo non garantisce un profitto fisso, poiché il valore della moneta fluttua sul mercato. Per compensare l'ingresso o l'uscita dei miner dalla rete, il protocollo regola automaticamente la **difficoltà dell'hash puzzle**. Questo aggiustamento non serve a salvaguardare i profitti, ma a garantire che, indipendentemente da quanta potenza computazionale sia attiva, venga scoperto un nuovo blocco in media **ogni 10 minuti**. Se molti miner entrano attratti dai profitti, la difficoltà si alza; se il prezzo crolla e molti miner spengono le macchine per non andare in perdita, la difficoltà si abbassa, permettendo alla rete di continuare a funzionare regolarmente.

L'Attacco del 51%

Come abbiamo visto, l'algoritmo di consenso funziona fintanto che la maggioranza della potenza computazionale (hash power) è in mano a nodi onesti. Ma cosa accadrebbe se un singolo attaccante (o un cartello di miner) riuscisse a controllare il **51% dell'hash power** totale della rete? Analizziamo i vari scenari:

- **Può rubare le monete di altri utenti?** No. Per autorizzare una transazione e spendere fondi altrui è necessaria la firma digitale creata

con la chiave privata del legittimo proprietario. Nessuna potenza di calcolo (salvo i futuri computer quantistici) può falsificare questa firma.

- **Può cambiare il reward del blocco (creare monete dal nulla)?**

No. Anche se il miner al 51% creasse un blocco assegnandosi 1000 BTC di ricompensa invece di quelli previsti dal protocollo, gli altri nodi (i full node che verificano le regole) scarterebbero immediatamente il blocco considerandolo non valido, indipendentemente da quanta potenza di calcolo lo abbia generato.

- **Può sopprimere (censurare) del tutto le transazioni? Nella**

pratica, no. Teoricamente, se un attaccante riuscisse a mantenere la maggioranza assoluta dell'hash power per sempre, potrebbe ignorare sistematicamente i blocchi onesti contenenti la transazione sgradita, costringendo la rete ad adottare la sua catena (più lunga e priva della transazione). Tuttavia, la transazione non svanisce: rimane in attesa nella *mempool* di tutti i nodi onesti. Poiché mantenere il 51% dell'hash power comporta un costo energetico ed economico insostenibile nel lungo periodo, l'attaccante è destinato a perdere la maggioranza. Non appena ciò accade (o in momenti di forte varianza probabilistica in cui i miner onesti trovano più blocchi di fila), la rete onesta includerà la transazione. Inoltre, una censura palese e sistematica spingerebbe il *social consensus* della rete ad attuare un *Hard Fork* difensivo per neutralizzare l'hardware dell'attaccante, rendendo di fatto impossibile una censura definitiva.

- **Può distruggere la fiducia in Bitcoin (Double Spending)? Sì.**

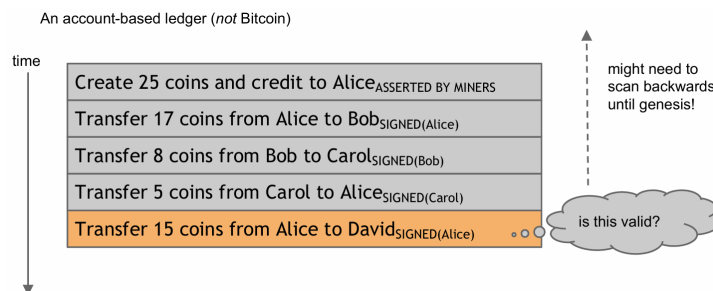
Il danno più grave di un attacco al 51% è la possibilità di effettuare *double spending* riscrivendo la storia recente della blockchain. L'attaccante può inviare dei fondi a un exchange, farsi convertire i BTC in dollari, e contemporaneamente minare in segreto una catena alternativa in cui invia quegli stessi BTC a un altro suo indirizzo. Avendo il 51%, la sua catena segreta diventerà presto la più lunga; una volta pubblicata, la rete cancellerà la transazione verso l'exchange, permettendo all'attaccante di riprendersi i fondi.

Perché non accade spesso? Oltre all'enorme difficoltà logistica e all'altissimo costo per procurarsi l'hardware e l'energia necessari per un simile attacco, vi è un fortissimo **disincentivo economico**. Se un attacco del genere avesse successo su scala visibile, la fiducia nel network crollerebbe istantaneamente. Il valore di Bitcoin precipiterebbe, rendendo di fatto inutile il bottino rubato dall'attaccante e distruggendo l'enorme investimento milionario fatto per ottenere il 51% dell'hardware.

2.4 Struttura delle Transazioni e Modello UTXO

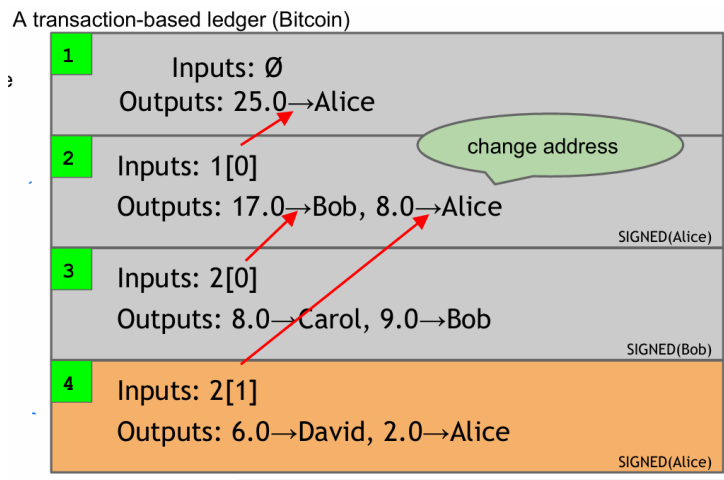
Andiamo ora ad analizzare nel dettaglio come vengono strutturate ed effettuate le transazioni all'interno della blockchain.

Uno dei problemi principali in un sistema finanziario decentralizzato è la verifica della disponibilità dei fondi: i nodi devono accertarsi che chi sta inviando del denaro ne sia effettivamente in possesso. Se adottassimo un approccio "ingenuo" (*naive ledger*), in cui il registro tiene semplicemente traccia dei trasferimenti $A \rightarrow B$, per verificare il saldo di un utente saremmo costretti a ricalcolare l'intero storico delle sue transazioni partendo dal *Genesis Block* (il primo blocco della catena). Questo causerebbe un dispendio di tempo e risorse computazionali insostenibile.



Per risolvere questo problema con la massima efficienza, Bitcoin non utilizza il concetto di "conto bancario" con un saldo aggiornabile, ma sfrutta il **Modello UTXO (Unspent Transaction Output)**.

In questo modello, le monete non esistono come saldi in un database, ma come "pezzi di metallo" digitali indivisibili (gli UTXO, appunto). Una transazione non fa altro che prendere in input dei vecchi UTXO, distruggerli, e creare in output dei nuovi UTXO.



Più nello specifico, una transazione è composta da:

- **Input:** Per giustificare e finanziare la spesa, la transazione deve fornire degli *hash pointer*. Questi puntatori indicano in modo univoco l'ID di una transazione precedente e l'indice specifico dell'output che si vuole spendere. Per poter usare questo input, l'utente deve fornire la firma digitale corrispondente (dimostrando di possedere la chiave privata di quell'indirizzo).
- **Output:** Definiscono i nuovi destinatari e i relativi importi. Poiché gli input (gli UTXO) **devono essere spesi per intero**, se il valore totale degli input è maggiore di quanto si vuole pagare al destinatario, la transazione dovrà generare un secondo output destinato al mittente stesso. Questo output aggiuntivo rappresenta il **resto** dell'operazione.

Grazie a questa struttura logica modulare, è possibile effettuare operazioni complesse con estrema facilità. Ad esempio, si possono realizzare **pagamenti congiunti** inserendo input provenienti da indirizzi diversi (per sommare i fondi); naturalmente, in questo caso, la transazione dovrà contenere una firma digitale separata e valida per ogni singolo input fornito. Inoltre, è anche possibile effettuare operazioni di **consolidamento** in cui si prendono in input più UTXO in proprio possesso per compattarli in un solo UTXO.

Capitolo 3

Consenso di Nakamoto

Prima abbiamo discusso in maniera piuttosto semplicistica come dovrebbe funzionare l'algoritmo di consenso ma, adesso, vediamo chiaramente il perché di alcune specifiche tecniche viste precedentemente.

Ricordiamo che abbiamo bisogno di:

- **Consistenza:** Cioè i nodi onesti devono poter essere d'accordo su quello che è il prossimo blocco da inserire (o quanto meno lo devono essere a lungo termine).
- **Liveness:** Cioè la blockchain deve essere in continua evoluzione.

È necessario e lo vedremo tra poco, notare che siamo in un sistema di messaggistico **asincrono** in cui i messaggi potrebbero non arrivare nello stesso ordine in cui sono stati inviati.

Analizziamo dunque il funzionamento dell'algoritmo di consenso. Come già abbiamo anticipato questo si basa sulla **Proof of Work**, una sorta di challenge in cui i miner competono per ottenere un hash di un blocco valido che abbia una certa forma dettata da vari zero all'inizio dell'hash. Ogni nodo della rete cercherà di inserire il prossimo blocco all'interno della catena più lunga. Inoltre, vogliamo che in ogni momento che tutti i blocchi a meno degli ultimi k siano considerati **definitivi**. Vogliamo dunque ottenere:

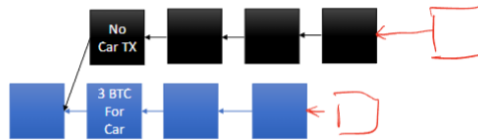
- **Consistenza:** Vogliamo che tutti i nodi onesti siano d'accordo su tutti i blocchi a meno degli ultimi k . La probabilità che il k -esimo blocco sia poi sostituito diventa $O(2^{-k})$.
- **Chain Quality:** Qualsiasi sequenza consecutiva di k blocchi contiene "un numero sufficiente di blocchi onesti". I miner che controllano una frazione p della potenza di calcolo mineranno all'incirca una frazione p dei blocchi. Cioè, se i nodi onesti possiedono l'80% dell'hash power,

allora questi dovranno poter creare all'incirca l'80% dei blocchi della catena.

- **Chain growth:** La catena cresce a un ritmo costante e prevedibile. È la proprietà di Liveness (vitalità) del sistema. Ci garantisce che la blockchain non si "congela" mai. Grazie alla calibrazione automatica della difficoltà dell'hash puzzle, la rete continuerà inesorabilmente a sfornare nuovi blocchi e a processare transazioni indipendentemente da quanti miner si disconnettono o si uniscono alla rete.

A questo punto quello che ci viene in mente è, possiamo rendere tutto il processo di mining più veloce? In effetti per come lo abbiamo descritto il sistema è molto lento e per pagare una semplice pizza dovremmo aspettare all'incirca un'ora (cioè che vengano prodotti altri 6 blocchi dopo quello che contiene la nostra transazione) e proprio questo fa' sì che la nostra moneta sia poco utilizzabile nel caso di un utilizzo in questo tipo di contesto.

Immaginiamo uno scenario in cui un attaccante cerchi di generare una catena alternativa per annullare una transazione passata, attacco noto come **Selfish Mining**. Supponiamo che il destinatario della transazione attenda z blocchi (conferme) prima di considerare il pagamento definitivo. A questo punto, l'attaccante parte con uno svantaggio di z blocchi rispetto alla catena onesta.



Definiamo le seguenti probabilità:

- p : la probabilità che il prossimo blocco venga minato dai nodi onesti.
- q : la probabilità che il prossimo blocco venga minato dall'attaccante.

Ad ogni nuovo blocco scoperto, il divario tra le due catene varia: aumenta di 1 se vince la rete onesta, diminuisce di 1 se vince l'attaccante. A vince se riesce a raggiungere il target z .

La probabilità P_z che l'attaccante riesca, prima o poi, a colmare il deficit di z blocchi e a superare la catena onesta è data dalla formula:

$$P_z = \begin{cases} 1 & \text{se } q \geq p \\ \left(\frac{q}{p}\right)^z & \text{se } p > q \end{cases}$$

Questa equazione ci fornisce due dimostrazioni cruciali sul funzionamento di Bitcoin:

1. **Vulnerabilità al 51%:** Se l'attaccante possiede la maggioranza dell'hash power ($q \geq p$), la probabilità di successo è 1 (certezza assoluta). Anche se non ci dice nulla riguardo a quanto tempo sia necessario per farlo.
2. **La regola delle 6 conferme:** Se i nodi onesti sono in maggioranza ($p > q$), la probabilità di successo dell'attaccante decade in modo esponenziale all'aumentare di z . Ad esempio, persino contro un attaccante formidabile che controlla il 30% della rete ($q = 0.3$), se si attendono $z = 6$ blocchi, la probabilità che l'attacco vada a buon fine scende a circa lo 0.6%. Per questo motivo, attendere 6 conferme rende la transazione statisticamente irreversibile.

3.1 Dimostrazione Matematica dell'attacco

Per ricavare la formula del *Nakamoto Consensus*, poniamoci l'obiettivo di trovare P_1 , ovvero la probabilità che l'attaccante recuperi uno svantaggio di esattamente 1 blocco.

Poiché il recupero di ogni singolo blocco è un evento indipendente, la probabilità di recuperare uno svantaggio di z blocchi è semplicemente:

$$P_z = (P_1)^z$$

Per calcolare P_1 , definiamo l'evento A come "l'attaccante riesce a recuperare lo svantaggio" e l'evento F come "il prossimo blocco viene minato dall'attaccante" (mentre $\neg F$ indica che viene minato dai nodi onesti). Possiamo calcolare la probabilità totale di A dividendo lo spazio degli eventi:

$$P_1 = \Pr[A] = \Pr[A \cap F] + \Pr[A \cap \neg F]$$

Applicando la definizione di probabilità condizionata, possiamo espandere la formula:

$$P_1 = \Pr[A | F] \Pr[F] + \Pr[A | \neg F] \Pr[\neg F]$$

Analizziamo i singoli termini sapendo che la probabilità di successo dell'attaccante è q e quella della rete onesta è p :

- $\Pr[F] = q$
- $\Pr[\neg F] = p$

- $\Pr[A \mid F] = 1$: se l'attaccante trova il blocco, il deficit si azzerava subito e ha recuperato con certezza.
- $\Pr[A \mid \neg F] = P_2$: se i nodi onesti trovano il blocco, il deficit passa da 1 a 2. La probabilità di recuperare da 2 è P_2 , che sappiamo essere uguale a $(P_1)^2$.

Sostituendo i valori nell'equazione otteniamo:

$$P_1 = 1 \cdot q + (P_1)^2 \cdot p$$

Riordinando i termini, arriviamo a un'equazione di secondo grado nell'incognita P_1 :

$$p(P_1)^2 - P_1 + q = 0$$

Risolvendo questa equazione (e ricordando l'identità $p + q = 1$), si ottengono direttamente due possibili soluzioni:

$$P_1 = 1 \quad \text{oppure} \quad P_1 = \frac{q}{p}$$

Poiché P_1 è una probabilità, il suo valore deve essere ≤ 1 . Se l'attaccante ha la maggioranza dell'hash power ($q \geq p$), la frazione q/p sarebbe ≥ 1 , rendendo $P_1 = 1$ l'unica soluzione logicamente accettabile (la frode avrà sicuramente successo). Se invece i nodi onesti sono in maggioranza ($p > q$), la teoria dei processi stocastici ci impone di considerare la radice strettamente minore, portandoci alla conclusione finale:

$$P_1 = \begin{cases} 1 & \text{se } q \geq p \\ \frac{q}{p} & \text{se } p > q \end{cases}$$

3.1.1 La scelta della Difficoltà e il Network Delay

Alla luce delle garanzie di sicurezza offerte dal *Nakamoto Consensus*, sorge spontanea una domanda: se il sistema è sicuro finché la maggioranza della potenza è onesta, perché imporre una difficoltà tale da generare un blocco solo ogni 10 minuti? Perché non abbassare il parametro di difficoltà per confermare le transazioni in pochi secondi?

La risposta risiede nei limiti fisici dell'infrastruttura di rete, in particolare nella **latenza di propagazione** (*Network Delay*), indicata con Δ .

In una rete P2P globale, quando un nodo trova un blocco, necessita di un tempo Δ affinché questo venga trasmesso e verificato da tutti gli altri peer. Durante questo lasso di tempo, i nodi onesti che non hanno ancora ricevuto

l'aggiornamento continuano a impiegare potenza di calcolo per risolvere un blocco ormai obsoleto. L'energia spesa in questo gap Δ è di fatto **sprecata**. Al contrario, un avversario malevolo fortemente centralizzato non subisce latenze interne, ottimizzando il 100% del suo *hash rate*.

3.1.1.1 Il Discount Rate della potenza onesta

Possiamo modellare matematicamente l'impatto di questo spreco definendo i seguenti parametri:

- n : numero totale di nodi nella rete.
- α : frazione dei nodi che si comportano onestamente (dunque vi sono in totale αn nodi onesti).
- p : probabilità che un *singolo* nodo risolva il puzzle crittografico in un dato round.

Vogliamo trovare la probabilità che la rete onesta nel suo complesso trovi un blocco in un singolo round. Per farlo, passiamo dall'evento complementare: la probabilità che **nessun** nodo onesto trovi il blocco è $(1-p)^{\alpha n}$. Dunque, la probabilità che **almeno un** nodo onesto ci riesca è:

$$1 - (1 - p)^{\alpha n}$$

Poiché in Bitcoin la probabilità p di trovare un hash valido è infinitesimamente piccola, possiamo applicare l'approssimazione binomiale (o sviluppo di Taylor al primo ordine, per cui $(1-x)^k \approx 1 - kx$ per $x \rightarrow 0$):

$$1 - (1 - p)^{\alpha n} \approx 1 - (1 - \alpha np) = \alpha np$$

Questa espressione, αnp , rappresenta la probabilità di successo per round dell'intera rete onesta. Di conseguenza, il tempo atteso (in round) per la scoperta di un nuovo blocco è semplicemente il suo inverso:

$$\frac{1}{\alpha np}$$

Poiché per ogni blocco trovato si "perdono" Δ round per la propagazione, l'efficienza reale della rete onesta è data dal rapporto tra il tempo di lavoro utile e il tempo totale impiegato (lavoro utile + spreco):

$$\text{Efficienza} = \frac{\frac{1}{\alpha np}}{\frac{1}{\alpha np} + \Delta} = \frac{1}{1 + \alpha np \Delta} \approx 1 - \alpha np \Delta$$

Questo fattore agisce come **Discount Rate** sulla potenza reale degli onesti. La potenza effettiva a disposizione dei nodi corretti non è più αn , ma diviene:

$$(1 - \alpha p n \Delta) \alpha n$$

Se la difficoltà del puzzle fosse troppo bassa, i blocchi verrebbero trovati troppo frequentemente, rendendo il parametro Δ dominante. Il *discount rate* abbatterebbe drasticamente l'efficacia dei nodi onesti, generando continue biforcazioni e avvantaggiando gli attaccanti centralizzati. Per mantenere la stabilità e la sicurezza, Satoshi Nakamoto ha calibrato la difficoltà affinché il tempo atteso per un blocco (10 minuti, ovvero 600 secondi) sia di ordini di grandezza superiore al tempo di latenza globale della rete Δ (pochi secondi), minimizzando così lo spreco di risorse.

Condizione di sicurezza

Nei sistemi distribuiti tradizionali, il consenso si basa sul conteggio dei nodi ("Un nodo, un voto") e la tolleranza viene espressa in base al numero di partecipanti malevoli (es. si tollerano al massimo 1/3 di nodi disonesti). In una rete non centralizzata come Bitcoin, questo approccio fallirebbe istantaneamente a causa del **Sybil Attack** poiché un singolo attaccante potrebbe generare milioni di nodi virtuali a costo zero, acquisendo artificialmente la maggioranza dei voti.

Per neutralizzare questo vettore di attacco, la Proof of Work sposta il paradigma di voto da "Un nodo, un voto" a "Una CPU (un hash), un voto". Di conseguenza, il protocollo Bitcoin **può tollerare un numero teoricamente infinito di nodi disonesti**, a un'unica condizione: che la somma della loro potenza computazionale (*Hash Power*) rimanga strettamente minoritaria rispetto a quella dei nodi onesti ($q < p$). Nelle analisi matematiche e statistiche del protocollo, parametri come la probabilità di successo p non rappresentano la percentuale di utenti onesti, ma la frazione di potenza di calcolo globale sotto il loro controllo.

3.1.2 Vantaggi e Svantaggi

Il sistema a consenso di Nakamoto presenta alcuni vantaggi tra cui:

- **Partecipazione anonima e facile accesso:** Poiché è possibile partecipare semplicemente con l'indirizzo pubblico della chiave generata, è chiaro che la partecipazione sia "anonima" anche se di questo parleremo dopo. Sempre per lo stesso motivo, poiché generare chiavi è estremamente semplice, è di facile accesso e, quindi, molto scalabile.

- **Impossibile prevedere il vincitore:** Se il sistema fosse prevedibile, ci dovremmo trovare ad avere a che fare con casi di collusioni in cui si cerca di corrompere il leader per fargli inserire o non inserire certe transazioni. Invece, poiché il vincitore non è prevedibile, questo non è chiaramente uno scenario fattibile.
- **Tolleranza di tanti nodi malevoli:** Per quanto detto prima, fintanto che la potenza di calcolo dei nodi malevoli è inferiore rispetto alla potenza di calcolo dei nodi onesti, vi è una grossa tolleranza.

Come tutte le cose belle, però, abbiamo alcuni svantaggi parecchio ovvi che riguardano principalmente il fatto che questo metodo è **lento** e, quindi, non permette pagamenti immediati e, a causa delle proof-of-work, abbiamo un **grosso dispendio energetico** rendendo, di fatto, il consumo di energia tra i più alti rispetto perfino ad alcuni paesi del mondo.

Capitolo 4

Anonimato e Privacy

Prima di analizzare le tecniche di anonimato all'interno della blockchain, è necessario definire con precisione cosa significhi questo termine in ambito informatico. Generalmente, il concetto si presta a due interpretazioni principali:

- Interagire **senza utilizzare il proprio vero nome** (ma impiegando un alias testuale o numerico).
- Interagire **senza rivelare alcuna identità** o traccia riconducibile al singolo individuo.

Nel caso di Bitcoin, la prima interpretazione è chiaramente soddisfatta, poiché gli utenti utilizzano gli hash delle proprie chiavi pubbliche (gli indirizzi) come identità. Tuttavia, la seconda interpretazione non lo è. Questa distinzione ci porta a definire il sistema Bitcoin non come anonimo, bensì come **pseudo-anonimo**.

Lo pseudo-anonimato, da solo, non è sufficiente a garantire la privacy su un registro pubblico. In crittografia, il vero anonimato si definisce come la somma di due proprietà: **Pseudo-anonimato + Unlinkability** (Non-collegabilità). L'*unlinkability* è la proprietà per cui differenti interazioni dello stesso utente non possono essere correlate tra loro dal sistema o da un osservatore esterno. Per ottenere un'effettiva privacy in Bitcoin, dovrebbero verificarsi tre condizioni di *unlinkability*:

- Deve essere computazionalmente difficile ricollegare indirizzi diversi allo stesso utente reale.
- Deve essere difficile ricollegare transazioni diverse al medesimo utente.
- Deve essere difficile ricollegare il mittente di un pagamento al suo effettivo destinatario.

Mentre per i primi due casi è possibile adottare euristiche e buone pratiche (come generare un nuovo indirizzo per ogni transazione), il terzo caso è intrinsecamente violato dall'architettura base del protocollo: analizzando una singola transazione standard, il collegamento logico tra l'input (mittente) e l'output (destinatario) è pubblico e palese. Offuscare questo legame diventa fattibile solo se consideriamo la transazione non come un singolo evento isolato, ma come un flusso all'interno di un grafo di transazioni complesse.

Poiché garantire una completa *unlinkability* sull'intero network è impossibile, l'obiettivo crittografico è quello di rendere anonima almeno una porzione delle transazioni, nascondendole all'interno di un **Anonymity Set** (Insieme di Anonimato). L'efficacia di queste tecniche dipenderà direttamente dal *Threat Model*: dobbiamo cioè stabilire a priori **chi è l'attaccante e quali sono le sue capacità** di analisi della rete.

4.1 Perché vogliamo l'anonimato?

Poiché la blockchain è un *ledger* (registro) completamente pubblico, l'assenza di anonimato comporta enormi criticità. Nel sistema bancario tradizionale, il registro delle transazioni è privato e custodito da un ente fiduciario; nessuno, tranne le autorità competenti, può accedere allo storico finanziario di un cittadino. Al contrario, in Bitcoin chiunque può ispezionare il registro, calcolare il saldo di una certa identità pseudo-anonima, e tracciarne ogni singolo movimento in entrata e in uscita.

Questa trasparenza estrema espone gli utenti a veri e propri tracciamenti finanziari. Se l'identità reale di un utente viene associata al suo indirizzo, è possibile per chiunque compiere azioni di spionaggio (ad esempio, un'azienda potrebbe monitorare i fornitori e i volumi d'affari di un competitor).

Spesso si associa erroneamente la richiesta di anonimato finanziario ad attività illecite (acquisto di beni illegali, riciclaggio, evasione fiscale). Sebbene la tecnologia possa essere abusata, il diritto alla privacy finanziaria è un'esigenza legittima per proteggersi da sorveglianza di massa e attacchi mirati. Inoltre, anche i fondi di provenienza illecita, per poter essere utilizzati nell'economia reale, prima o poi devono transitare attraverso un punto di conversione in valuta *fiat* (*exchange* con procedure KYC/AML), rendendo di fatto possibile il tracciamento da parte delle forze dell'ordine. La neutralità della tecnologia crittografica è simile a quella della rete Tor: uno strumento *agnostico* che fornisce privacy a prescindere dall'intento etico o meno dell'utente finale.

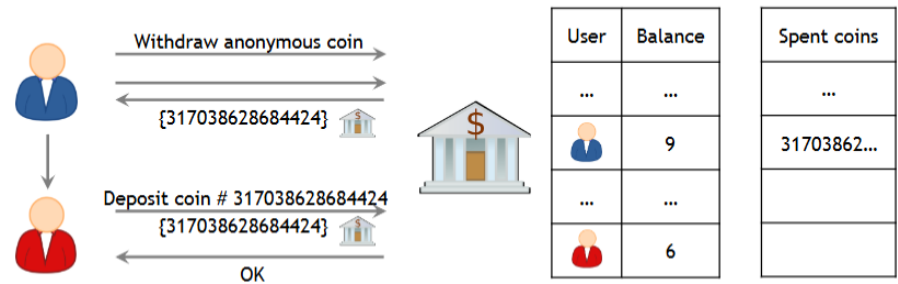
4.1.1 Anonymous e-cash e Blind Signatures

Una delle primissime e più eleganti soluzioni per i pagamenti digitali anonimi fu proposta dal crittografo David Chaum nel 1982. L'idea alla base di *e-cash* era quella di consentire pagamenti digitali mantenendo una banca come autorità centrale fidata per la contabilità, ma impedendole matematicamente di tracciare le abitudini di spesa dei propri clienti.

Il protocollo si basa sul concetto crittografico delle **Blind Signatures** (Firme Cieche) e si articola in tre fasi:

- **1. Fase di Prelievo (*Withdrawal*):** L'utente Alice genera un numero seriale casuale S , che rappresenterà univocamente la moneta elettronica. Per nascondere alla banca, Alice applica una funzione di *blinding* (mascheramento) utilizzando un fattore casuale r , ottenendo un *commitment* $M = f(S, r)$. Alice invia M alla banca richiedendo un prelievo. La banca autentica Alice, verifica i fondi sul suo conto e li detrae. Successivamente, la banca applica la propria firma digitale sul *commitment* cieco, generando $\text{Sign}_{\text{bank}}(M)$, e la restituisce ad Alice. La banca ha firmato la moneta senza averne mai visto il vero numero seriale S .
- **2. Fase di Unblinding:** Ricevuta la firma cieca, Alice utilizza il fattore r per rimuovere il mascheramento matematico applicato in precedenza. Grazie alle proprietà omomorfe delle *blind signatures*, questa operazione restituisce una firma della banca perfettamente valida applicata direttamente al seriale in chiaro: $\text{Sign}_{\text{bank}}(S)$.
- **3. Fase di Deposito (*Deposit*):** Alice desidera acquistare un bene da Bob e gli consegna la moneta $(S, \text{Sign}_{\text{bank}}(S))$. Bob verifica criticograficamente la firma usando la chiave pubblica della banca e, per concludere la transazione, invia la moneta alla banca per farsi accreditare l'importo. La banca riceve il seriale S , verifica la propria firma e controlla nel suo database che S non sia mai stato speso prima (prevenendo il *Double Spending*). Se S è nuovo, la banca accredita i fondi sul conto di Bob.

La garanzia di Anonimato: Durante il deposito, la banca vede il numero seriale S per la prima volta. Non ha alcun modo crittografico per collegare questo seriale S al *commitment* M che aveva firmato ad Alice durante la fase di prelievo. Di conseguenza, pur mantenendo il controllo assoluto contro la doppia spesa, l'autorità centrale non può assolutamente sapere chi ha pagato Bob, garantendo così un anonimato totale e matematicamente provato per il pagatore.



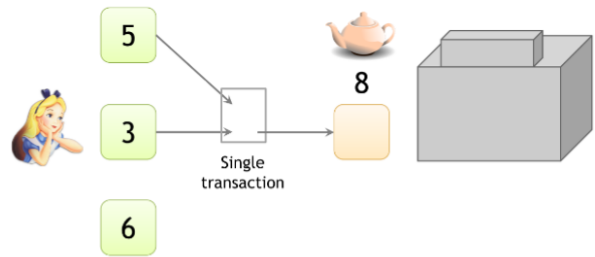
Naturalmente, per garantire tale anonimato abbiamo bisogno di spese unitarie o, al più, un numero limitato di quantità "prestabilite" in maniera tale che la banca non possa fare linking mediante la cifra spesa. Si presuppone chiaramente che il sistema sia utilizzato da più persone poiché altrimenti il link risulta banale.

Il vero e proprio problema è il fatto che i protocolli interattivi con la banca sono complicati da gestire e da decentralizzare soprattutto.

4.1.2 Anonimato grazie a nuovi indirizzi

Supponiamo che A voglia ottenere il massimo anonimato dal sistema block-chain e che, intuitivamente, decide di usare sempre indirizzi diversi per ogni transazione.

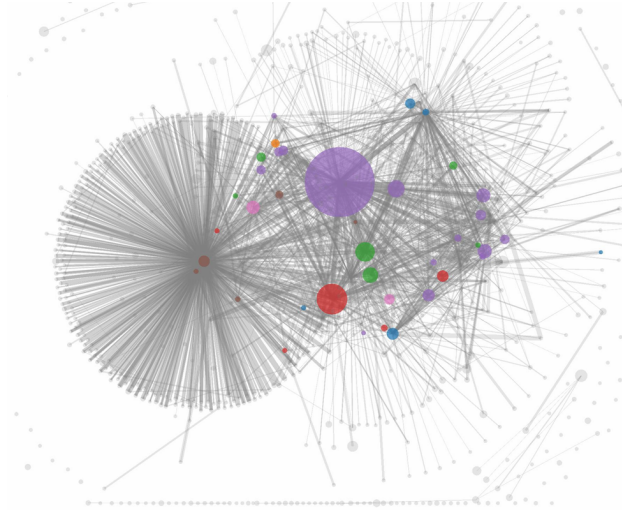
Siamo adesso nella situazione in cui A voglia comprare un oggetto e, per farlo, necessita di un certo numero di monete che sono, quindi, dentro tanti UTXO diversi che hanno una firma diversa.



Poiché un attaccante vede una transazione verso una certa autorità,¹ questo vede prima la transazione di compattazione degli uTXO e, quindi, riesce a collegare che quei due UTXO appartenevano alla stessa entità. Non solo, se il pagamento non è per intero, allora è necessario dare il resto che, quindi, apparterrà alla stessa entità del pagamento.

¹Vedremo dopo come questa cosa venga capita

Tramite queste intuizioni, possiamo tracciare un grafo di questo tipo:



Possiamo notare che quelli che ricevono tante transazioni sono, in genere, le autorità di cui parlavamo prima. Tra queste, possiamo trovare le più note **silkroad**, **satoshi dice**, ecc.. Siamo a tutti gli effetti riusciti a capire come individuare le grossi autorità ma non è ancora ben chiaro come possiamo a tutti gli effetti deanonimizzare i singoli utenti. Per farlo, l'approccio più semplice è quello del *social engineering*:

- **Transazioni dirette:** Se io faccio in modo che qualcuno mi paghi, trovo il suo indirizzo e, tramite il blockchain explorer, posso vedere quanto si aveva nel portafoglio, da chi ha ricevuto questi soldi ecc.
- **Fornitori di Servizi:** Poiché non tutti possono minare Bitcoin, la maggior parte li compra da servizi esterni come binance e quei servizi devono per leggere conoscere l'identità della persona.
- **Careless:** La gente può deanonimizzarsi da sola pubblicando l'indirizzo online per ricevere donazioni ecc.
- **Attacchi retroattivi:** Poiché vengono scoperte algoritmi sempre più efficaci per raggruppare indirizzi e dato che la blockchain è immutabile, è possibile che in futuro venga scoperto un modo ancora più forte per deanonimizzare qualcuno.

4.2 De-anonimizzazione a livello Network

Un altro problema è che pur evitando di pubblicare il proprio indirizzo online, la comunicazione a livello di rete potrebbe fare in modo che qualcuno riesca a deanonimizzarci attraverso l'indirizzo IP.

Una regola generale è *"Il primo nodo che informa qualcuno di una transazione, è probabile che sia la sua fonte"*.

4.2.1 Rete Tor

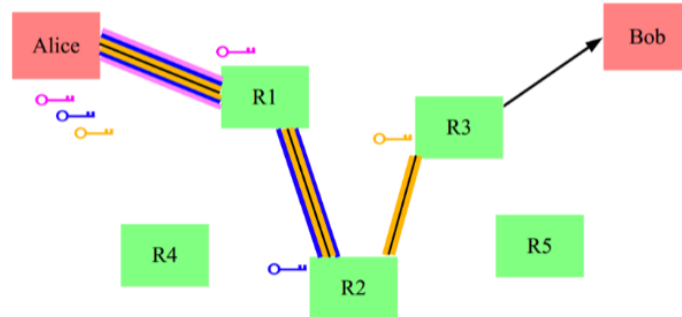
Per evitare questo problema, la soluzione è l'uso della rete **Tor** un protocollo di rete che si pone come obiettivo quello di garantire l'anonimato a livello di rete.

Quello che vogliamo è una comunicazione di questo tipo:

- **Senders:** Questi invieranno i messaggi all'interno della rete di comunicazione.
- **Communication Network:** Qui arriveranno i messaggi e lì non è ben chiaro il mittente e il destinatario. Questo a patto che questo sistema sia usato da un certo numero di persone poiché altrimenti sarebbe banale il collegamento.
- **Recipients:** Questi riceveranno i messaggi attraverso la rete in maniera del tutto anonima. Cioè, il recipients non saprà nemmeno chi gli ha inviato il messaggio. Per quanto possa sembrare assurdo, questo è appropriato dato che Tor è progettato per la navigazione sul web.

Per fare questo, Tor opera secondo un modello di **cifratura a strati**. Questo sistema è considerato sicuro se **almeno uno dei router è onesto**.

Quando Alice deve comunicare con Bob utilizzerà delle chiavi di cifratura valide per la sessione corrente per **cifrare un messaggio n volte, dove n è il numero di macchine intermedie per cui passa il messaggio**. Ogni macchina intermedia applica una chiave di cifratura per togliere uno strato di cifratura al messaggio fino all'ultima macchina del path prestabilito da Alice. A quel punto il messaggio, ormai in chiaro passa dall'ultima macchina ad Alice.



Questo sistema è sicuro nel caso in cui ci sia almeno uno nodo onesto. Guardiamo i vari casi:

- **Se R2 e R3 sono malevoli:** R1 è onesto e sa da chi viene il messaggio. Tuttavia, poiché onesto questa informazione non verrà trasmessa e, dunque, R2 e R3 possono conoscere B ma non A.
- **Se R1 e R3 sono malevoli:** In questo caso, R2 riceverà l'identità di A inoltrata da R1, tuttavia poiché onesto ignorerà tutte le identità inoltrate e garantirà l'anonimato di A.
- **Se R1 e R2 sono malevoli:** In quest'ultimo caso R3 non invierà comunque le identità ricevute da R2 garantendo l'anonimato di A.

Statisticamente, più server ci sono più il sistema è probabile che sia sicuro poiché è più probabile che ci siano server router onesti. Al tempo stesso, più server ci sono maggiore è la latenza.

4.3 Mixing e Anonimato nelle Criptovalute

Un'idea utile per ottenere anonimato nelle criptovalute è quello di usare degli intermediari che fanno da **mixing network**.

Questo può essere utilizzato per ottenere anonimato nelle transazioni di criptovalute operando come meccanismo di dissociazione tra indirizzi di input e indirizzi di output per evitare il linking visto precedentemente. L'idea è quella di avere un intermediario per noi a cui mandiamo i messaggi e che poi li manda per noi attraverso dei **chunk**.

4.3.1 Online Wallet

I wallet online rappresentano una prima forma di mixing, sebbene questo non sia generalmente il loro scopo primario. Questi servizi gestiscono collettivamente i fondi di numerosi utenti, facilitando implicitamente un certo grado di commistione tra i depositi.

Quando un utente trasferisce Bitcoin a un wallet online e successivamente li preleva, tipicamente riceve unità di valuta diverse da quelle originariamente depositate. Questo processo introduce un elemento di dissociazione che rende più complessa l'analisi della provenienza dei fondi.

Questi, però, presentano diverse limitazioni:

1. **Assenza di garanzie:** La maggior parte dei wallet online non si impegna formalmente a implementare pratiche di mixing robuste. L'effetto di mixing deriva principalmente da procedure operative interne finalizzate all'ottimizzazione della gestione dei fondi, piuttosto che da un impegno esplicito verso la tutela della privacy degli utenti.
2. **Conservazione dei registri:** Anche ipotizzando un efficace processo di mixing interno, i wallet online generalmente mantengono registri dettagliati delle transazioni effettuate dagli utenti. Questa pratica è adottata sia per ragioni di convenienza operativa, facilitando la risoluzione di controversie e la gestione del servizio, sia per ottemperare a requisiti normativi sempre più stringenti in materia di antiriciclaggio e contrasto al finanziamento del terrorismo.
3. **Requisiti di Identificazione:** I wallet online sono sottoposti a regolamentazioni e richiedono la verifica degli utenti.

Quello che vorremmo in realtà sono due caratteristiche fondamentali:

- **Impegno a non conservare registri:** I servizi di mixing dedicati dichiarano esplicitamente di non mantenere archivi delle transazioni processate, eliminando teoricamente la possibilità di ricostruire i collegamenti tra indirizzi di input e output.
- **Assenza di requisiti di Identificazione:** A differenza dei wallet regolamentati, i mixing service non richiedono la verifica dell'identità degli utenti, preservando l'anonimato anche rispetto al provider del servizio stesso

4.3.2 Principi di Mixing

Sono stati ideati dei principi fondamentali per ottenere anonimato attraverso sistemi di mixing:

1. **Serie di Mixer:** L'idea è quella di implementare una sequenza di operazioni di mixing successive piuttosto che affidarsi a un singolo intermediario. Questo ha diversi vantaggi:
 - **Riduzione del rischio di compromissione:** L'utilizzo di molteplici mix riduce drasticamente la vulnerabilità del sistema poiché per eliminare l'anonimato bisognerebbe compromettere tutti i mixer contemporaneamente.
 - **Incremento dell'Anonymity set:** Ogni passaggio attraverso un mix diverso espande l'insieme delle transazioni con cui i fondi dell'utente vengono mescolati, aumentando progressivamente la complessità dell'analisi necessaria per deanonimizzare le transazioni.
2. **Uniformità delle Transazioni:** Per evitare il link delle transazioni attraverso l'analisi degli importi, è opportuno che i servizi di mixing paghino attraverso transazioni con pagamenti standard detti **chunk**. Così facendo, poiché tutti spendono/ricevono la stessa quantità di monete in una transazione, non è possibile ricollegare l'origine della moneta alla sua destinazione. Vanno bene anche **diverse taglie di chunk**.
3. **Automatizzazione Lato Client:** Per evitare problemi legati all'elevata complessità della gestione di questo tipo di sistema e, dunque, il possibile errore umano, avrebbe senso far sì che sia lo stesso wallet a gestire questo tipo di meccanismi lato Client in maniera del tutto nascosta al client.
4. **Commissioni con All-or-Nothing:** Poiché questo è un servizio di un qualche tipo, è necessario chiaramente pagarlo. Dunque, per garantire l'anonimato anche del servizio stesso, è necessario adottare un meccanismo **All-or-Nothing** in cui la commissione non è una percentuale della transazione stessa, bensì l'intero importo della transazione che va al servizio con una probabilità ad esempio del 0,01%. Questo preserverebbe l'uniformità delle transazioni e proteggerebbe l'anonimato del servizio stesso.

Allo stato attuale delle cose, nessuno adotta realmente questo tipo di mixing per vari motivi tra cui la legge stessa ma anche perché, volendo i servizi

lavorare in anonimo, è chiaro che non sarebbe possibile imputarli in alcun modo e, dunque, nessuno avrebbe fiducia in un sistema del genere.

4.4 Coinjoin

Poiché i sistemi di mixing centralizzati hanno dei vincoli imposti dalla centralizzazione stessa, si è cercato di passare a un sistema di **mixing decentralizzato** così che non ci sia la necessità di un ente a cui affidarsi e che sia l'utente stesso a gestire i suoi fondi all'interno del processo di mixing.

Qui nasce **CoinJoin** proposto da *Greg Maxwell*. Questo è un protocollo di mixing decentralizzato che sfrutta una proprietà fondamentale delle transazioni di Bitcoin ossia il sistema di **firme aggregate** in un'unica transazione. Vedremo nei capitoli successivi i dettagli di questa implementazione. Per fare capire come funziona, l'idea è che per firmare una transazione facciamo in modo che la firma sia generata da tutti coloro che partecipano a questo meccanismo generando firme indipendenti che singolarmente non sono valide ma che aggregate assieme lo sono.

L'algoritmo procede come segue:

1. **Individuazione dei Peer:** I partecipanti interessati a un round di mixing per un importo specifico devono trovarsi. Comunemente, ciò avviene tramite un server di coordinamento a cui ci si connette, spesso via Tor, per preservare l'anonimato dell'indirizzo IP. I partecipanti si registrano per un round specifico, indicando l'intenzione di mixare un determinato importo, senza ancora rivelare gli UTXO specifici.
2. **Fornitura Input/Output:** Ogni utente invia al coordinatore l'UTXO che vuole mixare e fornisce una prova di possesso (firma). In questa fase, il coordinatore sa che l'indirizzo *A* appartiene all'utente 1. L'utente deve fornire l'indirizzo di "uscita" dove riceverà i fondi puliti. Per evitare che il coordinatore colleghi l'input all'output, l'utente invia l'indirizzo di output tramite una nuova connessione Tor e utilizza una firma cieca.
3. **Costruzione della Firma:** Il coordinatore invia la transazione completa a tutti i partecipanti. Ogni utente controlla la transazione. Firma solo se vede che tra gli output è presente il suo indirizzo con la cifra corretta. Se il coordinatore provasse a rubare i soldi o a cambiare un indirizzo, l'utente non firmerebbe e la transazione fallirebbe. Non c'è rischio di furto.

4. **Verifica della Firma:** Quando il coordinatore raccoglie le firme di tutti i partecipanti (o della soglia minima richiesta), la transazione diventa valida secondo le regole del protocollo Bitcoin. Il coordinatore verifica che tutte le firme siano valide e che la transazione non violi alcuna regola. La transazione viene inviata alla rete Bitcoin. Sulla blockchain apparirà un unico evento in cui 100 persone hanno mescolato i loro soldi, rendendo impossibile per un osservatore esterno sapere chi ha ricevuto cosa.

Questo sistema presenta però alcuni problemi:

- **Individuazione dei Peer:** Un primo ostacolo significativo consiste nell'identificazione di altri utenti interessati al mixing, mantenendo contemporaneamente l'anonimato.
- **Denial of Service:** Partecipanti malevoli potrebbero compromettere l'efficacia del protocollo rifiutandosi di firmare la transazione nelle fasi finali, causando un fallimento del processo di mixing dopo che gli altri utenti hanno già investito tempo e risorse.

Per quanto riguarda l'individuazione dei Peer abbiamo già parlato di come le rete Tor sia una soluzione a questo problema creando delle lobby in cui effettuare la ricerca. Per quanto riguarda, invece, il Denial of Service, possiamo applicare lo stesso meccanismo che adottiamo su Bitcoin cioè il **proof of work** o il **proof of burn**.

Capitolo 5

Zero Knowledge Proofs

Un tipico scenario in crittografia è quello in cui due parti che non si fidano l'un l'altra devono rivelare qualche informazione segreta in maniera sicura **senza che queste rivelino ulteriori informazioni che non vorremmo diffondere**.

Nello specifico, nel nostro caso vogliamo dimostrare di possedere determinati requisiti senza rivelare troppe informazioni.

Definiamo una **dimostrazione** come un qualcosa che convince della validità di un certo asserto. In matematica questa cosa è intesa in senso statico ma per il nostro caso sarà un concetto dinamico effettuato attraverso un sistema interattivo.

Distingueremo tra Prover P cioè colui che vuole dimostrare la validità dell'asserto e verifier V ha il compito di verificare la prova fornita dal prover.

In genere, è più facile verificare che dimostrare o meglio provare e, questo concetto, è rappresentato perfettamente dai problemi NP. Questi sono dei particolari problemi la cui risoluzione è complessa ma che, una volta fornita la soluzione, questa è facile da verificare.

Poiché vogliamo che non venga rivelata alcuna informazione in termini di conoscenza, è necessario definire cosa si intende per conoscenza. In un tipico scenario di comunicazione tra A e B , diremo che B ha un **guadagno in termini di conoscenza se la sua capacità computazionale è arricchita dalla conversazione**.

5.1 Prove Interattive

Come già accennato, effettueremo dimostrazioni secondo prove interattive. Queste rispecchiano due proprietà specifiche:

- **Completeness:** P dovrebbe essere in grado di convincere V della validità di asserti veri.

$$\forall x \in L \quad \Pr[\langle P, V \rangle(x) = 1] \geq 2/3$$

- **Soundness:** Un P disonesto non dovrebbe essere in grado di convincere V della validità di un asserto falso.

$$\forall x \notin L, \forall B \quad \Pr[\langle B, V \rangle(x) = 1] \leq 1/3$$

Informalmente, un sistema di prove interattive per un linguaggio L è detto **zero knowledge** se qualsiasi cosa è calcolabile interagendo con P su input $x \in L$ è calcolabile a partire da x soltanto.

Cioè, il Verificatore (V) non impara nulla di nuovo dall'interazione con il Prover (P) perché, se avesse voluto, avrebbe potuto simulare l'intera conversazione da solo, senza parlare con nessuno.

Supponiamo che un'entità A possa sapere con certezza il risultato di una determinata operazione e che un'entità B voglia verificare questa sua abilità. Dunque B sceglie a caso un'operazione e poi A darà la sua risposta. Se indovina, potrebbe avere avuto fortuna e lo potremmo accettare o fare di nuovo il test, altrimenti lo rigettiamo. Via via che facciamo altre prove, la probabilità che lui abbia solo avuto fortuna va scemando.

Sia *view* la variabile aleatoria che denota ciò che vede V durante l'esecuzione di $(P, V)(x)$ che, nel nostro caso, saranno i messaggi scambiati tra P e V. Diciamo che (P, V) è un interactive proof system che fornisce **Perfect Zero Knowledge** per il linguaggio L se $\exists M \quad PPTM : \forall x \in L, a > 0, \forall h : |h| < |x|^a \quad M(x, h)^1$ sono distribuite in modo identico.

Cioè quello che riesce a produrre la macchina a stati finiti M è del tutto indistinguibile dall'esecuzione reale del protocollo.

In generale, per dimostrare la proprietà di Zero knowledge, guardiamo alla View e cerchiamo di ricostruire i parametri a ritroso poiché l'importante qui è proprio l'ordine in cui avvengono le cose, senza di questo, la conversazione sarebbe banale.

5.1.1 Isomorfismo tra Grafi

Due grafi si definiscono **isomorfi** se esiste un'applicazione biunivoca tale che ogni nodo di G_1 può essere mappato in un nodo di G_2 preservando il grado.

¹h rappresenta la conoscenza pregressa di un verificatore malevolo che vuole rubare delle informazioni. Mentre x rappresenta l'input comune tra prover e verifier.

Decidere se due grafi sono isomorfi è un problema non risolvibile tramite un algoritmo polinomiale e, questo, lo rende perfetto per il nostro scopo di avere una **Zero Knowledge proof**.

Costruiamo il nostro schema come segue:

- Il Prover possiede due grafi, G_1 e G_2 , che sono isomorfi. Conosce cioè una funzione (una permutazione ρ) tale che $G_2 = \rho(G_1)$. Questo ρ è il **segreto** che il Prover non vuole rivelare.
- Il Prover sceglie una permutazione casuale π :
 - Crea un nuovo grafo H , che è l'immagine di G_1 tramite π ($H = \pi(G_1)$).
 - Invia il grafo H al Verifier.

Poiché H è "coperto" dalla casualità di π il Verifier non può capire come H sia collegato a G_1 o G_2 solo guardandolo.

- Il Verifier invia una sfida casuale c . Può scegliere solo tra due valori: 1 o 2.
 - Se $c = 1$, sta chiedendo: "Dimostrami che H è isomorfo a G_1 ".
 - Se $c = 2$, sta chiedendo: "Dimostrami che H è isomorfo a G_2 ".
- Il Prover calcola la risposta σ in base alla sfida:
 - Se $c = 1$ Il Prover rivela $\sigma = \pi$. Infatti, H era stato costruito proprio come $\pi(G_1)$.
 - Se $c = 2$: Il Prover deve mostrare la relazione tra G_2 e H . Poiché conosce sia il segreto ρ (che collega G_1 a G_2) sia π (che collega G_1 a H), può calcolare la composizione $\sigma = \pi \circ \rho^{-1}$.
- In fine, il Verifier riceve σ e controlla se $H = \sigma(G_c)$ se l'uguaglianza è verificata, se è così, allora il prover passa il turno, altrimenti viene rigettato.

Dimostriamo le proprietà

- **Completeness:** La proprietà di completezza è verificata banalmente. Se il Prover è onesto e possiede effettivamente l'isomorfismo ρ tale che $G_2 = \rho(G_1)$, allora:
 - Se $c = 1$, può sempre fornire $\sigma = \pi$, poiché per costruzione $H = \pi(G_1)$.

- Se $c = 2$, può sempre fornire $\sigma = \pi \circ \rho^{-1}$. Infatti:

$$\sigma(G_2) = (\pi \circ \rho^{-1})(G_2) = \pi(\rho^{-1}(G_2)) = \pi(G_1) = H$$

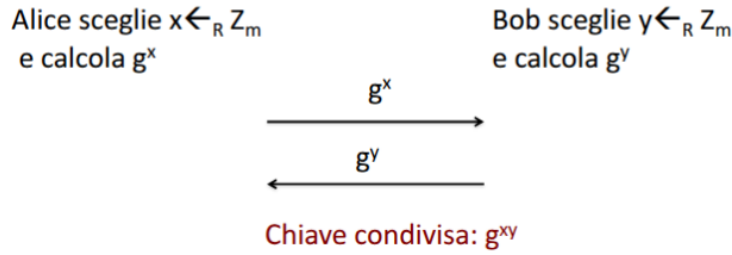
In entrambi i casi, il Verifier accetterà la prova con probabilità 1.

- **Soundness:** La soundness garantisce che un Prover disonesto non possa ingannare il Verifier. Se G_1 e G_2 non sono isomorfi, il Prover non può generare un grafo H che sia contemporaneamente isomorfo a entrambi. Egli deve quindi "scommettere" sulla sfida del Verifier: Può generare H isomorfo a G_1 (sperando che esca $c = 1$). Può generare H isomorfo a G_2 (sperando che esca $c = 2$). In ogni round, la probabilità di successo dell'attaccante è esattamente $1/2$. Iterando il protocollo per n round indipendenti, la probabilità che un baro riesca a convincere il Verifier scende esponenzialmente a $(1/2)^n$. Per $n = 100$, tale probabilità è talmente piccola da essere considerata trascurabile.
- **Perfect Zero Knowledge (PZK):** dobbiamo dimostrare che il Verifier non riceve alcuna informazione sul segreto ρ . Costruiamo una macchina M (il Simulatore) che non conosce ρ ma che è in grado di produrre una trascrizione (una *view*) indistinguibile da quella reale. Il Simulatore M opera nel seguente modo:
 1. Sceglie a caso una sfida "prevista" $c^* \in \{1, 2\}$.
 2. Sceglie una permutazione casuale σ^* .
 3. Calcola il grafo $H^* = \sigma^*(G_{c^*})$.
 4. Produce la tripla (H^*, c^*, σ^*) .

Notiamo che nella realtà il Verifier riceve prima H , poi sceglie c , e infine riceve σ . Il Simulatore, invece, costruisce la risposta a ritroso: sceglie prima la sfida e poi "aggiusta" il grafo H di conseguenza. Poiché in un protocollo PZK le distribuzioni sono identiche, un osservatore che guarda il log della comunicazione non potrà distinguere se H sia stato inviato prima o dopo la scelta di c . Dato che M è riuscito a generare una *view* valida senza conoscere ρ , concludiamo che la *view* reale non contiene alcuna traccia di ρ . Il guadagno di conoscenza del Verifier è quindi pari a zero.

5.2 Diffie-Hellman Zero-Knowledge

Parliamo adesso di un altro protocollo, il protocollo **Diffie-Hellman**. Siano G, g, m i parametri pubblici:



Questo rappresenta un protocollo standard di distribuzione di chiave crittografica simmetrica. Nel nostro caso vorremo provare che (g, g^x, g^y, g^{xy}) è una **quadrupla Diffie-Hellman** valida, cioè che:

$$DL_g(g^y) = DL_{g^x}(g^{xy})$$

Vediamo come funziona la prova Zero Knowledge. Siano $g^x = X, g^y = Y, g^{xy} = W$:

1. Il Prover sceglierà un valore random $r \xleftarrow{R} \{1, \dots, m\}$ e porrà $u \leftarrow g^r, v \leftarrow X^r$ inviandoli al verifier.
2. Il verifier sceglierà $c \xleftarrow{R} \{1, \dots, m\}$ e lo invierà al prover.
3. Il Prover calcolerà $z = r + cy \pmod m$ e lo invierà al verifier.
4. A questo punto il verifier verificherà se:

- $g^z = u \cdot Y^c$
- $X^z = v \cdot W^c$

Dimostriamo le proprietà

- **Completeness:** Se la quadrupla è valida, allora avremo che:

- $g^z = g^{r+cy} = g^r \cdot (g^y)^c$.
- $X^z = (g^x)^{r+cy} = g^{xr} \cdot (g^{xy})^c = v \cdot W^c$.

- **Soundness:** Supponiamo che un attaccante voglia convincere il Verifier che (g, X, Y, W) sia una quadrupla di Diffie-Hellman valida, quando in realtà non lo è. Poniamo quindi $W = g^s$ con $s \neq xy$. Per superare il controllo, l'attaccante deve scegliere i valori di impegno u e v . Siano questi $u = g^\alpha$ e $v = g^\beta$ per qualche α, β scelti dall'attaccante. Al passo 4, il Verifier accetterà la prova solo se l'attaccante è in grado di fornire un valore z che soddisfi contemporaneamente le due equazioni di verifica:

1. $g^z = u \cdot Y^c \implies g^z = g^\alpha \cdot g^{yc} = g^{\alpha+yc}$
2. $X^z = v \cdot W^c \implies (g^x)^z = g^\beta \cdot (g^s)^c \implies g^{xz} = g^{\beta+sc}$

Dalla prima equazione ricaviamo che deve essere $z \equiv \alpha + yc \pmod{m}$. Sostituendo questo valore di z nella seconda equazione (espressa in termini di esponenti), otteniamo:

$$\begin{aligned} x(\alpha + yc) &\equiv \beta + sc \pmod{m} \\ x\alpha + xyc &\equiv \beta + sc \pmod{m} \end{aligned}$$

Raggruppando i termini che dipendono dalla sfida c , l'equazione diventa:

$$c(xy - s) \equiv \beta - x\alpha \pmod{m}$$

Poiché per ipotesi la quadrupla è falsa il termine $(xy - s)$ è diverso da zero. In un gruppo di ordine primo m , questo termine ammette un inverso moltiplicativo. Di conseguenza, esiste un unico valore di c che soddisfa l'uguaglianza:

$$c \equiv (\beta - x\alpha)(xy - s)^{-1} \pmod{m}$$

Tuttavia, l'attaccante deve inviare u e v (e quindi fissare α e β) al passo 1, prima che il Verifier scelga c al passo 2. Poiché c è scelto uniformemente a caso tra m valori possibili, la probabilità che l'attaccante abbia scelto i valori di impegno corretti per "indovinare" l'unica sfida possibile è esattamente:

$$Pr[\text{Successo dell'attaccante}] = \frac{1}{m}$$

Dato che m è un numero estremamente grande, questa probabilità è trascurabile, garantendo così la proprietà di Soundness.

- **Zero-Knowledge:** Cerchiamo innanzitutto di capire la view: (u, v) , c , z . Vediamo come costruire una tripletta a posteriori:

1. Scegliamo c random tra $1, \dots, m$
2. Scegliamo z random tra $0, \dots, m$
3. Calcoliamo $u = g^z / y^c$ e $v = X^z / W^c$. In questo modo, quando il Verificatore andrà a controllare se $X^z = v \cdot W^c$, l'uguaglianza sarà vera per costruzione, anche se il simulatore non sa assolutamente nulla di x o y .

Teorema Fondamentale sulla Zero Knowledge

Se esistono funzioni unidirezionali, allora ogni linguaggio in NP ammette un **zero knowledge proof**.

5.3 Prove di Conoscenza

Le prove che abbiamo visto, sono **prove di appartenenza** in cui noi dimostriamo che una certa quadrupla nel caso Diffie-Hellman appartiene a un certo linguaggio NP formato da tutte le quadruple Diffie-Hellman.

Potremmo non solo voler dimostrare l'appartenenza ad un certo linguaggio, ma potremmo voler anche mostrare come questo vi appartenga. Per intenderci, non solo voglio dimostrare che il grafo possiede un ciclo Hamiltoniano, ma voglio anche **mostrare di conoscere** questo cammino. Queste sono dette **prove di conoscenza** in cui vogliamo dimostrare di conoscere qualcosa senza che questa conoscenza venga trasferita.

5.3.1 Formalizzazione di Conoscenza

Consideriamo relazioni in NP, ovvero linguaggi L che ammettono una funzione (o relazione) polinomiale ρ tale che, per ogni $x \in L$, esiste y detto **testimone** tale che soddisfa $\rho(x, y) = 1$.

Se voglio dimostrare che un grafo è Hamiltoniano, y sarebbe proprio il cammino vero e proprio.

Una prova di conoscenza vorremmo che sia un qualcosa che dimostri che noi possediamo questo tipo di conoscenza senza che, però, la mostri al verificatore.

Per fare una cosa di questo tipo, richiediamo l'esistenza di un algoritmo **estrattore E** che, interagendo con il prover, estrarrà l'informazione in questione.

Esempio di Prova di conoscenza

Supponiamo di avere h, g con $h = g^x$ con x segreto e g e h scelti opportunamente e con l'insieme degli esponenti del tipo $\{1, \dots, m\}$. Facciamo la prova di conoscenza in questo modo:

1. Il prover sceglie $R = g^r$ con r la randomness e manda R al verificatore.
2. Il verificatore sceglie $c \in \{1, \dots, m\}$.
3. Il prover calcola $s = r + cx \pmod m$ e lo manda

4. Il verificatore verifica che $g^s = R \cdot h^c$.

Se il prover conosce x , allora non è un problema rispondere due volte alla challenge con lo stesso R . Per cui, supponiamo di avere queste due risposte:

$$\begin{aligned} g_1^S &= R \cdot h_1^C \\ g_2^S &= R \cdot h_2^C \end{aligned}$$

Avendo questi due, possiamo calcolare:

$$g^{S_1 - S_2} = h^{C_1 - C_2}$$

Siano $S_1 - S_2 = \Delta S$ e $C_1 - C_2 = \Delta C$, allora avremo che $x = \frac{\Delta S}{\Delta C}$ e, dunque, $h = g^{\frac{\Delta S}{\Delta C}}$.

Nella pratica, un prover onesto non risponderà mai a due sfide diverse utilizzando la stessa componente casuale (R), poiché tale negligenza permetterebbe a chiunque di estrarre immediatamente la sua chiave privata. Tuttavia, la validità della Proof of Knowledge risiede proprio in questa possibilità teorica: la proprietà di Knowledge Soundness garantisce infatti che, se un'entità (l'estrattore) fosse in grado di ottenere due risposte diverse per lo stesso impegno, il segreto estratto sarebbe univocamente quello corretto. Questo 'esperimento mentale' ci assicura matematicamente che il prover non sta fornendo risposte casuali per pura fortuna, ma che deve necessariamente essere in possesso della conoscenza che dichiara di avere.

5.3.2 ZeroKnowledge Proof Non Interattiva

Nello scenario delle blockchain, l'interazione diretta tra Prover e Verifier risulta spesso impraticabile poiché richiederebbe che entrambe le parti siano online simultaneamente per scambiarsi messaggi in tempo reale.

Per ovviare a questo limite, si ricorre alle Non-Interactive Zero-Knowledge Proofs (NIZK). Una prova si definisce non interattiva quando il Prover può generare e inviare un singolo messaggio (la prova) che chiunque può verificare in modo asincrono, senza ulteriori comunicazioni. L'integrazione di queste prove in sistemi distribuiti è fondamentale per garantire la scalabilità e la verificabilità pubblica: un miner o un nodo deve poter validare una transazione privata anche ore o giorni dopo la sua creazione. Il metodo standard per trasformare un protocollo interattivo (come quello di Schnorr) in uno non interattivo è l'euristica di Fiat-Shamir.

Tale tecnica consiste nel sostituire la sfida casuale c del Verifier con il risultato di una funzione hash crittografica applicata ai dati pubblici e all'impegno iniziale del Prover:

$$c = H(x, R)$$

Dove x rappresenta l'istante pubblico e R il commitment iniziale. Poiché l'output di una funzione hash è imprevedibile, il Prover non può conoscere la sfida prima di aver fissato il proprio impegno, rendendo matematicamente impossibile barare. In questo modo, la sequenza di messaggi viene "compressa" in un unico oggetto statico, garantendo che la sicurezza del protocollo rimanga integra pur eliminando la necessità di un dialogo diretto.

Capitolo 6

Zerocoin e Zerocash

L'anonimato nelle transazioni finanziarie rappresenta un diritto fondamentale per molti utenti di criptovalute. Sappiamo che Bitcoin offre solamente pseudonimità, con transazioni potenzialmente tracciabili attraverso tecniche di analisi del grafo e clustering degli indirizzi.

In questo contesto, Zerocoin e la sua evoluzione Zerocash emergono come protocolli particolarmente significativi, rappresentando non semplici miglioramenti ma vere rivoluzioni concettuali nell'approccio all'anonimato, attraverso l'utilizzo innovativo di primitive crittografiche avanzate.

6.1 Zerocoin

L'idea dello Zerocoin può essere spiegata attraverso una metafora di una bacheca pubblica condivisa. Supponiamo che un utente desideri creare uno Zerocoin, questo utente spende un qualcosa detto **base coin** e crea un **numero seriale** di cui fa un **commitment** cioè il seriale è in una busta opaca.

Gli altri utenti controllano che ciò che ha speso sia effettivamente valido e controllano anche che il commitment sia valido.

A questo punto supponiamo di voler spendere questo Zerocoin, l'utente produrrà una Zero Knowledge che dimostri:

- Il fatto che l'utente conosca il commitment C .
- Il fatto che lei conosca (senza rivelarlo) il valore r tale che c esponga s .

Questo tipo di commitment è del tipo $C = g^Sh^r$ e l'utente, per spendere la moneta, mostrerà S e mostrerà in Zero knowledge che con questo S è stato creato C con un certo r che solo lei conosce.

Questa prova verrà fatta tramite una prova **non interattiva**, generando $c = H(R||\dots)$ e, poiché l'hash non è prevedibile, allora ci si può fidare.

Inoltre, poiché S viene mostrato, allora quello Zerocoin non è più spendibile poiché presente nella blockchain.

6.1.1 Minting

Cerchiamo di formalizzarlo correttamente adesso. La coniazione rappresenta la fase iniziale nel ciclo di vita di uno Zerocoin, dove un utente converte un'unità di valuta base in uno Zerocoin. Il processo si articola nei seguenti passaggi:

1. **Generazione del seriale:** L'utente genera un numero seriale casuale S che identificherà univocamente la moneta, mantenendolo inizialmente segreto.
2. **Calcolo del commitment:** Viene calcolato un commitment crittografico $C = \text{Com}(S, r)$ utilizzando il numero seriale S e un valore casuale r (*randomness*). Il commitment funziona come una cassaforte digitale con le proprietà di:
 - **Hiding:** impossibilità di estrarre S a partire da C ;
 - **Binding:** impossibilità di cambiare S una volta pubblicato C , permettendo di verificare successivamente che C contenga effettivamente quel determinato S .
3. **Pubblicazione:** L'utente pubblica il commitment C sulla blockchain insieme a un'unità della valuta base tramite una transazione di tipo "Mint".
4. **Verifica:** I nodi della rete verificano che il commitment sia correttamente formato e che l'unità di valuta base sia autentica.

Gli Zerocoin vengono emessi in **denominazioni standard**, creando un "anonymity set" omogeneo che impedisce il tracciamento basato sui valori delle transazioni.

A livello di protocollo, Zerocoin è anonimo, tuttavia, tramite side channel, è possibile rompere l'anonimato.

6.1.2 Redeeming

Il processo di riscatto si sviluppa nel seguente modo:

1. **Identificazione dei commitment:** L'utente esamina la blockchain per identificare tutti i commitment Zerocoin registrati, costruendo l'insieme $(C_1, C_2, \dots, C_n)$ che include il proprio commitment.
2. **Generazione della prova:** Genera una prova zero-knowledge non interattiva π che dimostra due fatti:
 - (a) l'utente conosce un commitment C presente nell'insieme pubblico $C_1 \dots C_n$;
 - (b) l'utente conosce un valore r tale che C si apre correttamente sul numero seriale S che sta per essere rivelato.
3. **Pubblicazione della transazione:** Pubblica sulla blockchain una transazione di spesa contenente la coppia (S, π) , composta dal numero seriale e dalla prova zero-knowledge.
4. **Verifica:** I validatori verificano che il numero seriale non sia già stato utilizzato (prevenendo la doppia spesa) e che la prova π sia matematicamente valida.

La proprietà "zero-knowledge" della prova garantisce che, pur dimostrando la validità del riscatto, non venga rivelato quale specifico commitment l'utente stia utilizzando. Sebbene tutte le transazioni siano pubbliche, risulta matematicamente impossibile collegare il numero seriale rivelato a uno specifico commitment originale.

6.1.3 Limiti Tecnici

I commitment sono pubblicati potenzialmente da tutti gli utenti e sappiamo che più è grande lo statement, più è grande la prova in Zero Knowledge. Nel nostro caso, la prova che dovremmo mostrare è che

$$(C = C_1) \cup (C = C_2) \cup \dots (C = C_n)$$

E questo, a livello computazionale è insostenibile e irrealizzabile in pratica.

6.1.4 Accumulatori crittografici

La funzione degli accumulatori essenziale è trasformare l'insieme di commitment $(C_1, \dots, C_n)$ in un singolo digest crittografico A la cui dimensione rimane costante indipendentemente dalla cardinalità dell'insieme.

Le proprietà fondamentali di un accumulatore crittografico includono:

- **Compattezza:** La dimensione dell'accumulatore rimane costante indipendentemente dal numero di elementi accumulati.
- **Efficienza di verifica:** La verifica dell'appartenenza di un elemento all'accumulatore ha complessità computazionale sublineare (idealmente costante o logaritmica) rispetto alla dimensione dell'insieme.
- **Unidirezionalità:** È computazionalmente intrattabile derivare gli elementi originali dal valore dell'accumulatore.
- **Resistenza alle collisioni:** È computazionalmente intrattabile trovare un elemento valido che non è stato incluso nell'accumulatore.

Nel contesto di Zerocoin, gli accumulatori permettono di riformulare la prova di appartenenza in modo drammaticamente più efficiente. Anziché dimostrare una disgiunzione su tutti i commitment, l'utente deve semplicemente provare di conoscere un "testimone" (*witness*) w che attesti l'appartenenza del proprio commitment c all'accumulatore A .

6.1.4.1 RSA

Prima di andare avanti, è necessario discutere di RSA e spiegarlo semplicemente. Sia N il prodotto di due primi p, q e e un esponente pubblico. La funzione RSA è definita come:

$$RSA[N, e](x) = x^e \mod N$$

Questa è la prima **funzione trapdoor** per cui, dati e, N e $y = x^e \mod N$, è possibile ritrovare x solo se si ha la conoscenza di p, q . Per estrarre x utilizziamo l'algoritmo esteso di Euclide con input la p, q ed e . In sostanza questo algoritmo ci permette di estrarre radici e -esime.

A partire dallo stesso N , si possono creare più y utilizzando diversi e .

RSA è uno strumento molto utile che serve anche per fare le firme blind ecc.

6.1.5 Implementazione degli Accumulatori

La costruzione di questi accumulatori è fatta mediante l'uso di diversi algoritmi:

- **Setup:** Viene generato $N = pq$ e w , che è un elemento non nullo minore di N (la base dell'accumulatore).

- **Accumulate(params, $(c_1, \dots, c_n)$):** Consente di creare il **commitment di un insieme di valori**. Calcoliamo l'accumulatore come $A = w^{c_1 \dots c_n} \pmod{N}$.
- **GenWitness(params, v , $(c_1, \dots, c_n)$):** Genera un testimone per dimostrare che l'elemento v appartiene all'insieme $V = \{c_1, \dots, c_n\}$. Se $v \in V$, il testimone W si calcola come l'accumulatore del sottoinsieme privo di v , ovvero: $W = \text{Accumulate}(\text{params}, V \setminus \{v\}) = w^{\prod_{c_i \neq v} c_i} \pmod{N}$.
- **Acc.Verify(params, v , W , A):** Il verificatore calcola $A' = W^v \pmod{N}$. Se $A == A'$, la prova è valida e l'appartenenza è verificata.

Questa costruzione è sicura perché l'entità che esegue il setup elimina i fattori p e q . Di conseguenza, poiché nessuno conosce la fattorizzazione di N (assunzione RSA), l'unico modo per falsificare una prova sarebbe calcolare la radice v -esima di A senza conoscere i fattori, il che è computazionalmente impossibile.

6.1.6 Integrazione in Zerocoin

Nel contesto di Zerocoin, l'integrazione degli accumulatori RSA permette di riformulare le prove di riscatto in modo sostanzialmente più efficiente. La prova di riscatto si trasforma in una dimostrazione zero knowledge strutturata come segue:

“Conosco (c, w, r) tali che:” (6.1)

- c è un commitment valido incluso nell'accumulatore A (verificabile attraverso il testimone w)
- c si apre correttamente sul numero seriale S dichiarato, ovvero $c = H(S, r)$

Questa formulazione comporta diversi vantaggi critici:

- **Efficienza computazionale:** La generazione e verifica della prova hanno complessità costante rispetto al numero di commitment nel sistema.
- **Dimensione della prova:** La prova stessa ha dimensione costante (tipicamente alcuni kilobyte), indipendentemente dalla dimensione dell'insieme dei commitment.

- **Scalabilità:** Il sistema può supportare un numero arbitrariamente grande di commitment senza degradazione significativa delle prestazioni.

6.1.7 Sfide di Integrazione e Problematiche

Nonostante l'eleganza teorica, l'integrazione di Zerocoin nel protocollo Bitcoin presenta ostacoli tecnici e fiduciari che ne hanno impedito l'adozione su larga scala nella rete principale. Le criticità si possono riassumere in tre punti fondamentali:

- **Limitazioni di Bitcoin Script:** Bitcoin utilizza un linguaggio di scripting non completo (non Turing-completo) e volutamente limitato per ragioni di sicurezza e prevedibilità. Verificare le operazioni matematiche complesse necessarie per gli accumulatori RSA e per le prove Zero-Knowledge richiederebbe l'introduzione di nuovi codici operativi (*opcodes*) estremamente pesanti. Attualmente, Bitcoin Script non è in grado di processare tali verifiche in modo nativo.
- **Divergenza nei Meccanismi di Verifica:** Mentre Bitcoin si affida alle firme digitali **ECDSA** (o Schnorr) per autorizzare il trasferimento di UTXO, Zerocoin sostituisce il concetto di firma con una **dimostrazione Zero-Knowledge**. Questo cambio di paradigma richiederebbe una modifica strutturale al modo in cui i nodi validano le transazioni: non si tratterebbe più di verificare la proprietà di una chiave, ma la correttezza di una prova di conoscenza relativa a un accumulatore.
- **Il problema del Trusted Setup (TTP):** Come discusso nella sezione relativa a RSA, il sistema si basa su un modulo $N = p \cdot q$. Per garantire che nessuno possa falsificare monete dal nulla, è necessario che i fattori primi p e q vengano distrutti immediatamente dopo la generazione di N . Questo processo richiede l'esistenza di una **Trusted Third Party** (TTP) che esegua il setup iniziale. In un ecosistema decentralizzato come quello delle criptovalute, l'esistenza di parametri che richiedono fiducia in un ente centrale (o in un evento di generazione "sacro") rappresenta un punto di fallimento e una contraddizione filosofica rispetto ai principi di Bitcoin.

Queste problematiche hanno portato la ricerca crittografica verso soluzioni più efficienti e meno dipendenti da setup fiduciari, culminando nello sviluppo di Zerocash.

6.2 Zerocash

Zerocash, presentato nel 2014 al simposio Usenix Security da E. Ben-Sasson et al., rappresenta un'evoluzione fondamentale rispetto a Zerocoin, superandone le limitazioni strutturali e introducendo un paradigma completamente nuovo per la privacy nelle criptovalute. A differenza di Zerocoin, che operava come un'estensione del protocollo Bitcoin richiedendo una valuta base, Zerocash è stato concepito come un sistema autonomo e completo, svincolato dalla necessità di una valuta sottostante.

Le innovazioni principali di Zerocash rispetto al suo predecessore sono duplici:

- **Efficienza crittografica:** l'implementazione di primitive crittografiche sostanzialmente più efficienti per la generazione e verifica delle prove di conoscenza zero. Queste sono le zk-SNARKs che permettono l'uso di prove zero knowledge con statement parecchio lunghi ma dando in output risultati piccoli.
- **Architettura indipendente:** lo sviluppo di un'architettura che funziona interamente attraverso transazioni anonime, eliminando il vincolo di una valuta base esterna come avveniva per Zerocoin.

6.2.1 Il problema del Trusted Setup in Zerocash

Nonostante i significativi avanzamenti tecnici, Zerocash eredita e per certi versi amplifica la problematica del "setup fidato" già presente in Zerocoin. La generazione dei parametri pubblici per le prove zk-SNARK richiede input casuali e segreti che devono essere irrecuperabilmente distrutti dopo la creazione dei parametri pubblici.

Questa fase rappresenta un punto critico nel sistema: chiunque entrasse in possesso di questi valori segreti potrebbe compromettere irrimediabilmente l'intero protocollo, generando fondi dal nulla senza possibilità di rilevamento. Le implicazioni di questa vulnerabilità sono particolarmente severe:

- **Compromissione totale:** A differenza di Zerocoin, dove la compromissione avrebbe consentito principalmente la falsificazione di prove, in Zerocash comprometterebbe l'intero sistema monetario, permettendo la creazione illimitata di valuta.
- **Difficoltà di rilevazione:** La natura delle prove rende potenzialmente impossibile rilevare l'avvenuta compromissione, in contrasto con altri attacchi crittografici che lasciano tracce identificabili.

- **Complessità della cerimonia:** La generazione dei parametri richiede una "cerimonia multi-parte" estremamente sofisticata, in cui diversi partecipanti contribuiscono alla creazione dei parametri in modo che tutti debbano essere compromessi per danneggiare il sistema.

Questa problematica rappresenta il principale ostacolo all'adozione di Zerocash e ha stimolato ulteriori ricerche sulla generazione di parametri in contesti a fiducia distribuita o minimizzata.

6.3 Il Continuum dell'Anonimato nelle Criptovalute

Zerocash si posiziona all'estremità del continuum dell'anonimato nelle criptovalute, rappresentando il massimo livello di privacy attualmente raggiungibile. Analizzando i diversi approcci alla privacy nelle criptovalute, è possibile identificare cinque livelli progressivi di anonimato, come illustrato nella tabella seguente:

System	Type	Anonymity attacks	Deployability
Bitcoin	Pseudonyms	Tx graph analysis	Default
Single mix	Mix	Tx graph analysis, bad mix	Usable today
Mix chain	Mix	Side channels, bad mixes/peers	Bitcoin-compatible
Zerocoin	Cryptographic mix	Side channels (possibly)	Altcoin
Zerocash	Untraceable	None	Altcoin, tricky setup

Questa progressione evidenzia il trade-off fondamentale tra livello di anonimato e complessità implementativa/computazionale, con Zerocash che rappresenta la frontiera attuale della privacy crittografica, al costo di significative sfide tecniche e di adozione.

Capitolo 7

Alternative Mining Puzzles

I puzzle crittografici costituiscono il nucleo fondamentale del sistema Bitcoin, rappresentando non solo un meccanismo di sicurezza, ma anche il principio cardine del sistema di incentivi che guida il comportamento dei partecipanti alla rete. Il sistema di mining di Bitcoin si basa su un delicato equilibrio: da un lato, il puzzle deve essere sufficientemente complesso da rendere **costosi eventuali tentativi di attacco, garantendo così la sicurezza della rete**; dall'altro, deve mantenere un livello di difficoltà tale da assicurare che i miner onesti ricevano compensi adeguati per il loro contributo. Questo equilibrio è essenziale per la sostenibilità a lungo termine dell'ecosistema Bitcoin.

Il design di questi puzzle crittografici non è tuttavia limitato alla configurazione attualmente implementata in Bitcoin. Esistono infatti numerose proposte alternative che perseguono obiettivi diversi, come la **resistenza agli ASIC (Application-Specific Integrated Circuits)**, la **resistenza ai mining pool** e la **generazione di benefici intrinseci oltre alla sicurezza della blockchain**. Questo capitolo analizza tali alternative, esaminandone le caratteristiche, i vantaggi potenziali e le sfide implementative.

7.1 Requisiti dei Mining Puzzles

Indipendentemente dalle specifiche implementazioni, qualsiasi puzzle crittografico adeguato all'utilizzo in un sistema di criptovalute deve soddisfare alcuni requisiti fondamentali. Questi criteri rappresentano il minimo necessario affinché il meccanismo possa garantire la sicurezza, la stabilità e l'equità del sistema.

I requisiti principali sono:

- **Verifica Economica (Efficienza di Verifica):** Un requisito imprescindibile è l'asimmetria computazionale: sebbene la ricerca della soluzione debba essere estremamente onerosa, la sua verifica deve essere immediata e computazionalmente economica. Questo aspetto è cruciale poiché, mentre il mining è un'attività svolta da un sottoinsieme di nodi, la verifica delle soluzioni deve essere eseguita da *tutti* i partecipanti per convalidare la blockchain. Un processo di verifica oneroso comprometterebbe la scalabilità e la decentralizzazione, aumentando eccessivamente i requisiti hardware.
- **Difficoltà Adattiva:** Il puzzle deve consentire un aggiustamento parametrico della difficoltà di risoluzione. Questa caratteristica è essenziale per mantenere costante il tempo medio di emissione dei blocchi, indipendentemente dalle fluttuazioni della potenza di calcolo totale della rete (*hashrate*). In Bitcoin, ad esempio, la difficoltà viene ricalibrata ogni 2016 blocchi (circa due settimane) per mantenere un tempo medio di inter-arrivo di 10 minuti.
- **Proporzionalità alle Risorse:** In un sistema equo, la probabilità di trovare una soluzione valida deve essere direttamente proporzionale alla potenza di calcolo impiegata. Questo principio garantisce che i partecipanti ricevano ricompense commisurate al loro contributo effettivo, prevenendo distorsioni che potrebbero favorire arbitrariamente determinati attori.
- **Assenza di Progresso (Progress-Free):** Il puzzle deve essere progettato in modo che ogni tentativo di risoluzione sia indipendente dai precedenti (proprietà *memoryless*). Questo garantisce che gli operatori di grandi dimensioni non ottengano un vantaggio sproporzionato dovuto all'accumulo di lavoro: la probabilità di minare il blocco successivo dipende solo dalla frazione di potenza di calcolo posseduta nel presente, non dal tempo trascorso a tentare la risoluzione.
- **Accessibilità e Decentralizzazione:** Complementarmente alla proporzionalità, è necessario che anche gli operatori con risorse limitate ricevano compensi che, nel lungo periodo, siano proporzionali al loro contributo. Questo previene una centralizzazione eccessiva e assicura che il sistema rimanga aperto alla partecipazione di una vasta base di utenti.

7.1.1 Il Problema dei Puzzle Sequenziali (Bad Puzzle)

Per comprendere meglio le proprietà necessarie a un mining puzzle, è utile analizzare l'inefficacia di un approccio puramente sequenziale. Immaginiamo un puzzle che richieda l'esecuzione di n passaggi computazionali strettamente dipendenti, dove l'operazione $i + 1$ necessita obbligatoriamente dell'output del passaggio i .

A prima vista, un simile meccanismo potrebbe sembrare equo: chiunque, completando gli n passi, può giungere alla soluzione. Tuttavia, in un contesto reale caratterizzato da un'elevata **eterogeneità dell'hardware**, questo design introduce una distorsione fatale per la decentralizzazione:

- **Monopolio del più veloce (Winner-Takes-All):** In un puzzle sequenziale, la competizione non è una lotteria, ma una gara di velocità pura. Se un miner possiede un'architettura hardware (es. un chip con frequenza di clock superiore) anche solo marginalmente più veloce degli altri, egli completerà gli n passi *sempre* prima di chiunque altro.
- **Erosione della Proporzionalità:** A differenza del Proof-of-Work di Bitcoin, dove un miner con l'1% della potenza di calcolo ha comunque l'1% di probabilità di vincere, in un puzzle sequenziale quel miner avrebbe lo **0% di probabilità** di successo. Il sistema degenererebbe istantaneamente in un monopolio assoluto controllato dall'attore con la latenza computazionale più bassa.
- **Sensibilità all'Architettura:** La "parità di architettura" è un'ipotesi teorica irrealizzabile. Lo sviluppo di circuiti integrati specifici (*Application-Specific Integrated Circuits*, ASIC) ottimizzati per quella specifica sequenza renderebbe obsoleto qualsiasi altro hardware general-purpose, centralizzando l'emissione di moneta nelle mani di chi controlla la produzione di tali chip.

In sintesi, un mining puzzle efficace deve essere **probabilistico** (basato su tentativi indipendenti) e non deterministico/sequenziale, per garantire che la competizione rimanga aperta e che le ricompense siano distribuite in modo equo su tutta la rete.

7.1.2 Good Puzzle: Campionamento Pesato e Proprietà Progress-Free

Al contrario dei modelli sequenziali, un puzzle crittografico efficace deve basarsi su tentativi indipendenti e privi di memoria (*memoryless*). In questo

scenario, la risoluzione del puzzle non è vista come un percorso lineare, ma come una serie di esperimenti casuali (Bernoulli trials).

Le caratteristiche principali di questo modello sono:

- **Indipendenza dei Tentativi:** Ogni tentativo di risoluzione (ad esempio, il calcolo di un hash SHA-256 con un diverso *nonce*) ha la stessa probabilità di successo e non dipende dai risultati dei tentativi precedenti. Non esiste, quindi, un concetto di "avvicinamento" alla soluzione.
- **Equità Probabilistica:** Quello che si ottiene è un campionamento pesato della potenza di calcolo. Se un partecipante possiede il 10% della potenza di calcolo totale della rete (e quindi effettua il 10% dei tentativi globali), egli avrà euristicamente il 10% di probabilità di trovare la soluzione per primo. Anche contro un avversario che possiede il restante 90%, il miner più piccolo mantiene una probabilità di vittoria non nulla, proporzionale al suo sforzo.
- **Vantaggio del Throughput sulla Latenza:** In questo sistema, non vince necessariamente chi ha il processore più veloce in assoluto, ma chi riesce a generare il maggior numero di tentativi nel tempo. Questo permette a più macchine meno performanti di aggregarsi (come nei *mining pool*) per competere efficacemente contro hardware estremamente specializzato.

Questo design garantisce la stabilità del sistema: la sicurezza della rete non dipende dalla velocità del singolo, ma dalla potenza collettiva, rendendo il costo di un attacco (il superamento del 50% della potenza totale) estremamente elevato e prevedibile.

7.2 Puzzle Resistenti agli ASIC

L'evoluzione del mining di Bitcoin ha portato al passaggio dall'hardware *general-purpose* (CPU, GPU) ai circuiti integrati dedicati (**ASIC**), estremamente più efficienti nell'eseguire SHA-256. Questa transizione ha sollevato dubbi sulla decentralizzazione, spingendo la ricerca verso puzzle resistenti agli ASIC per due obiettivi principali:

- **Democratizzazione del Mining:** Ridurre il divario prestazionale tra hardware *commodity* (PC, laptop) e dispositivi specializzati. L'obiettivo è abbassare le barriere d'ingresso, permettendo a un numero maggiore di utenti di partecipare alla sicurezza del network.

- **Prevenzione della Dominanza Manifatturiera:** Impedire che pochi produttori controllino l'ecosistema. Attualmente, i leader di mercato godono di vantaggi competitivi critici:
 - *Vantaggio da rodaggio (burn-in):* Possibilità di utilizzare i chip più efficienti per il mining proprio prima di immetterli sul mercato.
 - *Progettazione interna:* Accesso esclusivo a tecnologie di ottimizzazione non disponibili ai concorrenti minori o agli utenti finali.

In sintesi, l'approccio mira a minimizzare il gap tecnologico tra le ultime innovazioni e l'hardware già disponibile sul mercato, rendendo meno profittevole lo sviluppo di chip ultra-specializzati.

7.2.1 Scrypt e i Memory-Hard Puzzles

I **Memory-Hard Puzzles** sono una classe di algoritmi progettati per rendere il costo della computazione dipendente non solo dalla velocità della CPU, ma anche dalla memoria RAM disponibile. In sintesi, possono essere risolti in due modalità:

- **Approccio con Memoria:** Sfruttando lo spazio di archiviazione per efficientare il calcolo, ottenendo un tempo di esecuzione lineare $O(N)$.
- **Approccio "Brute-Force":** Tentando di risolvere il puzzle senza buffering, il che costringe a ricalcolare i dati intermedi, portando la complessità a un tempo quadratico $O(N^2)$, proibitivo per valori di N elevati.

Sviluppato nel 2009, **Scrypt** è stato il primo protocollo di questo tipo adottato su larga scala per contrastare il dominio degli ASIC. Il suo funzionamento si divide in due fasi principali:

1. Write Stage (Riempimento)

In questa fase si costruisce una struttura dati (vettore V) riempiendola in maniera iterativa con valori pseudocasuali derivati dall'input x :

$$V_0 = H(x), \quad V_1 = H(V_0), \quad \dots, \quad V_{N-1} = H(V_{N-2})$$

L'obiettivo è occupare una quantità significativa di memoria RAM che sia difficile da replicare in piccoli chip specializzati.

2. Read Stage (Accesso Casuale)

La fase di lettura introduce la dipendenza dai dati salvati:

1. Si inizializza un accumulatore $A = H(V_{N-1})$.
2. Per ogni passo k da 0 a $N - 1$:
 - Si determina un indice pseudocasuale $i = A \pmod{N}$.
 - Si aggiorna l'accumulatore: $A = H(A \oplus V_i)$.
3. Il valore finale di A costituisce l'output.

Analisi del Time-Memory Trade-off (TMTO)

Senza l'utilizzo di un buffer (memoria), il miner dovrebbe ricalcolare ogni V_i partendo da V_0 ogni volta che l'indice i viene richiamato. Poiché calcolare V_i richiede mediamente $N/2$ operazioni di hashing, l'intero ciclo di lettura passerebbe da N operazioni a circa $N \cdot \frac{N}{2} \approx \frac{N^2}{2}$.

È possibile implementare dei compromessi: memorizzando solo metà degli elementi, il tempo di calcolo aumenta solo leggermente (da $2N$ a circa $2.5N$), mantenendo comunque una crescita lineare. Tuttavia, Script presenta lo svantaggio che la verifica è costosa quanto il mining, poiché il verificatore deve possedere la stessa quantità di memoria del prover. Nonostante le premesse, la resistenza agli ASIC di Script è stata superata dalla creazione di hardware dedicato (come il miner *Zeus*), dimostrando come la corsa agli armamenti tecnologici sia difficilmente arrestabile se gli incentivi economici sono sufficientemente alti..

7.2.2 Approcci Alternativi alla Resistenza agli ASIC

Oltre all'utilizzo intensivo della memoria, sono state proposte altre strategie per mitigare la centralizzazione del mining:

Funzioni di Hash Complesse

Questa strategia prevede l'utilizzo di una catena di diversi algoritmi di hash. Un esempio celebre è l'algoritmo X11, utilizzato inizialmente da Darkcoin (oggi DASH), che concatena ben 11 diverse funzioni di hash. L'idea alla base è che progettare un ASIC capace di ottimizzare simultaneamente 11 algoritmi differenti sia tecnicamente più impegnativo e molto più costoso rispetto a un singolo chip SHA-256.

Target Mobile

Un approccio più radicale consiste nel modificare periodicamente l'algoritmo del puzzle stesso. Cambiando le regole del gioco, gli ASIC esistenti diventano istantaneamente obsoleti. Sebbene efficace sulla carta, questa strategia presenta notevoli sfide implementative, tra cui il rischio di fork della blockchain e la necessità di aggiornamenti software estremamente frequenti per tutti i nodi della rete.

SHA-256

È interessante notare che esiste una corrente di pensiero che sostiene che l'algoritmo SHA-256 di Bitcoin sia già sufficientemente adeguato. Questa posizione si basa sull'osservazione che l'evoluzione degli ASIC per Bitcoin sta rallentando, con miglioramenti marginali tra generazioni successive. In questa prospettiva, i grandi ASIC più costosi offrono vantaggi prestazionali sempre più ridotti rispetto ai modelli più economici:

Un ASIC economico contiene un numero inferiore di circuiti SHA-256 standard rispetto a un ASIC costoso, ma l'architettura fondamentale rimane la stessa. Questa standardizzazione limita il vantaggio derivante da economie di scala o da progetti proprietari avanzati, potenzialmente stabilizzando l'ecosistema del mining nel lungo periodo.

7.3 Proof-of-Useful Work

Una critica frequentemente rivolta a Bitcoin riguarda lo "spreco" di energia elettrica per il mining. Con un consumo energetico stimato tra 150 MW e 900 MW (a metà del 2014), e significativamente aumentato negli anni successivi, sorge spontanea la domanda: "è possibile riciclare questa potenza computazionale per scopi utili?"

Tra le applicazioni potenzialmente adatte a questo approccio vi sono problemi del tipo "ago nel pagliaio", che richiedono l'esplorazione di vasti spazi di ricerca. Esempi promettenti includono:

- **Folding proteico:** La ricerca di configurazioni a bassa energia per proteine, con applicazioni in biologia e medicina.
- **Ricerca di segnali extraterrestri:** L'analisi di regioni anomale in segnali astronomici, utilizzata nella ricerca di intelligenze extraterrestri.

Questi problemi hanno dimostrato di essere adatti a progetti di calcolo distribuito come i progetti @Home, suggerendo una potenziale compatibilità con un sistema di mining distribuito.

Sfide Implementative

L'implementazione di *proof-of-useful-work* deve affrontare diverse sfide:

- **Difficoltà controllata:** Le istanze del problema scelte casualmente devono mantenere un livello di difficoltà prevedibile.
- **Selezione imparziale:** Si pone il problema di chi debba scegliere i problemi da risolvere, con il rischio di introdurre centralizzazione o bias.

7.3.1 Primecoin e le Catene di Cunningham

Un esempio concreto di *proof-of-useful-work* è **Primecoin**, una criptovaluta lanciata da Sunny King nel 2013. Primecoin sostituisce il tradizionale puzzle basato su SHA-256 con la ricerca di sequenze matematiche di valore scientifico, specificamente le **catene di Cunningham**.

Una catena di Cunningham di prima specie è una sequenza di numeri primi $p_1, p_2, \dots, p_n$ tale che ogni elemento sia legato al precedente dalla relazione:

$$p_i = 2p_{i-1} + 1$$

ovvero ogni numero della sequenza è pari al doppio del precedente più uno.

Per garantire che il mining sia sicuro e legato al consenso della rete, il puzzle viene parametrizzato nel seguente modo:

- **Precisione (n):** Rappresenta la dimensione in bit dei numeri primi ricercati, definendo la complessità computazionale del test di primalità.
- **Binding (m):** Un numero stabilito di bit significativi (MSB) del primo numero della catena (p_1) deve coincidere con l'hash del corpo del blocco (*block header*). Questo vincolo è fondamentale poiché impedisce ai miner di riutilizzare catene scoperte in precedenza o di "pre-minare" soluzioni.
- **Lunghezza (k):** Indica il numero di elementi primi che compongono la catena. Poiché la probabilità di trovare catene lunghe decresce esponenzialmente, k funge da parametro principale per la difficoltà adattiva.

Sebbene molte delle più grandi catene di Cunningham attualmente note siano state scoperte proprio dai miner di Primecoin, il dibattito sulla reale utilità di tali scoperte rimane aperto. Se da un lato esse contribuiscono alla teoria dei numeri, dall'altro l'utilità pratica al di fuori dell'ecosistema della

moneta è considerata da molti ricercatori come marginale rispetto all'enorme dispendio energetico richiesto.

7.3.2 Virtual Mining: Proof of Stake e Proof of Deposit

Il concetto di **Virtual Mining** rappresenta un cambio di paradigma volto a superare le inefficienze energetiche e i costi hardware della Proof of Work. In questo modello, la sicurezza della rete non è garantita dal consumo di elettricità, ma dall'impiego di risorse finanziarie interne al sistema. L'attività di mining viene "virtualizzata" attraverso due approcci principali:

- **Proof of Stake (PoS)**: Il diritto di validare nuovi blocchi e ricevere le relative ricompense è proporzionale alla quantità di moneta posseduta e "messa in gioco" (*staked*) dai partecipanti.
- **Proof of Deposit (PoD)**: Simile alla PoS, richiede che i fondi vengano bloccati in un deposito vincolato per un determinato periodo, fungendo da garanzia collaterale per il comportamento onesto del nodo.

L'argomentazione a favore della sicurezza di questi sistemi si basa sulla **convergenza degli incentivi**: un attacco del 51

Tuttavia, l'assenza di un costo fisico esterno (il cosiddetto "lavoro" termodinamico) solleva questioni critiche, come il problema del **Nothing at Stake**, dove i validatori non hanno un costo reale nel supportare più rami di una fork, e la tendenza alla centralizzazione della ricchezza ("chi più ha, più ottiene"). La sfida per i sistemi post-PoW rimane quindi dimostrare che il rigore matematico e gli incentivi economici possano sostituire interamente la sicurezza garantita dalla spesa energetica.

Capitolo 8

Bitcoin Scripting

Le transazioni reali nella rete Bitcoin possiedono una struttura dati cruda (*raw data*) complessa e standardizzata, spesso rappresentata in formati simil-JSON per facilitarne la lettura da parte degli sviluppatori.



Come si evince dall'immagine, una transazione è divisa in tre sezioni logiche principali:

- **Metadati:** Informazioni di servizio generali come l'hash della transazione stessa (TXID), la versione del protocollo e il *locktime* che è il momento prima del quale la transazione non può essere inclusa in un blocco.
- **Inputs:** Contengono il riferimento ai fondi che si vogliono spendere.

- **prev_out**: È l'*hash pointer* che punta univocamente all'UTXO di una transazione precedente. Contiene il TXID precedente e l'indice (n) dello specifico output all'interno di quella transazione.
 - **scriptSig**: Conosciuto anche come **Unlocking Script**, è la componente crittografica che fornisce la prova di possesso dei fondi. Contiene la **firma digitale** e la **chiave pubblica di colui che possiede l'UTXO**.
- **Outputs**: Definiscono dove andranno i fondi e a quali condizioni.
 - **value**: La quantità di bitcoin trasferiti, espressa in *satoshi* (la più piccola unità di Bitcoin, pari a 10^{-8} BTC).
 - **scriptPubKey**: Conosciuto anche come **Locking Script**, contiene l'**indirizzo del destinatario** che non è altro che l'hash della sua chiave pubblica e, soprattutto, le condizioni logiche necessarie affinché questi fondi possano essere spesi in futuro.

L'utilizzo dei termini "script" non è casuale. Il controllo sulla spesa dei Bitcoin non è affidato a una semplice verifica di saldi, ma all'esecuzione di veri e propri algoritmi scritti in **Bitcoin Script**.

Firma di un Input e del TXID

Volendo riassumere in modo rigoroso il processo di firma e la definizione dell'identificativo della transazione, per ogni input che si desidera spendere avviene quanto segue:

1. **Preparazione (Svuotamento e Sostituzione)**: Si crea una copia temporanea della transazione in cui tutti i campi `scriptSig` vengono svuotati. Successivamente, esclusivamente nell'input che si sta validando, si inserisce temporaneamente lo `scriptPubKey` dell'UTXO di origine al posto di `scriptSig`.
2. **Hashing temporaneo e Firma**: Si calcola l'hash di questa transazione temporanea per ottenere il *Transaction Digest*. Questo hash rappresenta il "messaggio" che viene effettivamente firmato crittograficamente utilizzando la chiave privata dell'utente.
3. **Assemblaggio finale**: Si scarta l'hash temporaneo appena calcolato. La firma digitale ottenuta al passo precedente, accompagnata dalla chiave pubblica in chiaro, viene inserita definitivamente nel campo `scriptSig` della transazione *originale*.

4. **Calcolo del TXID e Trasmissione:** A questo punto la transazione è completa e pronta per essere trasmessa (in *broadcast*) alla rete. Il **TXID** (Transaction ID) viene calcolato dai nodi applicando un doppio SHA-256 all'intera transazione finale. È fondamentale ricordare che il TXID *non* viene inserito come campo all'interno dei dati della transazione, ma funge esclusivamente da "impronta digitale" usata dal network per identificarla univocamente nella blockchain.

8.1 Il linguaggio Bitcoin Script

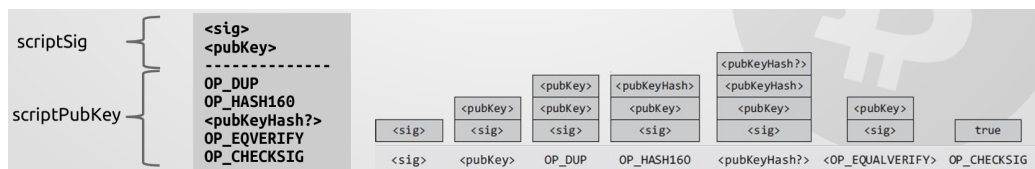
Bitcoin Script è un linguaggio di scripting semplice, basato su una struttura dati a **pila** (pila o *stack*). È progettato appositamente per essere sicuro e prevedibile:

- **Nessuna memoria (*stateless*):** L'esecuzione di uno script non ha memoria delle esecuzioni precedenti. Non esistono parametri di stato salvati.
- **Non Turing-completo:** Il linguaggio manca intenzionalmente di costrutti per cicli infiniti (come `for` o `while`), garantendo che ogni script termini la sua esecuzione in un tempo prevedibile, prevenendo attacchi di tipo *Denial of Service* (DoS) ai nodi della rete.

Con questo linguaggio di scripting è possibile effettuare controlli su **firme multiple con threshold** o **proof of burn**.

8.1.1 Il processo di validazione: Sblocco e Blocco

Il cuore della sicurezza di Bitcoin risiede nel meccanismo con cui un input valida la spesa di un output precedente. Per verificare se una transazione ha il diritto di spendere un determinato UTXO, il network non esegue un solo script, ma combina lo script di sblocco dell'input attuale con lo script di blocco dell'output precedente.



Come mostrato nel diagramma, il processo di valutazione standard (ad esempio per una transazione di tipo *Pay-to-Public-Key-Hash* o P2PKH) avviene seguendo questi passi logici:

1. **Esecuzione dell'Unlocking Script:** I nodi prendono lo **scriptSig** dall'input della transazione attuale. Questo script contiene la firma digitale e la chiave pubblica. Questi elementi vengono caricati (*push*) sulla pila uno dopo l'altro.
2. **Esecuzione del Locking Script:** Subito dopo, senza ripulire la pila, i nodi eseguono lo **scriptPubKey** prelevato dall'UTXO precedente che si vuole spendere. Questo script contiene una serie di operazioni logiche (opcodes) e l'hash dell'indirizzo del destinatario originale.
3. **Verifica Crittografica (Passi logici):** Durante l'esecuzione combinata, lo script esegue due controlli fondamentali per garantire che il mittente sia l'autentico proprietario dei fondi:
 - **Corrispondenza della Chiave Pubblica:** Lo **scriptPubKey** estrae la chiave pubblica fornita nello **scriptSig**, ne calcola l'hash e verifica che questo hash corrisponda esattamente all'hash dell'indirizzo salvato nello script di blocco originale. Questo assicura che la chiave pubblica fornita sia quella corretta.
 - **Verifica della Firma:** Infine, lo **scriptPubKey** utilizza la chiave pubblica appena verificata per validare la firma digitale crittografica (anch'essa fornita nello **scriptSig**). Questa firma deve essere stata creata matematicamente a partire dai dati della transazione attuale, dimostrando che il mittente possiede la chiave privata corrispondente e che la transazione non è stata alterata.

Se, al termine dell'esecuzione di entrambe le parti dello script combinato, la pila contiene un singolo elemento non nullo (interpretato come "Vero" o 1), la transazione è considerata crittograficamente valida e può essere propagata nella rete e inclusa in un blocco. In caso contrario, è scartata.

8.1.2 Multi Signatures con Threshold

Come abbiamo già visto, Bitcoin permette di spendere fondi provenienti da più UTXO a patto che si possa provare, tramite apposita firma, la proprietà di ognuno di essi. Abbiamo definito questo caso come un **pagamento congiunto** poiché, in termini pratici, gli UTXO possono appartenere a persone diverse e servono quindi le firme di tutti i coinvolti per autorizzare l'operazione.

Un caso particolare è il sistema di **Multi Signatures con Threshold**. Tramite questa operazione, facciamo sì che un singolo UTXO sia vincolato a un gruppo di chiavi pubbliche, ma possa essere speso fornendo solo un

numero minimo di firme (la soglia o *threshold*) rispetto al totale delle chiavi presenti. Ad esempio, una tipica multi-signature potrebbe elencare 5 chiavi pubbliche nell'UTXO, ma per spendere i fondi ne basterebbero solo 3, indipendentemente da quali siano i firmatari tra i 5 autorizzati.

Esiste inoltre la possibilità che un UTXO richieda la firma di tutti i suoi proprietari per poter essere speso. Possiamo definire questa situazione come un caso specifico di **Multi Signature con threshold N-su-N**, dove per muovere i fondi è necessaria la firma di tutti coloro che possiedono le chiavi associate all'UTXO, senza alcuna eccezione.

È chiaro che in questo scenario il rischio aumento poiché se anche una sola di queste chiavi viene persa, o se uno dei partecipanti si rifiuta di collaborare, l'UTXO diventa permanentemente bloccato e i fondi non potranno più essere recuperati in alcun modo.

8.1.3 Proof-of-Burn

La **Proof-of-Burn** è un'operazione mediante la quale è possibile letteralmente **buttare soldi**. Esistono principalmente due modi per farlo: il primo consiste nell'inviare i fondi a un cosiddetto **burn address**, ovvero un indirizzo di cui matematicamente nessuno possiede (o può generare) la chiave privata; il secondo è l'utilizzo dell'operazione *OP_RETURN*, che permette di contrassegnare un output come non spendibile a priori, consentendo però di aggiungere una piccola quantità di dati alla blockchain.

Questa è una pratica che nel contesto puro di Bitcoin ha poco senso, poiché ha come unico effetto quello di distruggere valuta. Tuttavia, viene utilizzata per scopi specifici come il **bootstrap** di una nuova moneta (dimostrando di aver "sacrificato" valore reale per ottenerla) o per il **timestam-ping**, ovvero usare la blockchain come un registro notarile per certificare l'esistenza di un dato in una certa data.

8.1.4 Timestamping

Il sistema di **Timestamping** permette di utilizzare la blockchain di Bitcoin come un registro notarile immutabile. Attraverso l'inserimento dell'hash di un documento all'interno di una transazione (spesso sfruttando l'operazione *OP_RETURN*), è possibile dimostrare con certezza assoluta che tale file esisteva già in una specifica data, certificata dal timestamp del blocco che contiene la transazione.

Inoltre, la flessibilità dello Script consente l'inserimento di quelli che vengono chiamati **Easter Eggs**: messaggi, citazioni o dati arbitrari nascosti all'interno della catena. Questa pratica, iniziata da Satoshi Nakamoto

con il messaggio nel blocco genesi, serve a lasciare una traccia indelebile di informazioni non finanziarie direttamente nel registro distribuito.

8.1.5 Real-life transaction con Escrow

Un problema classico derivante dal sistema Bitcoin è quello dei pagamenti online tra parti che non si fidano l'una dell'altra. Supponiamo che *A* voglia acquistare un bene da *B*: *A* non vuole rischiare di inviare i soldi e non ricevere nulla, mentre *B* non vuole spedire l'oggetto senza la garanzia di essere pagato. Questa situazione di stallo è nota in computer security come il problema dell'**Equa compravendita**. Per risolvere questa impasse, introduciamo una terza parte fidata, un **trusted third party** che chiameremo Giudice (*J*). La soluzione consiste nel creare una transazione con **multi signature a threshold 2-su-3** tra *A*, *B* e *J*. In questo modo i fondi vengono "bloccati" in un UTXO che richiede almeno due firme per essere speso:

- **Caso standard:** Se l'ordine arriva correttamente, *A* e *B* firmano insieme e i soldi vanno a *B*. Il giudice non interviene.
- **Caso di disputa:** Se nasce un problema, il giudice *J* analizza le prove. Se dà ragione ad *A*, firmerà insieme ad *A* per restituirgli i soldi (reso); se dà ragione a *B*, firmerà insieme a *B* per completare il pagamento.

L'aspetto fondamentale è che il giudice *J*, pur avendo il potere di decidere l'esito della disputa, non può mai scappare con i soldi da solo, poiché la soglia di firme richiesta è 2 e lui ne possiede soltanto una.

8.1.6 Micro-pagamenti e Lightning Network

Supponiamo che *A* e *B* vogliano scambiarsi pagamenti frequenti, ad esempio per un servizio di chiamata tariffato al minuto. Effettuare una transazione sulla blockchain per ogni singolo minuto sarebbe insostenibile: i tempi di conferma sono lunghi e le commissioni (fees) per i miner supererebbero probabilmente il valore del pagamento stesso. Per risolvere questo problema, sfruttiamo il **lock_time** combinato con una **multi-signature 2-su-2**. Il processo si divide in tre fasi:

1. **Apertura del canale:** *A* prepara una transazione di finanziamento (funding) che blocca una somma in un output richiedente le firme di entrambi (*A* e *B*). **Attenzione:** prima di pubblicare questa transazione, *A* si fa firmare da *B* una transazione di **rescue** (con un *lock_time* futuro) che gli restituirebbe l'intero importo se il canale dovesse bloccarsi.

2. **Scambio off-chain:** Durante la chiamata, *A* invia a *B* delle "micro-transazioni" cumulative. Queste transazioni sono firmate solo da *A*, quindi non sono ancora valide per la rete. Ad ogni minuto, *A* aggiorna la transazione assegnando a *B* una quota maggiore e a se stesso (come resto) una quota minore.
3. **Chiusura:** Alla fine della chiamata, *B* prende l'ultima transazione ricevuta (quella con l'importo totale corretto), appone la sua firma e la pubblica sulla blockchain.

B è tranquillo perché ha in mano una transazione che può riscuotere in qualsiasi momento. *A* è tranquillo perché, se *B* dovesse sparire per non farlo cooperare, basterebbe attendere la scadenza del *lock_time* per riprendersi i soldi tramite la transazione di rescue. Una volta che la transazione finale viene pubblicata da *B*, quella di rescue diventa automaticamente nulla, poiché tenterebbe di spendere un UTXO che è già stato consumato (evitando così il *double-spending*).

8.1.7 Freezing Funds

Supponiamo che *A* voglia mettere da parte una certa somma di Bitcoin per il futuro, ad esempio come fondo di risparmio per un figlio, oppure che voglia semplicemente "auto-imporsi" di non spendere i propri fondi per un determinato periodo di tempo. In una transazione standard, chiunque possieda la chiave privata può muovere i fondi in qualsiasi momento; tuttavia, Bitcoin permette di "congelare" letteralmente un UTXO grazie all'istruzione *OP_CHECKLOCKTIMEVERIFY*. Tramite questa operazione, facciamo sì che l'output della transazione non sia solo vincolato a una firma, ma anche a una condizione temporale specifica (espressa in data o in altezza del blocco). In pratica, lo script dell'UTXO dirà: "Questi soldi possono essere spesi solo se viene fornita la firma di *A* **E** se il tempo trascorso è superiore a *X*". Naturalmente, questa condizione è scritta sulla blockchain e verificata dai nodi, il che la rende inviolabile: anche se *A* possiede la chiave privata, se prova a spendere i fondi prima della scadenza, la rete rifiuterà la transazione considerandola invalida. Questa funzione è fondamentale non solo per il risparmio a lungo termine, ma anche per la sicurezza di protocolli complessi. Come abbiamo visto nei micro-pagamenti, il sistema di "congelamento" permette di creare delle vie d'uscita (le transazioni di *rescue*): se una controparte smette di collaborare, i fondi non restano bloccati per sempre nel limbo della multi-signature, ma ritornano automaticamente al proprietario originale una volta che il "tempo di blocco" è scaduto.

8.1.8 Pay-to-Script-Hash (P2SH)

Tutte le funzionalità che abbiamo analizzato finora, come le multi-signature o i sistemi di `lock_time`, hanno un "difetto" naturale: rendono lo script di pagamento estremamente lungo e complesso. Se un venditore B chiedesse a un acquirente A di pagare verso uno script così complicato, A si ritroverebbe a dover inserire una mole enorme di dati nell'output della transazione, pagando di conseguenza commissioni molto più alte ai miner.

Per risolvere questo problema, utilizziamo il sistema **Pay-to-Script-Hash (P2SH)**. In questo scenario, immaginiamo che gli script complessi siano gestiti interamente dal venditore B. Invece di inviare a A l'intero set di istruzioni, B crea prima lo script (chiamato *redeem script*), ne calcola l'**hash** e invia solo quest'ultimo ad A.

Questa procedura garantisce diversi vantaggi:

- **Efficienza:** Lo *scriptPubKey* rimane molto piccolo perché contiene solo l'hash dello script. Questo rende la transazione d'acquisto economica per A, indipendentemente dalla complessità delle regole imposte da B.
- **Sicurezza:** Il venditore deve generare l'hash e "pubblicarlo" (fornendolo ad A) prima del pagamento. In questo modo, B non può più modificare le condizioni dello script per provare a "barare", poiché se cambiasse anche una sola virgola dello script originale, l'hash non corrisponderebbe più e i fondi risulterebbero bloccati.

In pratica, i soldi rimangono bloccati nell'hash finché B non decide di spenderli. Solo in quel momento, all'interno dello *scriptSig*, B dovrà fornire sia lo script completo che tutte le firme necessarie per farlo girare. La rete Bitcoin verificherà prima che lo script corrisponda all'hash e poi eseguirà le istruzioni per autorizzare il pagamento.

Capitolo 9

La Blockchain

Una volta che le transazioni vengono create e firmate tramite la logica degli script, esse devono essere raggruppate e "sigillate" all'interno di un **blocco**. La blockchain non è altro che una catena sequenziale di questi contenitori, dove ogni blocco garantisce l'immutabilità dei dati che contiene.

9.1 Il Blocco

Ogni blocco è identificato da un **Header** (intestazione), che possiamo immaginare come la carta d'identità del blocco stesso. Esso contiene i metadati fondamentali per la sicurezza della rete:

- **Hash del blocco precedente (*prev_block*):** È il legame che crea la catena. Ogni blocco contiene l'impronta digitale di quello precedente; se qualcuno provasse a modificare una transazione in un blocco passato, l'hash cambierebbe, rendendo istantaneamente invalidi tutti i blocchi successivi.
- **Nonce e Bits:** Sono parametri legati al mining. Il *nonce* è il numero che i miner continuano a variare per trovare un hash che soddisfi la difficoltà della rete (*bits*), confermando così il blocco tramite la Proof-of-Work.
- **Merkle Root (*mrkl_root*):** Una stringa che riassume in modo univoco tutte le transazioni presenti nel blocco

9.1.1 Merkle Tree e Proof of Membership

Per gestire i dati delle transazioni in modo efficiente, Bitcoin non le elenca semplicemente una dopo l'altra, ma utilizza una struttura chiamata **Merkle**

Tree (albero di Merkle). Si tratta di un albero binario in cui si parte dagli hash delle singole transazioni alla base e, accoppiandoli, si sale di livello calcolando nuovi hash fino ad arrivare alla radice (**Merkle Root**). Questa struttura è fondamentale per la cosiddetta **Proof of Membership**. Supponiamo che un utente *A* utilizzi un wallet "leggero" sul suo smartphone. Grazie al Merkle Tree, *A* non deve scaricare tutte le migliaia di transazioni del blocco, ma gli basta ricevere solo una piccola serie di hash intermedi. Combinando questi hash con quello della propria transazione, *A* può ricalcolare la Merkle Root: se il risultato coincide con quello presente nell'header del blocco, allora ha la certezza matematica che la sua transazione è stata confermata. Infatti, non tutti i partecipanti alla rete scaricano l'intera blockchain. Esistono due tipologie di nodi:

- **Full Node:** Sono i nodi che contengono l'intera cronologia delle transazioni e verificano ogni singola regola del protocollo. Sono "pesanti" in termini di memoria ma garantiscono la massima sicurezza. I miner possono solamente essere dei nodi di questo tipo.
- **Lightweight Node (o nodi SPV):** Sono nodi che non contengono tutte le transazioni, bensì conservano solo i **Block Header**. Quando questi nodi hanno necessità di verificare l'esistenza di una certa transazione, interrogano i Full Node chiedendo loro di inviargli i dati necessari (il "cammino" nell'albero di Merkle) per effettuare la Proof of Membership.

9.1.2 Coinbase transaction

Fino ad ora abbiamo visto come *A* possa inviare Bitcoin a *B* partendo da fondi già esistenti (UTXO). Ma come entrano in circolazione i nuovi Bitcoin se non esiste una banca centrale? La soluzione è la **Coinbase Transaction**, una transazione "speciale" che deve essere presente obbligatoriamente come prima operazione di ogni singolo blocco. Questa transazione ha tre caratteristiche fondamentali:

- **Emissione Monetaria:** È l'unico modo previsto dal protocollo per **creare nuovi Bitcoin**. Non avendo un input che proviene da una transazione precedente, essa "genera" valuta dal nulla seguendo una regola matematica precisa.
- **Remunerazione dei Miner:** Rappresenta il guadagno principale per chi mette in sicurezza la rete. Il miner che riesce a chiudere il blocco assegna a se stesso una somma composta da:

1. Il **Block Subsidy** (il premio per il blocco).
2. Le **Fees** (le commissioni) di tutte le transazioni incluse nel blocco.

Il premio iniziale era di 50 BTC, ma ogni 210.000 blocchi (circa 4 anni) questa cifra viene dimezzata tramite un evento chiamato **Halving**. Come si vede nell'immagine, nel 2020 il premio è sceso a 6.25 BTC e dall'aprile 2024 è passato a **3.125 BTC**.

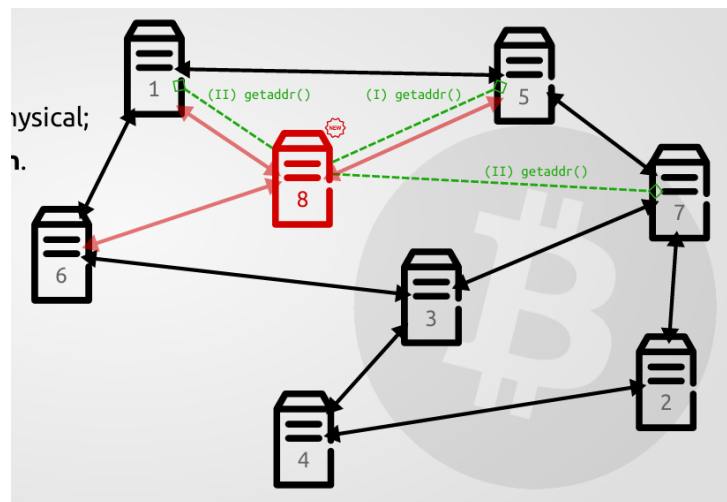
- **Il campo Coinbase:** A differenza delle transazioni normali, l'input della Coinbase Transaction non punta a un UTXO precedente (infatti il campo *hash* è composto da soli zeri). Al suo posto troviamo un campo libero chiamato *coinbase*, dove il miner può inserire dati arbitrari. L'esempio più famoso è quello di Satoshi Nakamoto nel Blocco 0, dove inserì il titolo del quotidiano *The Times* per denunciare l'instabilità del sistema bancario tradizionale.

9.2 Il Network Bitcoin

Dopo aver analizzato la struttura dei blocchi, è fondamentale capire come le informazioni (transazioni e blocchi) viaggino tra i partecipanti. Bitcoin non si appoggia a un server centrale, ma utilizza una rete **P2P ad-hoc** dove tutti i nodi sono considerati uguali.

La rete Bitcoin presenta alcune proprietà fondamentali che ne garantiscono la resilienza:

- **Connessioni TCP:** I nodi comunicano solitamente sulla porta 8333.
- **Topologia Logica vs Fisica:** La rete è organizzata in modo "random". Un nodo in Italia potrebbe essere logicamente connesso a un nodo in Giappone; la struttura logica non rispecchia quella geografica.
- **Discovery e Oblivion:** Quando un nuovo nodo entra nella rete (come il nodo 8 nell'immagine), deve "scoprire" i suoi vicini. Lo fa inviando messaggi di *getaddr()* per ottenere liste di indirizzi IP di altri nodi attivi. Allo stesso modo, se un nodo smette di rispondere, viene dimenticato dalla rete (*oblivion*).



9.2.1 Gossip Protocol

Quando un utente A crea una transazione per pagare B , questa non finisce immediatamente nella blockchain. Deve prima essere propagata a tutti i nodi della rete tramite un **Gossip Protocol** (o *Flooding*). Il funzionamento è simile al diffondersi di un pettegolezzo: A invia la transazione ai propri vicini, questi la verificano e, se valida, la inviano ai propri vicini, e così via fino a coprire l'intera rete in pochi secondi.

La Transaction Pool e i Controlli Standard

Ogni singolo nodo mantiene in memoria una "vasca" temporanea chiamata **Transaction Pool** (o *Mempool*), dove conserva le transazioni ricevute che sono in attesa di essere inserite in un blocco dai miner. Prima di accettare una transazione nella propria pool e inoltrarla agli altri, ogni nodo effettua dei **controlli standard**:

1. **Validità:** La transazione deve avere firme corrette e formattazione valida.
2. **Script Whitelist:** Lo script deve rientrare tra quelli considerati "standard".
3. **Unicità:** La transazione non deve essere già presente nella pool.
4. **Assenza di conflitti:** La transazione non deve tentare di spendere UTXO già impegnati da altre transazioni presenti nella pool (evitando il double spending immediato).

Poiché la rete è distribuita e la propagazione richiede tempo, le Mempool dei vari nodi non sono sempre identiche (*out-of-sync*). Se due transazioni in conflitto (ad esempio *A* prova a mandare gli stessi soldi sia a *C* che a *B* contemporaneamente) si diffondono nella rete, si crea una **Race Condition**. In questo caso vale la regola del **"the first win!"**: ogni nodo terrà in memoria solo la prima transazione che riceve e scarnerà la seconda. Sarà poi il miner, scegliendo quale includere nel prossimo blocco, a decretare quale delle due transazioni diventerà definitiva. Esiste però un'eccezione chiamata **Replace-By-Fee (RBF)**, che permette a un utente di "sovrascrivere" una propria transazione in attesa con una nuova che offra una mancia (*fee*) più alta ai miner.

9.2.2 Propagazione dei Blocchi

Allo stesso modo di come funziona la diffusione di nuove transazioni, avviene la propagazione dei nuovi blocchi.

QUI i controlli sono di diverso tipo, un blocco viene inoltrato solo se:

- È stato risolto il **puzzle crittografico**.
- Contiene solo **transazioni valide**.
- È costruito sulla **catena più lunga** evitando, così, fork.

Quando un miner scopre un nuovo blocco valido, lo trasmette immediatamente ai suoi peer. Ogni nodo che lo riceve verifica la sua validità e, se soddisfa tutti i requisiti, lo inoltra ai propri peer e lo aggiunge alla propria copia locale della blockchain. Questo meccanismo di propagazione a cascata garantisce che i blocchi validi si diffondano rapidamente attraverso l'intera rete.

Questi controlli assicurano che solo blocchi legittimi vengano incorporati nella blockchain, prevenendo attacchi e divergenze non intenzionali.

9.2.3 Tipologie di Nodi

Come già accennato precedentemente esistono due tipi di nodi:

- I **Full Nodes**;
- I **Lightweight Node**;

Per quanto riguarda i full node, questi possiedono le seguenti caratteristiche:

- **Connettività permanente:** Mantengono una presenza costante nella rete per massimizzare la resilienza del sistema.
- **Archiviazione completa:** Conservano l'intera blockchain, comprensiva di tutte le transazioni dalla genesi (circa 460 GB nel 2023).
- **Verifica esaustiva:** Validano indipendentemente ogni transazione e blocco secondo tutte le regole di consenso.
- **Ottimizzazione UTXO:** Mantengono una cache del set UTXO in memoria RAM per accelerare le verifiche delle transazioni.

Questi nodi forniscono il livello massimo di sicurezza e indipendenza, in quanto non necessitano di fidarsi di terze parti per la validazione delle transazioni che li riguardano. Rappresentano la manifestazione concreta del principio **don't trust, verify** fondamentale in Bitcoin.

La distribuzione geografica e numerica dei full node ha implicazioni significative per la resilienza della rete. Una rete con molti nodi completi distribuiti globalmente offre maggiore resistenza a censura, interruzioni localizzate e tentativi di manipolazione.

I Lightweight node, invece offrono un'alternativa più leggera rispetto ai full node così da rendere possibile la connessione a dispositivi come smartphone o computer con potenza limitata. Possiedono le seguenti caratteristiche:

- **Archiviazione selettiva:** Memorizzano solo gli header dei blocchi (circa 460 MB vs. 460 GB), riducendo drasticamente i requisiti di storage.
- **Verifica mirata:** Verificano solo le transazioni di diretto interesse per l'utente, anziché l'intera blockchain.
- **Dipendenza dai full node:** Richiedono la connessione a full node per ottenere dati sulle transazioni specifiche e le relative prove di inclusione.
- **Verifica basata su Merkle proof:** Utilizzano prove crittografiche compatte basate sull'albero di Merkle per confermare l'inclusione di transazioni in blocchi validi.

I nodi SPV offrono un equilibrio tra efficienza e sicurezza, permettendo la partecipazione alla rete con risorse limitate, ma accettando un modello di fiducia delegata verso i full node per alcune verifiche.

9.3 Limiti Strutturali di Bitcoin

L'architettura di Bitcoin impone dei vincoli intrinseci, derivanti dalle scelte progettuali originarie, che ne definiscono il perimetro operativo e la sicurezza, ma ne limitano al contempo la flessibilità e la scalabilità. I principali parametri che determinano questi limiti sono:

- **Intervallo tra blocchi:** Il protocollo è calibrato per generare un nuovo blocco mediamente ogni 10 minuti. Questo equilibrio è mantenuto attraverso l'aggiustamento dinamico della difficoltà di mining, necessario per garantire una propagazione sicura dei blocchi ed evitare un eccesso di blocchi orfani.
- **Dimensione dei blocchi:** Storicamente limitata a 1 MB per prevenire attacchi di tipo DoS e mantenere bassi i requisiti per i full node. Sebbene aggiornamenti come SegWit abbiano introdotto il concetto di *block weight*, aumentando la capacità effettiva, lo spazio rimane una risorsa scarsa e contesa.
- **Limite alle operazioni crittografiche:** Esiste un tetto massimo di circa 20.000 verifiche di firma (*sigops*) per singolo blocco. Questo parametro funge da limite pratico alla complessità computazionale richiesta per validare un blocco, proteggendo la rete da transazioni eccessivamente onerose.
- **Politica Monetaria:** L'offerta è rigidamente fissata a un massimo di 21 milioni di bitcoin. L'emissione è regolata da uno schema di ricompensa predeterminato che prevede il dimezzamento del sussidio (*halving*) circa ogni quattro anni.

La combinazione di questi vincoli si traduce in un *throughput* massimo teorico di circa 7 transazioni al secondo (TPS). Tale valore appare estremamente ridotto se confrontato con i circuiti di pagamento centralizzati come Visa o Mastercard, capaci di gestire decine di migliaia di transazioni nello stesso intervallo temporale.

Un ulteriore limite è rappresentato dalla **rigidità delle primitive crittografiche**. Bitcoin si affida a standard consolidati come **ECDSA** (per le firme) e **SHA-256** (per l'hashing). Sebbene attualmente sicuri, la loro eventuale sostituzione — necessaria, ad esempio, in uno scenario di minaccia quantistica — richiederebbe modifiche strutturali al protocollo. Tali aggiornamenti, data la natura decentralizzata della rete, comporterebbero il rischio di *hard fork* o *soft fork* complessi, evidenziando la difficoltà di far evolvere un sistema che privilegia la stabilità sopra ogni altra caratteristica.

9.3.1 Evoluzione del Protocollo: Hard Fork e Soft Fork

Nonostante questi vincoli strutturali, Bitcoin dispone di meccanismi che permettono l'evoluzione del protocollo bilanciando l'esigenza di stabilità con l'esigenza di adattamento e miglioramento.

Queste modifiche possono essere implementate principalmente in due modi:

- **Hard Fork:** Rappresenta un cambiamento non retro compatibile che richiede l'aggiornamento di tutti i nodi. Questo comporterà il fatto che i vecchi blocchi **non accetteranno i nuovi blocchi** creando, dunque, un partizionamento tra i nodi che li accettano e i nodi che non li accettano.
- **Soft Fork:** Implementa modifiche retrocompatibili che restringono o precisano regole già esistenti. I blocchi saranno, dunque, **validi anche per nodi non aggiornati**. Il rischio qui è che i vecchi nodi producano ancora blocchi non conformi

La comunità Bitcoin ha storicamente privilegiato soft fork per minimizzare i rischi di frammentazione, utilizzando tecniche di progettazione che permettono significative innovazioni mantenendo la retro compatibilità.

9.4 Conservare Bitcoin

Come sappiamo, Bitcoin è implementato attraverso un sistema di **Crittografia Asimmetrica** in cui abbiamo:

- **Chiave privata:** Sequenza numerica segreta che permette di generare firme digitali valide per autorizzare transazioni. Rappresenta di fatto il pieno controllo sui fondi.
- **Chiave pubblica:** Derivata matematicamente dalla chiave privata attraverso operazioni sulla curva ellittica secp256k1.
- **Indirizzo Bitcoin:** Hash della chiave pubblica, utilizzato come "destinatario" nelle transazioni per ragioni di compattezza e sicurezza aggiuntiva.

Per possedere dei Bitcoin è necessario dimostrare di possedere la chiave privata associata a degli specifici **uTXO**. Quello che avremo dunque bisogno di conservare è proprio questa chiave privata.

9.4.1 Wallet

I wallet Bitcoin sono software o dispositivi hardware progettati per la gestione delle chiavi private e la facilitazione delle operazioni crittografiche necessarie alla rete. È fondamentale sottolineare che un wallet **non "contiene" fisicamente i bitcoin** i quali esistono esclusivamente come registrazioni contabili sulla blockchain ma funge da custode delle credenziali digitali indispensabili per disporne. Un wallet completo integra diverse funzionalità core e gestisce il paradigma della pseudonimia attraverso gli indirizzi.

L'operatività di un wallet si articola su diversi pilastri tecnici e metodologie di gestione della privacy:

- **Gestione Crittografica e Firme:** Il wallet archivia in modo sicuro le chiavi private e le utilizza per la **firma delle transazioni**, l'unico meccanismo che autorizza il trasferimento di fondi verso altri indirizzi.
- **Generazione e Formato degli Indirizzi:** Il software crea nuovi indirizzi Bitcoin, solitamente presentati in formato **base58** con l'inclusione di un *checksum* per prevenire errori di trascrizione. Per scopi estetici o identificativi, è possibile generare **indirizzi vanity**, ovvero indirizzi personalizzati che iniziano con una sequenza di caratteri specifica (es. 10DaVePP20...), ottenuti tramite ricerca esaustiva.
- **Monitoraggio degli UTXO:** Attraverso l'interrogazione costante della blockchain, il wallet tiene traccia degli **Unspent Transaction Outputs** (UTXO) disponibili, permettendo all'utente di conoscere in tempo reale il proprio saldo spendibile.
- **Ottimizzazione delle Commissioni:** Il wallet stima e suggerisce le commissioni (*fees*) più appropriate in base alla congestione attuale della rete, bilanciando velocità di conferma e costo.
- **Privacy e Pseudonimia:** Gli indirizzi svolgono un ruolo chiave nella protezione dell'identità. L'**uso multiplo**, ovvero la pratica di generare un nuovo indirizzo per ogni transazione ricevuta, rafforza significativamente la pseudonimia del sistema, rendendo estremamente complessa l'analisi dei flussi finanziari e il tracciamento delle connessioni tra i diversi attori.

9.4.2 Strategie di Conservazione: Hot Storage e Cold Storage

Nella gestione delle criptovalute, la dicotomia tra accessibilità e sicurezza viene risolta attraverso l'adozione di due paradigmi complementari: l'**hot storage** e il **cold storage**. Questa strategia riproduce metaforicamente la distinzione tra un conto corrente bancario, destinato alla conservazione del patrimonio, e un portafoglio fisico, utilizzato per le transazioni quotidiane.

Hot Storage

L'hot storage identifica sistemi di conservazione le cui chiavi private risiedono in ambienti costantemente **connessi alla rete**, come smartphone, computer desktop o servizi di wallet online.

- **Vantaggi:** Estrema facilità d'uso e immediatezza nell'esecuzione di pagamenti e scambi.
- **Rischi:** Elevata esposizione ad attacchi informatici, malware, phishing e vulnerabilità del software di sistema. È consigliabile mantenere in hot storage solo frazioni limitate di capitale necessarie all'operatività corrente.

Cold Storage

Il cold storage è progettato per la conservazione a lungo termine di ingenti capitali, minimizzando la superficie di attacco attraverso l'**isolamento fisico** dalla rete (*Air-gapping*). Oltre ai dispositivi offline già citati, la gestione del cold storage può avvalersi di diversi supporti e tecniche di derivazione:

- **Hardware e Tamper-proof Wallet:** Dispositivi fisici specializzati (come smart card o moduli HSM) dotati di chip resistenti alla manomissione (*tamper-proof*). Questi generano e conservano le chiavi in un elemento sicuro (*Secure Element*), impedendo l'esportazione della chiave privata anche in caso di accesso fisico forzato.
- **Paper Wallet e Supporti Esterni:** La stampa fisica delle chiavi su carta/metallo o l'archiviazione in **memorie USB isolate**. Questi supporti sono immuni agli attacchi remoti ma vulnerabili al deterioramento fisico o al furto.
- **Brain Wallet:** Una metodologia in cui la chiave privata non viene memorizzata su un supporto fisico, ma derivata da una password (*passphrase*) memorizzata esclusivamente nella mente dell'utente.

Generazione e Sicurezza delle Chiavi Partendo da una password (*pwd*), la chiave privata x viene ottenuta tramite una funzione di hash, mentre la chiave pubblica y è il punto sulla curva ellittica:

$$x = \text{hash}(\text{pwd}) \implies y = G^x$$

Poiché il nostro cervello non è in grado di generare informazioni con molta entropia, questo approccio introduce un rischio critico di **Password Cracking**. Un attaccante può eseguire attacchi *offline* massivi (global attacks) testando milioni di password comuni per vedere se corrispondono a indirizzi con fondi sulla blockchain.¹

Per mitigare questa vulnerabilità, si utilizzano due tecniche fondamentali:

- **Key Sequence Generation:** La generazione deterministica di una sequenza di chiavi (come visto negli HD Wallet) per evitare il riutilizzo degli indirizzi.
- **Key Stretching:** L'applicazione iterativa di funzioni di hash o l'uso di algoritmi *memory-hard* (come PBKDF2 o Argon2). Questa tecnica "allunga" il tempo computazionale necessario per ogni singolo tentativo di hash, rendendo gli attacchi di brute-force economicamente e temporalmente insostenibili per l'attaccante.

Il processo di firma Air-gapped

Per massimizzare la sicurezza senza rinunciare alla funzionalità, si adotta spesso un protocollo di firma in tre fasi che garantisce che la chiave privata non entri mai in contatto con un ambiente online:

1. **Preparazione:** La transazione viene creata su un dispositivo online (che osserva la blockchain ma non possiede le chiavi).
2. **Firma Offline:** La transazione non firmata viene trasferita al dispositivo in *cold storage* (tramite USB, scheda SD o QR code). Qui viene apposta la firma digitale in un ambiente isolato.
3. **Trasmissione:** La transazione firmata viene riportata sul dispositivo online e trasmessa (*broadcasted*) alla rete Bitcoin.

¹Si usano i classici attacchi brute force o a dizionario.

Hierarchical Deterministic (HD) Wallet

Una delle migliori prassi nel protocollo Bitcoin è l'**evitare il riutilizzo degli indirizzi** (*address reuse*). Se un utente inviasse tutti i propri risparmi a un unico indirizzo di *cold storage*, un osservatore della blockchain potrebbe facilmente raggruppare (*clustering*) tutte le transazioni in entrata, riconducendole a un'unica entità e compromettendo la privacy del patrimonio complessivo.

Per risolvere questo problema senza costringere l'utente a gestire migliaia di backup diversi, si utilizzano i **Wallet Gerarchici Deterministici (HD Wallet)**. L'idea alla base è la generazione di un intero albero di chiavi a partire da un unico segreto primario, il **Master Seed** (solitamente rappresentato da una sequenza di 12 o 24 parole mnemoniche).

Meccanismo di Derivazione Il sistema utilizza una funzione di derivazione che combina una chiave e un valore denominato **Chain Code** (k), il quale funge da ulteriore entropia. Il processo permette di separare la generazione delle chiavi private da quella degli indirizzi pubblici:

- **Derivazione delle Chiavi Private:** Permette di generare la i -esima chiave figlia partendo dalla chiave privata master.
- **Derivazione delle Chiavi Pubbliche:** Permette di generare l' i -esimo indirizzo pubblico partendo solo dalla chiave pubblica master (senza mai esporre la chiave privata).

Modello Matematico (ECDSA) In un sistema basato su curve ellittiche, la derivazione di una chiave figlia x_i e della relativa chiave pubblica P_i a partire dai parametri master (y, G^y) e dal chain code k avviene come segue:

- **Chiave privata i -esima:** $x_i = (y + \text{hash}(k \parallel i)) \pmod{n}$
- **Chiave pubblica i -esima:** $P_i = G^{x_i} = G^y \cdot G^{\text{hash}(k \parallel i)}$
- **Indirizzo i -esimo:** $A_i = \text{hash}(P_i)$

Sicurezza e Gerarchia Questo schema offre vantaggi cruciali in termini di resilienza e gestione del rischio:

- **Isolamento delle chiavi:** La compromissione della chiave privata di un singolo indirizzo figlio non compromette la chiave master né le chiavi "sorelle".

- **Backup unico:** È sufficiente conservare il *Master Seed* per poter rigenerare l'intero albero di chiavi in caso di smarrimento del dispositivo.
- **Gerarchia di accesso:** È possibile fornire a un terzo (es. un consulente o un software di monitoraggio) la sola chiave pubblica master (*Extended Public Key* o *xpub*). Questo permette di generare tutti gli indirizzi pubblici per monitorare il saldo senza che il terzo possa mai spendere i fondi, poiché non possiede la chiave privata master.

Tuttavia, la perdita della **Master Private Key** comporta la perdita totale e irreversibile di tutti i fondi contenuti nell'intera struttura gerarchica.

Derivazione Indurita e Sicurezza Un aspetto cruciale della specifica BIP-32 è la distinzione tra derivazione normale e **indurita (hardened)**. Se un attaccante compromettesse una chiave pubblica estesa (*xpub*) e una chiave privata figlia derivata normalmente, potrebbe ricostruire la chiave privata master. Per prevenire questo scenario, si utilizza la derivazione indurita (per indici $i \geq 2^{31}$), che utilizza la chiave privata del genitore come input per la funzione HMAC-SHA512, interrompendo la relazione matematica tra le chiavi pubbliche e garantendo che la compromissione di un ramo non risalga mai alla radice.

9.5 Algoritmo di Firma Digitale ECDSA

ECDSA rappresenta l'algoritmo di firma digitale utilizzato in Bitcoin per autenticare le transazioni. Esso garantisce che solo il legittimo proprietario di una chiave privata possa autorizzare il movimento dei fondi, assicurando l'integrità e l'originalità del messaggio (transazione).

L'implementazione di ECDSA nel protocollo Bitcoin si basa sulla specifica curva ellittica **secp256k1**. I componenti fondamentali sono:

- **Parametri Pubblici:** Includono il punto generatore G della curva e l'ordine q del gruppo.
- **Chiave Privata:** Un intero x scelto casualmente nell'intervallo $\{1, \dots, q-1\}$. Deve rimanere segreto.
- **Chiave Pubblica:** Il punto $Y = G^x$ sulla curva ellittica, ottenuto attraverso la moltiplicazione scalare del generatore per la chiave privata.

9.5.1 Processo di Firma

La generazione di una firma per un messaggio (o hash di una transazione) m segue i seguenti passaggi:

1. **Selezione della randomness:** Si sceglie un numero casuale k (denominato *nonce*) nell'intervallo $\{1, \dots, q-1\}$.
2. **Calcolo del punto di impegno:** Si calcola il punto $R = G^k$ sulla curva.
3. **Estrazione di r :** Si estrae l'ascissa r del punto R . Questo rappresenta la prima componente della firma.
4. **Calcolo di s :** Si calcola la seconda componente come:

$$s = k^{-1}(m + r \cdot x) \pmod{q}$$

5. **Output:** La firma completa è la coppia (r, s) .

Importanza del parametro k

La sicurezza di ECDSA dipende criticamente dalla gestione del valore casuale k . Esso deve essere:

- Generato con **elevata entropia** (imprevedibile).
- **Utilizzato una sola volta** (*never reused*).
- Mantenuto **segreto**.

Nota storica: La riutilizzazione o la prevedibilità di k ha portato a fallimenti catastrofici in passato (come nei casi di PlayStation 3 e di alcune vulnerabilità Android), permettendo ad attaccanti di derivare la chiave privata x con semplici operazioni algebriche.

Processo di Verifica

Qualsiasi nodo della rete può verificare la validità di una firma (r, s) rispetto a un messaggio m e alla chiave pubblica Y :

1. Si calcola l'hash m del messaggio originale.
2. Si calcolano i coefficienti:

$$u_1 = m \cdot s^{-1} \pmod{q} \quad \text{e} \quad u_2 = r \cdot s^{-1} \pmod{q}$$

3. Si ricostruisce il punto R sulla curva:

$$R = G^{u_1} + Y^{u_2}$$

4. Si verifica che l'ascissa del punto R così ottenuto sia congruente a $r \pmod{q}$.

Se la condizione è soddisfatta, la firma è valida. Poiché questo processo richiede solo informazioni pubbliche (Y, m, r, s) , la verifica è un'operazione completamente decentralizzata che chiunque può eseguire autonomamente.

9.5.2 Secret Sharing e Threshold Cryptography

La gestione sicura delle chiavi private può beneficiare di tecniche di condivisione del segreto, che permettono di distribuire il rischio tra più componenti, evitando che la compromissione di un singolo elemento esponga l'intero sistema.

9.5.3 Principi del Secret Sharing

Il *secret sharing* è una tecnica crittografica che permette di dividere un segreto S (come una chiave privata) in n parti, dette **shares**, in modo che:

- Il segreto originale possa essere ricostruito solo combinando un numero sufficiente di shares.
- Nessuna informazione sul segreto sia rivelata analizzando un numero insufficiente di esse.

Le implementazioni elementari includono:

- **XOR sharing:** $S = S_1 \oplus S_2$. Il segreto è diviso in due parti; entrambe sono necessarie per riprodurre S tramite l'operazione XOR.
- **Additive sharing:** $S = S_1 + S_2 + \dots + S_n \pmod{p}$. Il segreto è la somma modulo p di tutte le n componenti.

9.5.4 Threshold Secret Sharing (Shamir's Scheme)

Lo schema di Shamir (t -out-of- n) rappresenta un'evoluzione che permette di specificare un **numero minimo** t (**soglia**) di shares necessario per la ricostruzione.

Il meccanismo si basa sulle proprietà dei polinomi:

- **Costruzione:** Il segreto S viene incorporato come termine costante di un polinomio di grado $t - 1$:

$$P_S(x) = S + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p}$$

dove i coefficienti a_i sono scelti casualmente.

- **Distribuzione:** Vengono generate n chiavi come coppie $(i, s_i = P_S(i))$ per $i \in [1, n]$.
- **Ricostruzione:** Il segreto può essere recuperato attraverso l'**interpolazione polinomiale di Lagrange** usando almeno t shares:

$$S = P_S(0) = \sum_{j=1}^t \lambda_j s_j \pmod{p}$$

La sicurezza deriva dal fatto che, matematicamente, per $t - 1$ punti passano infiniti polinomi di grado $t - 1$, ciascuno con un termine costante differente. Ciò rende il segreto perfettamente sicuro finché non si raggiunge la soglia critica t .

9.5.5 Threshold Signatures

Le firme a soglia estendono il concetto di *secret sharing* al processo di firma digitale, permettendo a un gruppo di partecipanti di autorizzare una transazione senza che nessuno di essi possieda l'intera chiave privata.

- La chiave privata x viene condivisa tra n partecipanti tramite *threshold secret sharing*.
- Una firma valida può essere generata solo con la collaborazione di almeno t partecipanti.
- **Sicurezza avanzata:** La chiave privata non viene mai ricostruita integralmente durante il processo di firma poiché i partecipanti generano "firme parziali" che vengono poi aggregate. Questo riduce drasticamente il rischio di esposizione.

L'integrazione con Bitcoin rappresenta una sfida specialmente per ECD-SA che non è nativamente lineare. Tuttavia, l'introduzione di **Schnorr Signatures** con l'aggiornamento *Taproot* offre proprietà matematiche molto più favorevoli, permettendo implementazioni di firme a soglia estremamente efficienti e indistinguibili da firme standard sulla blockchain.

Le Firme di Schnorr

L'algoritmo di firma di Schnorr è molto utile grazie alla sua proprietà di **linearità**. Mentre in ECDSA il processo di firma distribuita richiede protocolli complessi e pesanti, in Schnorr è possibile aggregare firme parziali con semplici operazioni aritmetiche.

Prima di analizzare il caso a soglia, vediamo come si compone una firma di Schnorr singola per un messaggio msg :

1. **Generazione del Nonce**: Si sceglie un valore casuale k e si calcola l'impegno pubblico $R = G^k$.
2. **Sfida (Challenge)**: Si calcola l'hash che lega il messaggio all'impegno: $e = \text{hash}(msg \parallel R)$.
3. **Risposta (Response)**: Si calcola $z = k + e \cdot x \pmod{q}$, dove x è la chiave privata.
4. **Verifica**: La firma (R, z) è valida se $G^z = R \cdot Y^e$ (dove Y è la chiave pubblica).

Schema a Soglia (t, n) Nel caso di una firma a soglia, la chiave privata x è divisa in share x_i tra n server. Per generare una firma valida, una sottoclasse di almeno t server deve collaborare attraverso tre fasi:

- **Fase 1 (Commitment Distribuito)**: Ogni server S_i sceglie un nonce locale k_i , calcola $R_i = G^{k_i}$ e lo trasmette agli altri. Al termine della fase, tutti i partecipanti calcolano l'impegno globale come somma dei singoli impegni:

$$R = \prod R_i = G^{\sum k_i}$$

- **Fase 2 (Firma Parziale)**: Ogni server calcola la sfida comune $e = \text{hash}(msg \parallel R)$ e produce una firma parziale z_i :

$$z_i = k_i + e \cdot \lambda_i \cdot x_i \pmod{q}$$

Dove λ_i è il coefficiente di Lagrange necessario per ricostruire il segreto dal subset di partecipanti.

- **Fase 3 (Aggregazione)**: Le firme parziali vengono sommate per ottenere la componente z finale della firma standard:

$$z = \sum z_i$$

Il risultato finale (R, z) è una firma di Schnorr perfettamente valida rispetto alla chiave pubblica totale Y . L'aspetto rivoluzionario è che, sulla blockchain, questa firma **appare identica** a una firma singola. Non vi è alcun modo per un osservatore esterno di capire se la firma sia stata generata da un singolo individuo o da un gruppo di t server, garantendo massima privacy ed efficienza (occupa sempre lo stesso spazio).

9.5.6 Tecniche di Mining

Il mining è un processo di ricerca esaustiva (**brute force**) di tipo stocastico. Grazie alle proprietà delle funzioni hash crittografiche (pseudo-casualità e resistenza alle pre-immagini), ogni tentativo è statisticamente indipendente dai precedenti. Questa caratteristica, definita *progress-free*, garantisce che un miner con l'1% della potenza di calcolo globale abbia sempre l'1% di probabilità di trovare il blocco successivo, indipendentemente da quanto tempo stia lavorando.

Tuttavia, il campo nonce nell'header del blocco è limitato a **32 bit** (circa 4,2 miliardi di combinazioni). Con gli attuali *hashrate*, un moderno ASIC esaurisce questo spazio in una frazione di secondo. Per continuare la ricerca senza attendere un nuovo blocco, i miner utilizzano la transazione **coinbase**:

- Modificando un campo arbitrario nella coinbase (chiamato *extraNonce*), cambia l'hash della transazione stessa.
- Questo cambiamento si propaga lungo l'albero di Merkle, modificando la **Merkle Root** nell'header del blocco.
- Una nuova Merkle Root genera un puzzle intrinsecamente diverso, fornendo altri 4,2 miliardi di nonce da testare.

9.5.6.1 Evoluzione dell'Hardware

L'architettura hardware è passata da soluzioni generiche a circuiti ultraspecializzati:

- **Mining con CPU:** L'approccio iniziale basato su processori *general-purpose*. Ormai obsoleto, poiché le CPU sono ottimizzate per istruzioni complesse e bassa latenza, non per l'esecuzione massiva di hash in parallelo.
- **Mining con GPU:** Le schede grafiche offrono migliaia di *Arithmetic Logic Units* (ALU) in parallelo. Questo passaggio ha permesso di aumentare il *throughput* da circa 20 MH/s a oltre 200 MH/s, essendo l'algoritmo SHA-256 facilmente parallelizzabile.

- **Mining con ASIC:** Gli *Application-Specific Integrated Circuits* sono chip progettati esclusivamente per calcolare hash SHA-256. Un singolo miner ASIC moderno può raggiungere i 100 TH/s (Tera-hash al secondo), con un'efficienza energetica (Watt/Terahash) irraggiungibile da qualsiasi hardware generico.

9.5.7 Mining Pool

Dato l'altissimo valore della difficoltà, la probabilità per un singolo miner (anche dotato di vari ASIC) di trovare un blocco è estremamente bassa, portando a una varianza dei guadagni insostenibile. Per risolvere questo problema, i miner si aggregano in **Mining Pool**, coordinati da un *Pool Operator*.

Il funzionamento si basa sui seguenti pilastri:

- **Riduzione della Varianza:** Combinando la potenza di calcolo, la pool trova blocchi con frequenza regolare, garantendo entrate costanti ai partecipanti.
- **Lavoro Distribuito:** Il server della pool assegna a ciascun partecipante un template di blocco con una transazione *coinbase* unica (legata all'indirizzo del miner o della pool) e uno specifico intervallo di *nonce* da esplorare. Ciò assicura che i partecipanti non sovrappongano il proprio lavoro.
- **Shares (Near-misses):** Per dimostrare il proprio lavoro (*Proof of Publication*), i miner inviano alla pool delle **shares**. Una share è una soluzione del puzzle che non soddisfa la difficoltà della rete, ma soddisfa una **difficoltà ridotta** stabilita dalla pool.
- **Schema di Ricompensa:** Quando la pool trova un blocco valido, il premio viene distribuito proporzionalmente al numero di shares inviate. I modelli comuni includono il *Pay Per Share* (PPS) o il *Pay Per Last N Shares* (PPLNS), detraendo una piccola commissione per l'operatore.

Capitolo 10

Smart Contract ed Ethereum

L'idea fondamentale degli **smart contract** (concetto introdotto da Nick Szabo negli anni '90) nasce dalla volontà di superare i limiti strutturali dei contratti tradizionali, spesso definiti in modo provocatorio *dumb contracts*. A differenza di un contratto legale che richiede la redazione di un avvocato, dipende dalla giurisdizione statale e necessita di interpretazione umana, uno smart contract è un protocollo informatico.

Esso presenta tre caratteristiche distintive:

- **Assenza di ambiguità:** Essendo scritto in codice deterministico, il contratto esegue esattamente ciò che è stato programmato, riducendo i margini di interpretazione.
- **Auto-esecuzione:** Non necessita di intermediari (come giudici o notai) per essere applicato; la piattaforma di esecuzione ne garantisce l'adempimento forzoso.
- **Trasparenza:** Il codice è pubblico, verificabile da chiunque e memorizzato in modo immutabile sulla blockchain.

10.1 Limitazioni di Bitcoin Script

Bitcoin possiede un linguaggio di scripting interno (**Script**), progettato però per essere intenzionalmente limitato per ragioni di sicurezza. I suoi principali vincoli sono:

- **Logica limitata:** Script non è *Turing-complete*. Supporta principalmente operazioni di accettazione o rifiuto di transazioni (*stateless logic*).

- **Assenza di loop:** Per evitare attacchi *Denial of Service* (DoS) basati su cicli infiniti, non sono ammesse istruzioni di iterazione.
- **Mancanza di stato:** Lo script non ha memoria persistente se non lo stack temporaneo di esecuzione; non può "ricordare" dati tra una transazione e l'altra.
- **Blockchain Blindness:** Lo script è "cieco" rispetto ai dati ambientali della blockchain (come timestamp o difficoltà), limitandosi a verificare le condizioni di sblocco degli UTXO.

Queste restrizioni, insieme all'uso di una **whitelist** di script standard accettati dai miner, rendono Bitcoin un sistema eccellente per il trasferimento di valore, ma inadatto a eseguire programmi complessi.

10.1.1 Il Caso Namecoin: Limiti della Logica di Bitcoin

Per comprendere appieno le limitazioni di Bitcoin Script, è utile analizzare il caso di **Namecoin**, il primo progetto che ha tentato di implementare un sistema DNS (Domain Name System) decentralizzato sulla blockchain. L'obiettivo era creare un mapping immutabile del tipo:

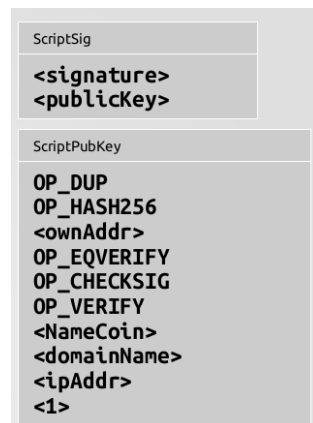
$$\text{Domain Name} \longrightarrow \text{IP Address}$$

Un sistema di questo tipo richiederebbe il supporto per tre operazioni fondamentali:

- `NameCoin.register(ownerAddr, domainName, initialVal)`: operazione per la registrazione iniziale di un dominio non ancora censito.
- `NameCoin.update(domainName, newVal, newOwner, ownerSig)`: operazione per aggiornare l'indirizzo IP associato o trasferire la proprietà del dominio. Quest'ultima richiede necessariamente la firma crittografica del proprietario attuale (*ownerSig*).
- `NameCoin.lookup(domainName)`: funzione di sola lettura, accessibile a chiunque, per risolvere il nome di dominio nel rispettivo indirizzo IP.

In linea teorica, si potrebbe tentare di implementare questa logica in Bitcoin estendendo la **ScriptPubKey** con nuovi comandi, salvando all'interno

dell'UTXO la coppia (dominio, IP) e vincolando la spesa alla verifica della chiave pubblica del proprietario.



Tuttavia, emerge un limite tecnico insormontabile: **Bitcoin Script non può verificare se un dominio sia già stato registrato**. Poiché ogni script viene eseguito in isolamento e ha accesso solo ai dati della transazione corrente (proprietà *stateless*), non può interrogare l'intero set di UTXO per convalidare la policy di unicità del nome (fondamentale per un DNS).

A causa di questa incapacità di gestire uno stato globale condiviso, l'idea non ha potuto trovare spazio nel protocollo Bitcoin originale. Namecoin è stato quindi costretto a implementare un **hard fork**, dando vita a una nuova criptovaluta indipendente. Questo ha comportato la necessità di un nuovo processo di *bootstrap* per creare una community e attirare potenza di calcolo dedicata, frammentando di fatto l'ecosistema invece di integrarsi in esso.

10.2 La Piattaforma Ethereum

Ethereum nasce con l'obiettivo di superare queste barriere, fornendo uno strato computazionale universale sopra la blockchain. Questo ha permesso la nascita di ecosistemi complessi come gli **NFT**, la finanza decentralizzata (**DeFi**), il **gaming on-chain** e le organizzazioni autonome (**DAO**).

A differenza di Bitcoin, basato sul modello UTXO (un insieme di "monete" non spese), Ethereum utilizza un **Account-based model**. La blockchain non è solo una lista di transazioni, ma una **transizione di stati** che modifica costantemente il **World State** (lo stato globale del network).

10.2.1 L'unità monetaria: Ether

Il valore all'interno della rete è denominato in **Ether** (ETH). Sebbene l'utente medio ragioni in ETH, internamente il sistema opera esclusivamente in **Wei**, la minima unità divisibile:

- 1 Ether = 10^{18} Wei
- 1 Finney = 10^{15} Wei (10^{-3} ETH)
- 1 Szabo = 10^{12} Wei (10^{-6} ETH)

10.2.2 Consenso: dal PoW al Proof of Stake

Dalla genesi fino a settembre 2022 (evento noto come **The Merge**), Ethereum ha utilizzato la Proof of Work (basata sull'algoritmo *Ethash*). Per prevenire il dominio degli ASIC, l'algoritmo era stato progettato per essere *memory-hard*, favorendo il mining tramite GPU.

Il passaggio alla **Proof of Stake** era previsto fin dal design iniziale. Per forzare la migrazione e disincentivare i miner a restare sulla vecchia catena, è stata introdotta la **Difficulty Bomb** un meccanismo che aumenta esponenzialmente la difficoltà del puzzle crittografico fino a rendere impossibile la produzione di blocchi, rendendo la rete PoW di fatto inutilizzabile e accelerando la transizione verso il nuovo modello basato sullo *staking*.

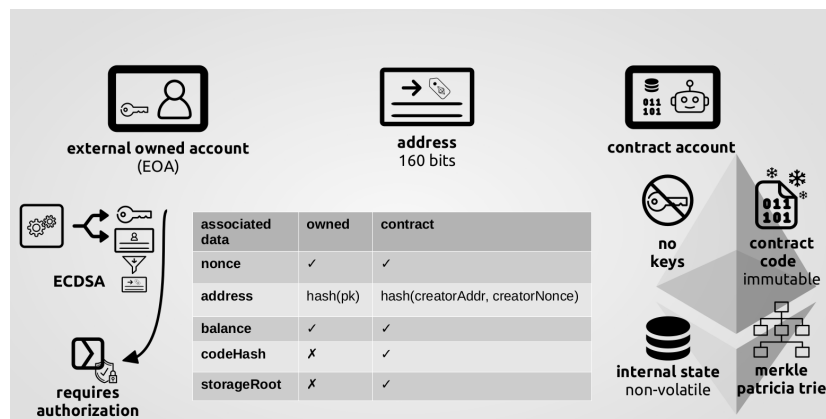
A differenza di Bitcoin, dove il tempo tra i blocchi è di 10 minuti, Ethereum produce un blocco (o *slot*) ogni 12 secondi, garantendo una finalità delle transazioni molto più rapida. Inoltre, l'emissione di ETH non ha un tetto massimo fisso di 21 milioni, ma è regolata da meccanismi di *burn* e ricompense per i validatori che bilanciano inflazione e deflazione.

10.2.3 Tipologie di Account

In Ethereum esistono due tipi di account, entrambi identificati da un indirizzo e dotati di un bilancio in Ether:

- **Externally Owned Accounts (EOA)**: Controllati da utenti umani tramite chiavi private e firme ECDSA. Possono inviare transazioni per trasferire ETH o interagire con i contratti.
- **Contract Accounts**: Account governati esclusivamente dal loro codice (**EVM Bytecode**). Non possiedono chiavi private e le loro azioni sono innescate solo se ricevono una transazione da un EOA o da un altro contratto. Il codice di un contratto è **immutabile**; per aggiornarlo, gli

sviluppatori utilizzano spesso il pattern *Proxy*, che permette di puntare l'indirizzo del contratto a una nuova logica in modo trasparente.



Analizziamo adesso uno per uno i dati degli account:

- **Nonce:** La nonce negli account rappresenta un numero che serve a prevenire gli attacchi di replay. La firma di ogni transazione, infatti, oltre che i dati contiene anche questa nonce che, subito dopo la firma, viene aggiornata di uno. La firma viene considerata valida solo se la nonce nella transazione corrisponde a quelle dell'account segnata in quel momento nel world state. In Bitcoin ricordiamo che non è effettivamente presente una nonce per prevenire attacchi di replay, tuttavia, gli uTXO se vogliamo possono fungere come una sorta di Nonce perché sono spendibili solo una volta. Nei contratti, la nonce serve a contare quanti contratti sono stati creati a partire da quel contratto ma questa funzione la vedremo dopo nel dettaglio.
- **Indirizzo:** Conosciamo già l'importanza degli indirizzi nel sistema blockchain per riferirci ad uno specifico account. Per quanto riguarda gli EOA, questo è banalmente l'hash della chiave pubblica mentre, per quanto riguarda i contratti questo è l'hash dell'indirizzo del creatore insieme alla sua nonce. Ricordiamo che per creatore di un contratto possiamo anche avere altri contratti.
- **Balance:** Come abbiamo già detto, sia gli EOA che i contratti possono possedere un bilancio per effettuare transazioni o attivare altri contratti.
- **CodeHash:** Il CodeHash è un campo posseduto solo dai contratti. Poiché il codice del contratto è salvato nella blockchain, che possiamo

immaginare come una sorta di database, possiamo dire che il CodeHash funge da ID per cercare e trovare velocemente il codice.

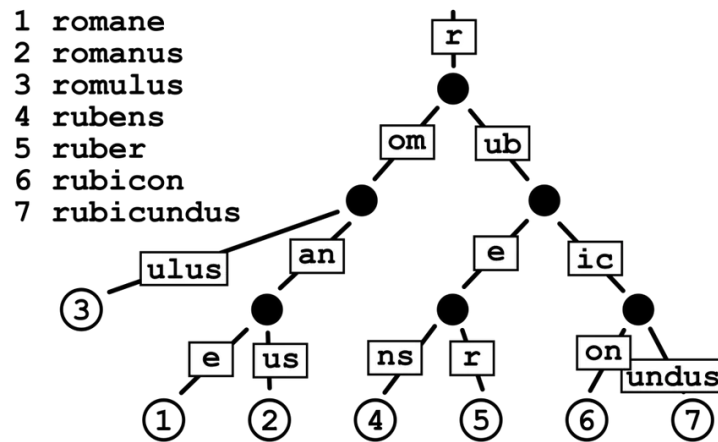
- **StorageRoot**: Ogni contratto possiede una certa memoria che gli servirà per mantenere al suo interno uno stato non volatile. Questa è memorizzata sotto forma di **Merkle Patricia Trie** che descriveremo tra poco. Nello specifico, anche in questo caso lo StorageRoot è la radice di questo albero che servirà per effettuare prove di appartenenza.

10.2.4 Merkle Patricia Trie (MPT)

I **Merkle Patricia Trie** rappresentano la struttura dati fondamentale su cui poggia l'architettura di Ethereum, utilizzati sia per la gestione del **World State** (lo stato globale di tutti gli account) che per lo **storage dei singoli contratti**.

Essi nascono come un'integrazione tra i *Patricia Trie* (noti anche come *Radix Tree*) e la logica crittografica dei *Merkle Tree*:

- **Struttura Patricia (Radix Tree)**: Implementano una mappa $key \rightarrow value$ dove la chiave (solitamente una stringa esadecimale derivata da un hash) viene utilizzata per navigare l'albero. Il vantaggio principale è la condivisione dei prefissi: chiavi simili condividono lo stesso percorso iniziale, riducendo drasticamente la ridondanza e lo spazio di archiviazione. Se una chiave non è presente, il percorso si interrompe restituendo un valore nullo.
- **Componente Merkle**: Ogni nodo dell'albero è identificato dall'hash dei suoi contenuti (e dei puntatori ai suoi figli). Questo trasforma l'intera struttura in un **commitment crittografico**: la radice (*Root Hash*) è un'impronta digitale unica di tutto lo stato. Ciò permette di generare **Proof of Membership** (o prove di Merkle), consentendo ai nodi leggeri di verificare l'esistenza e il valore di un dato senza possedere l'intero database.



A differenza di un semplice Merkle Tree statico (come quello usato nei blocchi di Bitcoin), l'MPT è una **struttura dinamica**. È progettato per gestire inserimenti, modifiche e cancellazioni frequenti in modo efficiente:

- Quando un valore viene aggiornato (ad esempio il bilancio di un account), è necessario ricalcolare solo gli hash dei nodi lungo il percorso che va dalla foglia modificata fino alla radice.
- In un albero bilanciato, la complessità di queste operazioni è logaritmica rispetto al numero di elementi, garantendo prestazioni elevate anche con milioni di account.

10.2.5 World State

Il **World State** rappresenta lo stato globale e aggiornato dell'intero ecosistema Ethereum. Tecnicamente, è implementato come un **Merkle Patricia Trie** (MPT) la cui radice (*State Root*) è inclusa in ogni blocco della blockchain.

Questo archivio può essere visualizzato come un database chiave-valore dove:

- **Chiave:** L'indirizzo esadecimale di 20 byte dell'account (previa applicazione di un hash per bilanciare l'albero).
- **Valore:** Lo stato dell'account serializzato, che comprende *nonce*, *balance*, *storageRoot* e *codeHash*.

Grazie alla struttura MPT, è possibile fornire una **Proof of Membership** per dimostrare l'esistenza e il saldo di un account senza dover scansionare l'intera rete.

Per ogni contratto esiste un ulteriore **Storage Trie**. Qui vengono mappate chiavi di 256 bit (derivanti spesso dalla posizione o dal nome della variabile nel codice Solidity) in valori di 256 bit.

Se una variabile viene dichiarata ma mai inizializzata, o se viene esplicitamente posta uguale a 0, l'MPT non memorizza alcun nodo per quella chiave. Di conseguenza, una ricerca per una chiave inesistente restituisce sempre 0, permettendo un risparmio considerevole di spazio poiché si memorizzano solo i valori non nulli.

Per poter inserire un oggetto complesso come un account (composto da 4 campi distinti) all'interno di un nodo dell'albero di Merkle, è necessario prima trasformarlo in una sequenza di byte "piatta". Ethereum utilizza a questo scopo lo standard **RLP**.

L'RLP è il sistema di serializzazione primario di Ethereum che serve a codificare strutture dati (liste, stringhe, interi) in un formato binario compatto.

10.2.6 Transazioni

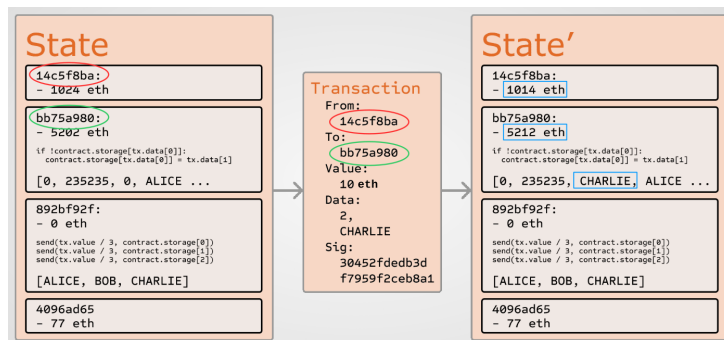
In Ethereum esistono le classiche transazioni che abbiamo già visto in Bitcoin che possono essere effettuate solo da EOA. Queste contengono al loro interno: Ogni transazione contiene i seguenti campi fondamentali:

- **Nonce**: Un contatore che indica quante transazioni sono state inviate dall'indirizzo mittente. Serve a garantire l'ordine di esecuzione e a prevenire attacchi di *replay*.
- **Recipient (to)**: L'indirizzo di 20 byte del destinatario. Se il campo è vuoto (nullo), la transazione viene interpretata come una creazione di contratto.
- **Value**: L'ammontare di Ether (espresso in *Wei*) da trasferire dal mittente al destinatario.
- **Gas Limit**: La quantità massima di unità di gas che il mittente è disposto a consumare per l'esecuzione.
- **Gas Price**: Il prezzo (in Wei) che il mittente è disposto a pagare per ogni unità di gas consumata.
- **Data (o Input)**: Un campo di lunghezza variabile che contiene i dati necessari per l'attivazione di funzioni nei contratti o il codice di inizializzazione per i nuovi contratti.

- **Firma ECDSA (v, r, s):** Ethereum utilizza una variante di ECDSA chiamata **public key recovery**. Invece di includere esplicitamente il campo *from* (mittente), il protocollo estrae la chiave pubblica (e quindi l'indirizzo) direttamente dalla firma e dal messaggio transazionale, ottimizzando lo spazio nella blockchain.

Queste transazioni possono essere di vari tipi:

- **Currency Transfer:** Questa è la classica transazione in cui un EOA invia dei soldi ad un altro EOA.
- **Contract Activation:** Tramite questo tipo di transazione è possibile attivare una funzione di un contratto. I contratti, infatti, metteranno a disposizione proprio delle funzioni che potranno prendere in input anche dei dati sul quale lavorare.
- **Contract Creation:** Questa transazione viene fatta verso un indirizzo nullo con una certa semantica e servirà alla generazione di un nuovo contratto. Tra questi valori sarà presente l'init che sarà un codice che, una volta eseguito, produrrà il codice reale del contratto.



10.2.7 Messaggi

Mentre le transazioni sono originate esclusivamente da account esterni (EOA), i contratti interagiscono tra loro attraverso i **messaggi**, spesso definiti **transazioni interne**.

Sebbene un messaggio sia concettualmente simile a una transazione (possiede un mittente, un destinatario, un valore e dei dati), presenta una differenza fondamentale: **non richiede una firma digitale**.

La sicurezza e l'autorizzazione nei messaggi non derivano dalla crittografia a chiave pubblica, ma dalla logica deterministica della *Ethereum Virtual Machine* (EVM):

- **Catena di Fiducia:** Ogni messaggio è l'effetto di una transazione originale firmata da un EOA. Se l'utente *A* firma una transazione per il Contratto *B*, e il codice di *B* prevede l'invio di un messaggio al Contratto *C*, l'intero processo è autorizzato dalla firma iniziale di *A*.
- **Assenza di Chiavi:** I contratti non possiedono una chiave privata. La loro "volontà" è espressa esclusivamente dal codice immutabile memorizzato sulla blockchain. È il protocollo stesso a garantire che il campo `msg.sender` (il mittente del messaggio) corrisponda effettivamente all'indirizzo del contratto che ha generato la chiamata.

Esattamente come per le transazioni, possiamo distinguere tre operazioni principali eseguite tramite messaggi:

- **Currency Transfer:** Un contratto trasferisce una parte del proprio bilancio in Ether a un EOA o a un altro contratto (solitamente tramite le istruzioni `transfer` o `call`).
- **Contract Activation (Message Call):** Un contratto chiama una funzione di un altro contratto, passando dei parametri e attendendo eventualmente un valore di ritorno.
- **Contract Creation:** Un contratto genera un nuovo contratto "figlio" utilizzando l'istruzione `CREATE` o `CREATE2`. In questo caso, l'indirizzo del nuovo contratto sarà derivato dall'indirizzo del contratto "padre" e dalla sua *nonce*.

10.2.8 Block Building

Il processo di creazione e validazione dei blocchi di Ethereum rappresenta una delle più grandi differenze rispetto al sistema Bitcoin. Ethereum, infatti, dopo un periodo iniziale basato sulla Proof-of-Work, ha abbandonato questo meccanismo per passare a un sistema chiamato Proof-of-Stake (PoS).

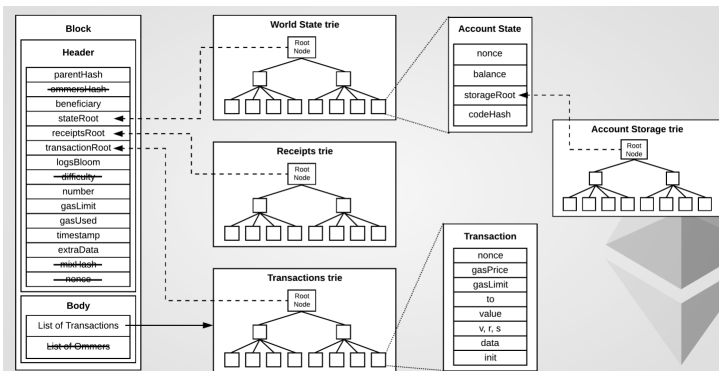
L'idea di fondo è quella di evitare l'eccessivo consumo elettrico causato dalla risoluzione dell'hash puzzle; al suo posto, viene utilizzato un sistema in cui i partecipanti mettono a disposizione il cosiddetto stake, ovvero una quota dei propri possedimenti in criptovaluta (Ether) bloccata come deposito di garanzia. Un validatore che propone o attesta un blocco non valido, o che tenta di frodare la rete, viene penalizzato attraverso il meccanismo di **slashing**, che comporta la distruzione di una parte o della totalità del patrimonio depositato. Il processo di costruzione e validazione di un blocco avviene come segue:

1. Raccolta delle transazioni: Il validatore selezionato per il turno corrente (chiamato Block Proposer) raccoglie una serie di transazioni $TX_0, \dots, TX_{n-1}$ dalla mempool (la vasca di memoria in cui sostano le transazioni in attesa).
2. Esecuzione e calcolo dello stato: Vengono eseguite le transazioni all'interno della Ethereum Virtual Machine (EVM), modificando i saldi dei conti ed eseguendo il codice degli smart contract coinvolti. Vengono calcolate le commissioni e, infine, viene generato il nuovo stato globale della rete, noto come world state.
3. Propagazione e validazione (Attestazione): Il blocco appena creato viene propagato nella rete. Un comitato di altri validatori (chiamati Attesters) verifica l'integrità e la correttezza del blocco. Se il blocco è valido, i validatori inviano la propria approvazione (attestation). Nel caso in cui un validatore dovesse attestare come valido un blocco che non lo è, verrebbe penalizzato con la perdita del proprio deposito.

Per poter essere scelti, i validatori devono aver depositato (messo in stake) esattamente 32 ETH. Più nodi validatori un utente possiede (ognuno con 32 ETH), maggiori sono le sue probabilità complessive di essere estratto. All'inizio di ogni epoca, il protocollo utilizza l'algoritmo RANDAO per generare un numero casuale condiviso e sicuro. Questo numero serve a estrarre a sorte, dall'intero pool globale, i 32 Block Proposers (uno per ogni slot dell'imminente epoca) e a formare i comitati di validatori (Attesters) che dovranno votare quei blocchi.

10.2.8.1 Struttura dei Blocchi

La struttura dei blocchi Ethereum codifica tutte le informazioni necessarie per la validazione e l'incorporazione di questi nella blockchain.



Un blocco è composto principalmente da due componenti:

- **Header:** Questo contiene i metadati essenziali a rappresentare i vari stati. Tra questi troviamo:
 - **parentHash:** L'hash Keccak a 256 bit dell'intero header del blocco precedente. È il collegamento fondamentale che incatena i blocchi tra loro garantendo l'immutabilità della blockchain.
 - **beneficiary:** L'indirizzo a 160 bit (l'account) a cui vengono inviate tutte le commissioni di transazione (i tips o mancie del gas) raccolte dall'esecuzione di questo blocco.
 - **stateRoot:** L'hash della radice del Merkle Patricia Trie dello stato globale (il world state). Rappresenta lo "scatto" di tutti i saldi e i dati dei contratti della rete dopo aver applicato le transazioni del blocco.
 - **receiptsRoot:** L'hash della radice del Merkle Patricia Trie che contiene le ricevute (receipts) di tutte le transazioni incluse in questo blocco. Serve a verificare rapidamente l'esito e i log prodotti dalle transazioni.
 - **transactionRoot:** L'hash della radice del Merkle Patricia Trie contenente tutte le transazioni incluse all'interno di questo specifico blocco.
 - **logsBloom:** Un filtro di Bloom (una struttura dati probabilistica) composto dai log delle transazioni. Permette ai client leggeri di verificare molto velocemente se un determinato log o evento è presente nel blocco senza dover scaricare l'intero stato.
 - **number:** Un valore numerico che indica l'altezza del blocco (il numero progressivo di blocchi generati a partire dal blocco genesis, che ha valore 0).
 - **gasLimit:** La quantità massima di gas che può essere consumata complessivamente dalle transazioni all'interno di questo blocco. Determina, di fatto, la dimensione massima computazionale del blocco stesso.
 - **gasUsed:** La quantità totale ed effettiva di gas che è stata consumata per eseguire tutte le transazioni contenute nel blocco.
 - **timestamp:** Il momento esatto in cui il blocco è stato convalidato, espresso in secondi secondo lo standard Unix (Unix epoch). Post-Merge, coincide precisamente con l'inizio dello slot temporale.

- **extraData**: Un campo di byte libero e opzionale che il validatore può utilizzare per inserire informazioni arbitrarie (ad esempio note testuali o firme), entro un limite prefissato di dimensioni.

I campi sbarrati nell'immagine (`ommersHash` e `difficulty`) sono stati rimossi o azzerati a seguito del passaggio di Ethereum alla Proof-of-Stake (The Merge), in quanto legati al vecchio sistema basato sulla Proof-of-Work.

- **Body**: Contiene la lista effettiva di tutte le transazioni del blocco mostrate sotto forma di **Transactions Trie** che non è altro che un altro Merkle Patricia Trie.

Notiamo come la struttura sia piena di Merkle Patricia Trie salvati nel blocco come radici. Troviamo:

- **World State Trie**: La mappa che memorizza tutti gli stati degli account.
- **Transaction Trie**: La mappa di tutte le transazioni incluse nel blocco.
- **Receipts Trie**: Memorizza le ricevute delle esecuzioni dei contratti.
- **Storage Trie**: Memorizza per ciascun contratto il suo stato interno detto **storage** che sarebbe la sua memoria non volatile.

Il fatto che sia il tutto salvato sotto forma di root all'interno dell'header permette anche ai client leggeri di poterli salvare senza un eccessivo consumo di risorse.

10.2.9 Il sistema di Gas

Ethereum è dotato di un linguaggio di programmazione che, a differenza di quello di Bitcoin, è **Turing-completo** e viene eseguito sulla **Ethereum Virtual Machine (EVM)**.

Il codice di ogni smart contract deve essere eseguito in modo deterministico da ogni validatore della rete. Tuttavia, trattandosi di un linguaggio Turing-completo che supporta cicli e iterazioni, **non è possibile prevedere a priori se e quando l'esecuzione terminerà** a causa del noto **problema della fermata** (*halting problem*). Di conseguenza, senza un meccanismo di controllo, la rete sarebbe vulnerabile ad attacchi di tipo Denial of Service (DoS), in cui gli attaccanti potrebbero pubblicare contratti contenenti loop infiniti col fine di bloccare indefinitamente i validatori.

Poiché il protocollo richiede la produzione di un nuovo blocco ogni 12 secondi, sorge la necessità di porre un limite stringente alle computazioni eccessivamente lunghe o infinite. Il meccanismo del Gas rappresenta la soluzione a questo problema:

- **Consumo computazionale:** Ogni singola istruzione (*opcode*) eseguita dall'EVM consuma una quantità prestabilita di Gas, proporzionale alle risorse computazionali e di memoria richieste. Esiste una netta differenza di costo tra l'uso della memoria volatile (*Memory* o *Stack*) e quella non volatile (*Storage*). Quest'ultima ha un costo sensibilmente superiore in quanto i dati modificati persistono nel **world state** e devono essere memorizzati permanentemente nei nodi della rete.
- **Pagamento anticipato:** Il Gas viene pagato in anticipo dall'utente che avvia la transazione.
- **Incentivo per i validatori:** Il Gas speso per l'esecuzione del codice viene assegnato al *Block Proposer*, ovvero al validatore che seleziona la transazione e propone il blocco. Sebbene anche tutti gli altri validatori della rete debbano rieseguire la transazione per verificarne la correttezza, la mancia spetta unicamente a chi ha composto il blocco.

Questo meccanismo di mercato disincentiva gli attacchi DoS e le computazioni inefficienti, poiché l'esecuzione di cicli infiniti o di codice eccessivamente gravoso richiederebbe una disponibilità economica teoricamente infinita e praticamente insostenibile.

10.2.9.1 Meccanismo Legacy

Prima dell'aggiornamento **London** (avvenuto a metà del 2021), Ethereum utilizzava un modello ad asta per gestire il calcolo e il pagamento delle commissioni delle transazioni.

Questo modello si basava su due parametri fondamentali inclusi dal mittente in ogni transazione:

- **GasLimit:** La quantità massima di unità di Gas che l'utente è disposto a consumare per l'esecuzione della transazione. Rappresenta un tetto di sicurezza per evitare consumi incontrollati.
- **GasPrice:** Il prezzo in *wei* che il mittente è disposto a pagare per singola unità di Gas. Questo valore determinava la priorità della transazione all'interno della *mempool*: i validatori, spinti da un incentivo

economico, selezionavano prioritariamente le transazioni con il *GasPrice* più alto. Il mercato stabiliva dinamicamente il prezzo ottimale attraverso meccanismi d'asta, supportato da servizi esterni (come *eth-gasstation.info*) che fornivano stime in tempo reale per diverse fasce di priorità.

Il Gas stanziato veniva consumato da ogni singola operazione eseguita dalla EVM secondo un rigido "listino prezzi" prefissato dal protocollo. Al termine dell'esecuzione si potevano verificare due scenari:

1. **Esecuzione completata con successo:** Se il Gas effettivamente consumato era inferior al *GasLimit*, il Gas residuo veniva interamente rimborsato al mittente al prezzo specificato nel *GasPrice*. La spesa effettiva era pari a:

$$\text{Spesa} = \text{Gas Consumato} \times \text{GasPrice}$$

2. **Esaurimento del Gas (*Out of Gas*):** Se la computazione richiedeva più Gas di quello stanziato nel *GasLimit*, l'esecuzione si interrompeva immediatamente. In questo caso, l'intero ammontare di Gas pari al *GasLimit* veniva comunque trattenuto dal validatore per ricompensarlo del lavoro computazionale svolto fino a quel momento. Tuttavia, per garantire la coerenza della blockchain, la transazione falliva e la EVM effettuava un **ripristino dello stato** (*state rollback*), annullando tutti gli effetti sui saldi e sugli smart contract e riportando il *world state* alla condizione precedente all'esecuzione.

Nell'ambiente di esecuzione di Ethereum, le anomalie e i fallimenti si dividono fondamentalmente in due categorie:

- **Errori Critici (Eccezioni a livello EVM):** Errori causati da eccezioni non gestibili, come l'esaurimento del gas (*Out of Gas*), l'overflow dello stack o l'esecuzione di istruzioni non valide (istruzione `INVALID`). Provocano la terminazione immediata della transazione, il rollback completo dello stato e il consumo totale del gas fornito.
- **Errori Gestiti (Eccezioni a livello applicativo):** Errori controllati nel codice dello smart contract (tramite le funzioni Solidity `require`, `revert` o `assert`). Sebbene comportino comunque il fallimento della transazione e il ripristino dello stato per motivi di sicurezza, l'istruzione `REVERT` permette di interrompere l'esecuzione restituendo al mittente il gas non ancora consumato fino a quel momento, oltre a un eventuale messaggio di errore.

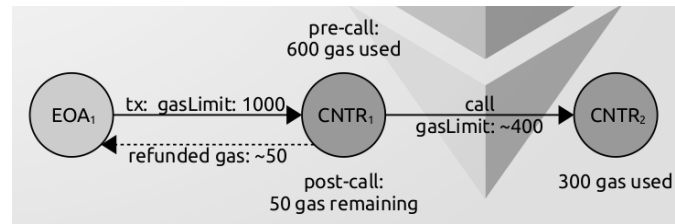


Figura 10.1: Flusso e passaggio del Gas nelle chiamate interne tra smart contract.

La Figura 10.1 mostra un esempio di flusso del Gas durante una chiamata inter-contratto (*internal call*). Una transazione ha inizio con un *GasLimit* iniziale pari a 1000 unità. Nella fase di pre-chiamata (all'interno del primo contratto) vengono consumate 600 unità. Al momento di invocare il secondo contratto, il contratto chiamante inoltra una porzione del Gas residuo (circa 400 unità). Il contratto chiamato esegue le proprie operazioni consumandone 350 e restituisce le 50 unità inutilizzate al mittente.

10.2.9.2 Meccanismo EIP-1559

Come accennato, a seguito dell'aggiornamento *London* a metà del 2021, il modello di mercato basato esclusivamente sulle aste è stato sostituito dall'introduzione della proposta di miglioramento **EIP-1559**. Il nuovo sistema si basa principalmente su tre parametri fondamentali all'interno di ogni transazione:

- **Base Fee:** È la commissione minima obbligatoria per unità di gas richiesta dal protocollo per includere una transazione nel blocco. A differenza del modello legacy, la *Base Fee* non viene assegnata ai validatori, ma viene stabilita algebricamente dal protocollo e interamente **bruciata** (*burned*), rimuovendo l'Ether dalla circolazione e introducendo una dinamica deflazionistica. Questo valore si modifica dinamicamente da un blocco all'altro per adattarsi alla congestione della rete: l'obiettivo del protocollo è un consumo di gas pari a 15 milioni di unità per blocco (il target). Se un blocco consuma più del target (fino a un massimo assoluto di 30 milioni), la *Base Fee* del blocco successivo aumenta (fino a un massimo del +12.5%); se consuma meno, diminuisce.
- **Priority Fee (Mancia):** È una commissione addizionale per unità di gas specificata opzionalmente dall'utente. Questa somma viene pagata direttamente al *Block Proposer* come incentivo economico per convincerlo a dare priorità alla transazione e includerla all'interno del blocco corrente, specialmente in periodi di alta congestione.

- **Max Fee:** Rappresenta il tetto massimo assoluto per unità di gas che il mittente è finanziariamente disposto a spendere per la transazione. Poiché la *Base Fee* varia in modo fluttuante tra un blocco e l'altro, questo valore funge da protezione contro picchi improvvisi di costo.

Il prezzo effettivo per unità di gas pagato dall'utente alla fine dell'esecuzione viene determinato dalla formula:

$$\text{Prezzo Effettivo} = \min(\text{Base Fee} + \text{Priority Fee}, \text{Max Fee})$$

Se la somma tra *Base Fee* e *Priority Fee* risulta inferiore alla *Max Fee*, la differenza viene integralmente rimborsato all'utente. Al contrario, se la rete subisce un improvviso picco di congestione e la *Base Fee* sale a tal punto da far superare la *Max Fee* impostata, la transazione rimane in attesa nella *mempool* poiché il wallet riduce la mancia effettiva (*Priority Fee*) fino a zero pur di non sfiorare il tetto massimo richiesto dall'utente.

Grazie a questo impianto algoritmico, l'EIP-1559 migliora drasticamente la prevedibilità del sistema per gli utenti. I wallet non devono più basarsi su stime probabilistiche complesse in un mercato ad asta volatile, ma conoscono a priori la *Base Fee* esatta del blocco corrente, riducendo i casi di transazioni pagate eccessivamente o rimaste bloccate a causa di commissioni insufficienti.